

UNIVERSITÀ
POLITECNICA
DELLE MARCHE

Corso di «Ingegneria del Software»

A.A. 2024/2025

Parte VI: I Pattern

Prof. Domenico Ursino

d.ursino@univpm.it

Introduzione

- Progettare applicazioni basate sul paradigma object-oriented non è affatto banale; riuscire a renderle anche riusabili ed estendibili a piacimento è ancora più arduo.
- Riuscire a definire al primo tentativo una struttura ad oggetti corretta, riusabile e flessibile è un'impresa quasi impossibile, soprattutto per applicazioni particolarmente complesse.
- In genere, durante la sua realizzazione, un'applicazione subisce innumerevoli modifiche.
- La mancanza di tempo porta spesso a scegliere soluzioni molto focalizzate sul modello reale attuale e poco orientate ad adattarsi ai cambiamenti futuri.
- In questo scenario diventa importante saper individuare delle soluzioni che siano riutilizzabili più volte.
- Per minimizzare il lavoro da svolgere, gli sviluppatori meno esperti solitamente tendono a ricorrere a tecniche "copia e incolla", con il risultato di duplicare parti di codice e di introdurre accoppiamenti e dipendenze tra gli oggetti.
- Gli architetti e gli sviluppatori con più esperienza tendono a preferire le soluzioni object-oriented che in passato si sono rivelate vincenti ed efficaci.
- Queste soluzioni, che in prima analisi possiamo definire pattern, sono orientate a risolvere particolari problematiche di progettazione e consentono grande flessibilità.

Classificazione dei pattern

- Esistono diverse tipologie di pattern in funzione della loro area di applicazione.
- In generale, essi possono essere raggruppati in macrocategorie specifiche, dette anche cluster, ciascuna delle quali contenente pattern orientati a risolvere problematiche similari.
- Per un pattern è possibile considerare come livello distintivo anche il livello di astrazione che lo contraddistingue.
- Possiamo individuare tre categorie di pattern caratterizzate da un diverso livello di astrazione:
  - Pattern architetturali: descrivono lo schema organizzativo della struttura che caratterizza un sistema software.
    - Un esempio significativo di pattern architetturale è rappresentato dal layering, che descrive come suddividere un'applicazione in strati logici sovrapposti e tra loro comunicanti.
    - I pattern architetturali operano ad un livello più ampio rispetto ai design pattern.
  - Pattern di disegno (design pattern): sono i pattern che si riferiscono alle problematiche legate al disegno object-oriented.

Classificazione dei pattern

- Pattern di implementazione (idiomi): sono pattern di basso livello specifici per una particolare tecnologia.
    - Descrivono le modalità implementative da utilizzare per risolvere problematiche di sviluppo sfruttando le peculiarità di una piattaforma.

- Ciascuno di questi gruppi è caratterizzato da un grado di astrazione differente.

- I design pattern si collocano tra i pattern architetturali, molto generici, e i pattern idiomatici, molto legati alla tecnologia.

Pattern architetturali - Introduzione

- Un pattern architetturale può essere immaginato come una descrizione stilizzata di una buona pratica, che è stata provata e verificata in vari sistemi e ambienti.
- Pertanto, un pattern architetturale dovrebbe descrivere l'organizzazione di un sistema che ha avuto successo in altri sistemi preesistenti.
- Dovrebbe includere le informazioni su quando il suo utilizzo è appropriato e i dettagli sui suoi vantaggi e svantaggi.
- I più importanti pattern architetturali, che vedremo nel seguito, sono:
  - Model-View-Controller.
  - Layered Architecture.
  - DAO (Data Access Object).
  - Repository.
  - Client-Server.
  - Pipe-and-Filter.

Pattern architetturali – Pattern Model-View-Controller

- La seguente figura descrive il noto pattern Model-View-Controller (MVC), che è la base per la gestione delle interazioni in molti sistemi web ed è supportato da molti linguaggi.

Nome	**MVC (Model-View-Controller)**
Descrizione	Presentazione e interazione separate dai dati del sistema. Il sistema è strutturato in tre componenti logiche che interagiscono tra loro. Il componente Model gestisce i dati del sistema e le operazioni associate. Il componente View definisce e gestisce il modo in cui i dati sono presentati all'utente. Il componente Controller gestisce l'interazione degli utenti (tasti premuti, click del mouse, etc.) e passa queste interazioni ai componenti Model e View. Si veda la figura seguente.
Esempio	Un esempio di pattern MVC può riguardare un'applicazione per la gestione di un orologio. Il model gestisce il passare del tempo. Esistono più coppie view-controller; ad esempio, la prima coppia gestisce una rappresentazione digitale dell'ora, la seconda coppia gestisce, invece, una rappresentazione basata sulle lancette.
Quando si usa	Si usa quando ci sono più modi di visualizzare e interagire con i dati; si usa anche quando non sono noti i requisiti futuri di interazione e presentazione dei dati.
Vantaggi	Consente ai dati di cambiare indipendentemente dalla loro rappresentazione, e viceversa. Supporta la presentazione degli stessi dati in modi differenti, con le modifiche fatte in una rappresentazione mostrate in tutte le altre.
Svantaggi	Potrebbe richiedere altro codice o un codice complesso quando il modello dei dati e le interazioni sono semplici.

Pattern architetturali – Pattern Model-View-Controller

- Il pattern Model-View-Controller si basa sul principio di separazione delle responsabilità.
- Esso separa il codice in tre classi distinte:
  - Model: questa classe si occupa solo dello stato e della logica dell'applicazione.
  - View: questa classe si occupa solo della gestione dell'interfaccia utente in risposta ad aggiornamenti generici che riceve dal model.
  - Controller: questa classe si occupa soltanto di tradurre l'input dell'utente in aggiornamenti che vengono poi passati al model.

Pattern architetturali – Pattern Model-View-Controller

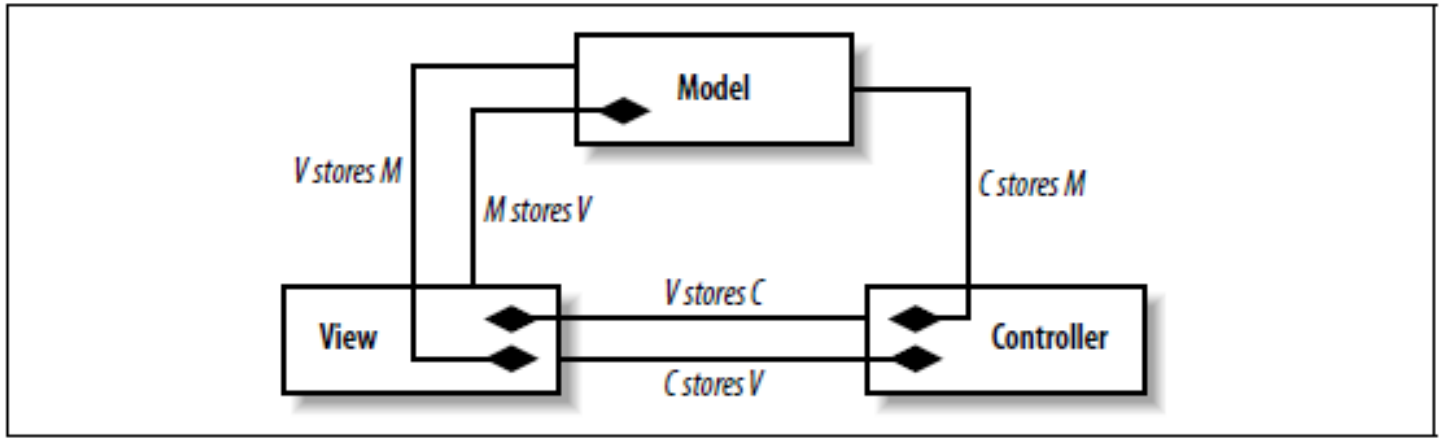
- Separare il codice che governa un'interfaccia utente nelle classi model, view e controller comporta i seguenti benefici:
  - Consente delle rappresentazioni multiple della stessa informazione.
  - Consente di aggiungere, rimuovere e cambiare facilmente le interfacce utente, sia a tempo di compilazione che a tempo di esecuzione.
  - Consente di cambiare facilmente le risposte agli input dell'utente, sia a tempo di compilazione che a tempo di esecuzione.
  - Promuove il riuso.
  - Consente a più sviluppatori di aggiornare simultaneamente l'interfaccia, la logica o l'input di un'applicazione senza influenzare il resto del codice sorgente.
  - Consente agli sviluppatori di focalizzarsi su un singolo aspetto alla volta dell'applicazione.

Pattern architetturali – Pattern Model-View-Controller

- Sebbene le classi model, view e controller in MVC sono intenzionalmente separate, esse devono comunicare regolarmente.

- Per facilitare questa comunicazione ciascun oggetto in MVC deve memorizzare un riferimento agli altri oggetti con cui interagisce.

- La figura seguente mostra come gli oggetti in MVC si riferiscono l'un l'altro; il rombo rappresenta una relazione di composizione tramite la quale un oggetto memorizza un'istanza di un altro oggetto:

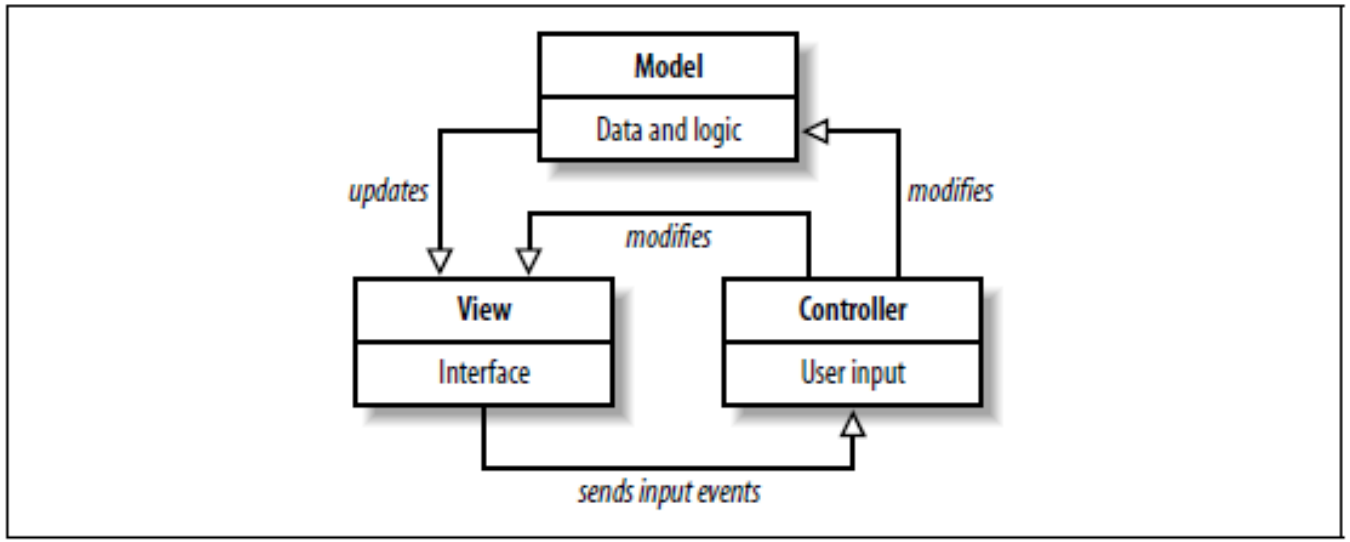


Pattern architetturali – Pattern Model-View-Controller

- La comunicazione procede in una sola direzione attraverso i riferimenti agli oggetti mostrati nella precedente figura:
  - La view riceve l'input dall'utente e lo passa al controller.
  - Il controller riceve l'input dell'utente dalla view.
  - Il controller modifica il model in risposta all'input dell'utente (o, in alcuni casi, il controller modifica la view direttamente e non aggiorna del tutto il model).
  - Il model cambia sulla base degli aggiornamenti del controller.
  - Il model notifica i cambiamenti alla view.
  - La view aggiorna l'interfaccia utente.

Pattern architetturali – Pattern Model-View-Controller

- La seguente figura mostra il ciclo di comunicazione MVC:



- Il punto di partenza del ciclo è tipicamente la ricezione dell'input dell'utente; tuttavia, un'altra parte del programma potrebbe anche far partire il ciclo modificando direttamente il model.

- La figura mostra il caso più semplice di MVC, con un singolo model, una singola view e un singolo controller.

- Tuttavia, non è raro per un'applicazione prevedere due o più view per un singolo model.

Pattern architetturali – Pattern Model-View-Controller

- Una view non necessariamente deve rappresentare un intero model e non deve neppure rappresentare la stessa informazione di un'altra view.

- Mentre un'implementazione MVC può avere più view per un model, ciascuna view ha esattamente un'istanza di controller, e viceversa.

Pattern architetturali – Pattern Model-View-Controller – Responsabilità del model

- Il model memorizza i dati e fornisce dei metodi specifici per l'applicazione per definire i valori di questi dati e per recuperarli successivamente.
- I metodi di gestione dei dati non sono generici; essi sono personalizzati per l'applicazione e devono essere conosciuti al controller e alla view.
- Il model di un'applicazione relativa ad un orologio definisce metodi specifici per un orologio, quali `setTime()` e `stop()`.
- I controller in MVC sono personalizzati per manipolare uno specifico model.
- I dati del model possono essere cambiati esternamente da una classe esterna, oppure internamente, dalla propria logica.
- Il model deve fornire alle view un modo con cui queste possono registrarsi e deve gestire una lista di view ad esso registrate.
- Se il model determina che il suo stato è cambiato significativamente deve inviare una notifica a tutte le view registrate.

Pattern architetturali – Pattern Model-View-Controller – Responsabilità del model

- Il model implementa la logica complessiva del pattern MVC; per esempio, nell'applicazione dell'orologio implementa un metodo `tick()` che viene eseguito una volta al secondo e aggiorna l'ora.

- Il model potrebbe anche fornire servizi di validazione dei dati e altre utility specifiche per le applicazioni quali, ad esempio, il caricamento dell'ora corrente da un'applicazione lato server.

Pattern architetturali – Pattern Model-View-Controller – Responsabilità della view

- La view deve creare l'interfaccia utente e mantenerla aggiornata.
- Essa si pone in attesa di cambiamenti di stato nel model e quando questi avvengono aggiorna l'interfaccia opportunamente.
- Per esempio, nell'applicazione dell'orologio, quando cambia il tempo il model invia una notifica alle view registrate.
- Ciascuna view deve passare tutti gli eventi di input al suo controller senza elaborare da sé nessun input.
- La view potrebbe interrogare il model per sapere lo stato di quest'ultimo per determinare cosa è cambiato in seguito alla ricezione di un aggiornamento.
- La view non cambia mai il model ma può richiedere ad esso informazioni.
- Uno si potrebbe chiedere se questo modo di procedere ha senso a livello architetturale.
- Nel caso dell'orologio, dove ci sono più view, l'architettura fa si che i cambiamenti in una view si riflettano anche sulle altre view.
- La logica è centralizzata nel model il che evita la duplicazione del codice e consente di aggiungere e rimuovere view a runtime con uno sforzo minimo.

Pattern architetturali – Pattern Model-View-Controller – Responsabilità del controller

- Il controller si pone in attesa di notifiche da parte della view relative agli input dell'utente e traduce quegli input in cambiamenti del model.
- In alcuni casi il controller interpreta opportunamente gli input prima di effettuare i cambiamenti nel model.
- In alcuni casi speciali il controller potrebbe addirittura istruire la view per effettuare cambiamenti nell'interfaccia utente chiamando i metodi della view.
- Ciò avviene ad esempio quando i cambiamenti sono puramente di apparenza e non hanno alcun effetto sul model.

Pattern architetturali - Layered Architecture

- I concetti di separazione e indipendenza sono fondamentali nella progettazione architetturale, in quanto consentono di localizzare le modifiche.
- Il pattern MVC, illustrato in precedenza, separa il Model, la View e il Controller consentendo loro di cambiare in maniera indipendente.
- Per esempio, aggiungere una nuova view o modificare una esistente può essere fatto senza cambiare i dati sottostanti del model.
- Il Layered Architecture Pattern è un altro modo di ottenere la separazione e l'indipendenza.
- In esso, le funzionalità del sistema sono organizzate in strati separati, ciascuno dei quali si basa sulle funzioni e sui servizi offerti dallo strato immediatamente sottostante.

Pattern architetturali - Layered Architecture

- Il Layered Architecture Pattern viene illustrato nella seguente figura:

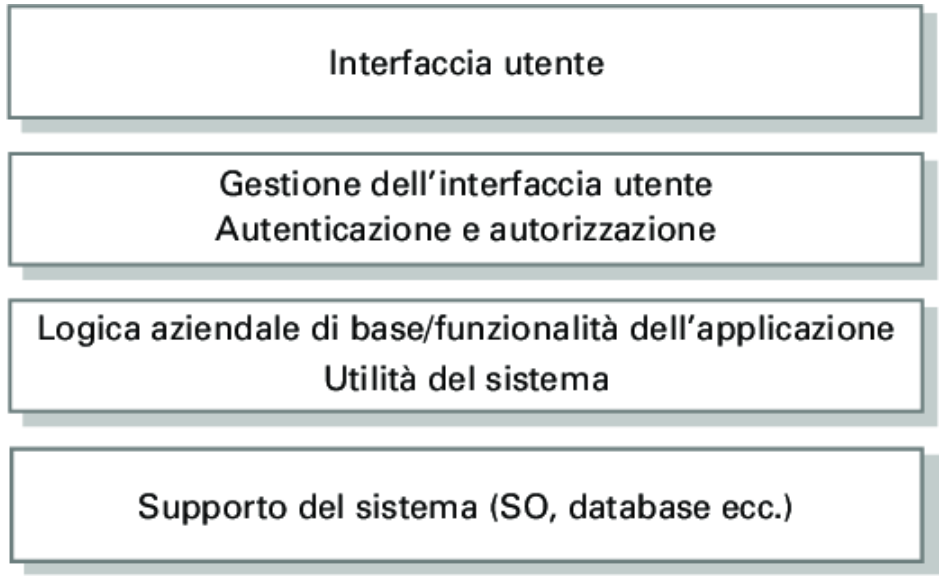
Nome	**Layered Architecture Pattern**
Descrizione	Organizza il sistema in vari strati, con funzionalità associate a ciascuno strato. Uno strato fornisce i servizi allo strato sopra di esso, quindi gli strati di livello più basso rappresentano i servizi di base che probabilmente saranno utilizzati in tutto il sistema. Ciò è rappresentato nella figura successiva.
Esempio	Un modello a strati di un sistema di apprendimento digitale che supporta l'apprendimento di tutti i soggetti nelle scuole. Questo è mostrato nella figura successiva alla figura successiva.
Quando si usa	Si usa per costruire nuove funzioni per un sistema esistente. Si usa, anche, quando lo sviluppo è distribuito su più team, dove ogni team ha la responsabilità di sviluppare le funzioni di uno strato. Si usa, infine, quando c'è una richiesta di protezione su più livelli.
Vantaggi	Consente la sostituzione di interi strati se l'interfaccia non viene modificata. Le funzioni ridondanti (per esempio, l'autenticazione) possono essere fornite in ciascuno strato per aumentare la fidatezza del sistema.
Svantaggi	Nella pratica è spesso difficile ottenere una netta separazione tra gli strati, e uno strato di alto livello potrebbe avere bisogno di interagire direttamente con strati di livelli inferiori, anziché con lo strato immediatamente sotto di esso. Le performance possono essere un problema a causa dei vari livelli di interpretazione di una richiesta di servizio, essendo questa elaborata in ciascuno strato.

Pattern architetturali - Layered Architecture

- Questo approccio a strati supporta lo sviluppo incrementale dei sistemi.
- Quando viene sviluppato uno strato, alcuni dei servizi da esso forniti sono resi accessibili agli utenti.
- Quest'architettura è anche modificabile e portabile; se la sua interfaccia non viene modificata, un nuovo strato con funzionalità estese può sostituire uno strato esistente, senza modificare le altre parti del sistema.
- Inoltre, se si modificano le interfacce di uno strato o vengono aggiunte nuove funzionalità solo lo strato adiacente ne è influenzato.
- Poiché i sistemi stratificati localizzano le dipendenze della macchina, è più semplice realizzare implementazioni multipiattaforma di un sistema applicativo.
- Solo gli strati dipendenti dalla macchina devono essere reimplementati per tenere conto delle funzionalità di un diverso sistema operativo o database.

Pattern architetturali - Layered Architecture

- La seguente figura è un esempio di architettura a strati con quattro strati:



- Lo strato più basso include il software di supporto del sistema, tipicamente un supporto per il sistema operativo e il database.

- Lo strato successivo è quello dell'applicazione, che include i componenti che riguardano le utilità e le funzionalità utilizzate da altri componenti dell'applicazione.

Pattern architetturali - Layered Architecture

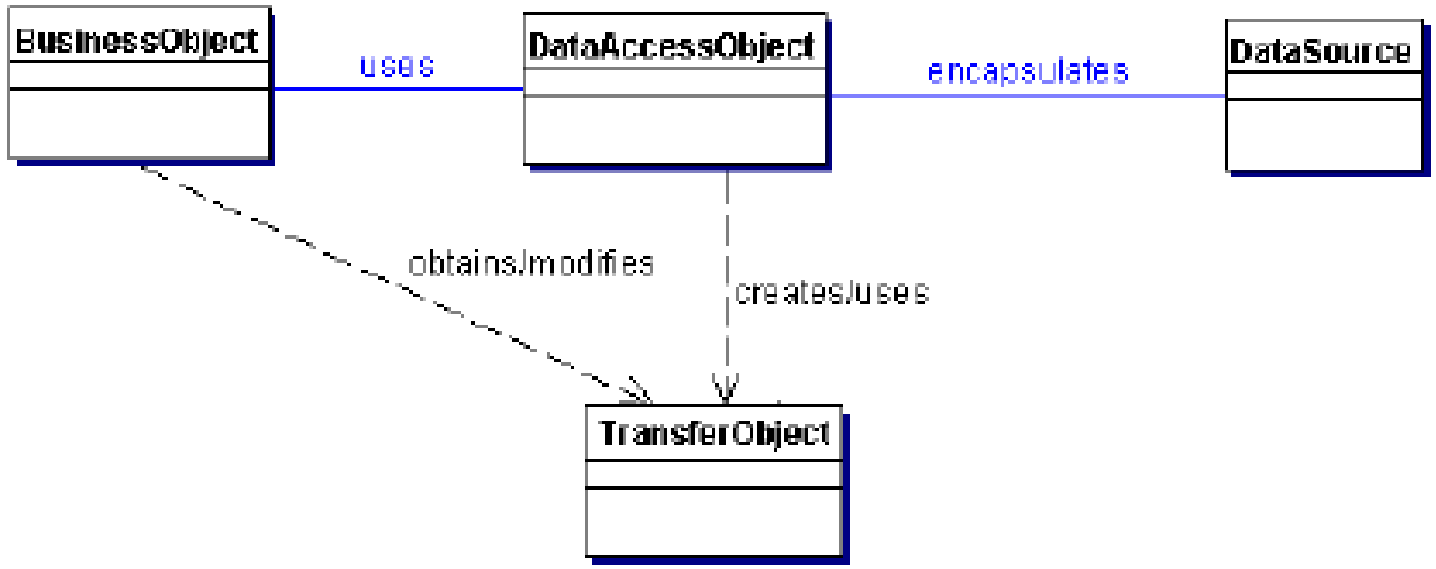
- Il terzo strato riguarda la gestione dell'interfaccia utente e le funzioni per l'autenticazione e l'autorizzazione degli utenti.
- Lo strato superiore fornisce le funzioni per l'interfaccia utente.
- Ovviamente il numero di strati è arbitrario, qualsiasi strato della figura precedente può essere suddiviso in due o più strati.

Pattern architetturali - DAO

- Il pattern DAO ha lo scopo di disaccoppiare l'accesso ai dati rispetto alla sua memorizzazione sottostante.
- Esso consente di scegliere il tipo di DBMS da utilizzare durante l'implementazione invece che in fase di progettazione.
- Gli oggetti DAO consentono la portabilità dell'applicazione da una sorgente di dati ad un'altra.
- Ci sono attualmente diverse tipologie di DBMS; utilizzando gli oggetti DAO si disaccoppia l'implementazione della sorgente dati rispetto all'effettivo utilizzo.
- Questo consente di spostarsi da una sorgente dati ad un'altra senza dover cambiare la logica di business.

Pattern architetturali - DAO

- La seguente figura mostra un diagramma delle classi per il pattern DAO:



- `BusinessObject` rappresenta la classe con la logica di business; essa ha la responsabilità di specificare cosa modificare e come modificarlo, ma non come memorizzare i dati.

- `DataAccessObject` nasconde la `DataSource` effettiva.

Pattern architetturali - DAO

- Una classe `DataAccessObject` può essere facilmente sostituita con un'altra qualora cambia la sorgente informativa.
- `DataSource` è associata all'effettiva sorgente informativa.
- `TransferObject` viene usato per trasferire i dati effettivi da `DataAccessObject` a `BusinessObject` e viceversa.
- Esso rappresenta i dati memorizzati nel database e non è direttamente connesso a `DataSource`.
- Qualunque cambiamento a `TransferObject` deve passare per `DataAccessObject` prima che possa diventare permanente.

Pattern architetturali – Repository

- Il Repository pattern spiega come una serie di componenti interattivi possono condividere i dati.
- La sua descrizione è la seguente:

Nome	**Repository Pattern**
Descrizione	Tutti i dati di un sistema vengono gestiti in un database centrale (detto repository) che è accessibile da tutti i componenti del sistema. I componenti non interagiscono direttamente ma soltanto attraverso il repository.
Esempio	La figura che mostreremo nelle prossime slide è un esempio di IDE dove i componenti usano un repository di informazioni per la progettazione di un sistema. Ogni strumento software genera informazioni che poi sono messe a disposizione degli altri strumenti.
Quando si usa	Si usa quando nel sistema vengono generati grandi volumi di informazioni che devono essere memorizzate per lungo tempo. Si può usare anche nei sistemi guidati dai dati dove l'inclusione dei dati nel repository innesca un'azione o l'utilizzo di uno strumento.
Vantaggi	I componenti possono essere indipendenti: un componente non deve necessariamente sapere dell'esistenza di un altro componente. Le modifiche fatte da un componente possono essere rese note agli altri componenti. Tutti i dati vengono gestiti in modo coerente (per esempio, i backup vengono effettuati contemporaneamente) in quanto si trovano nello stesso posto.
Svantaggi	Il repository è un punto comune di malfunzionamento, nel senso che i suoi problemi influiscono sull'intero sistema. Potrebbero esserci delle inefficienze nell'organizzazione di tutte le comunicazioni con il repository. Potrebbe essere difficile distribuire il repository su più computer.

Pattern architetturali – Repository

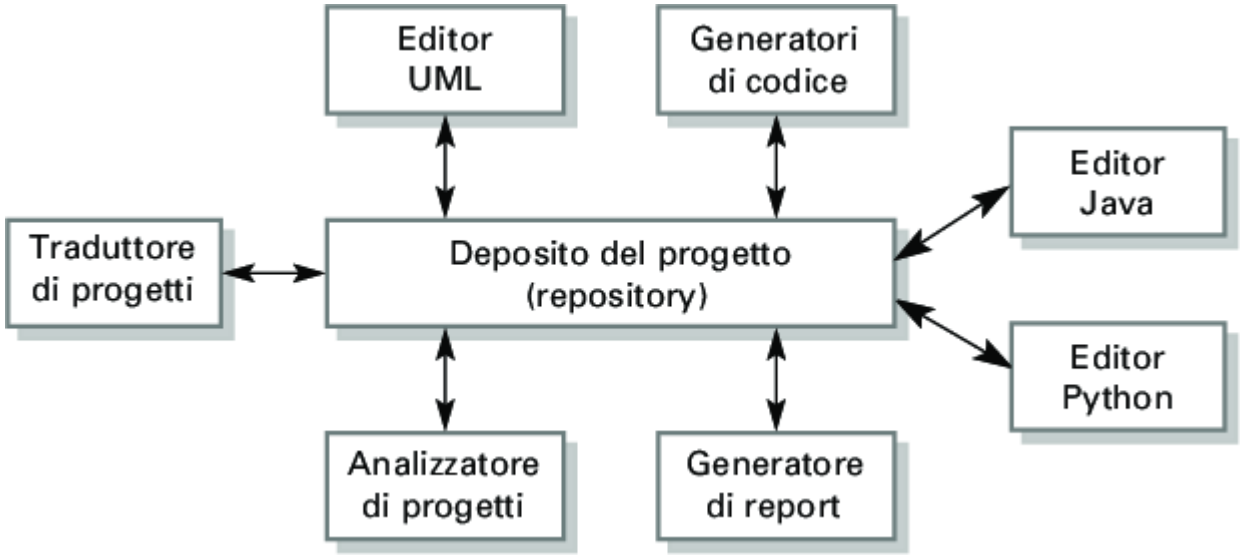
- La maggior parte dei sistemi che usano grandi quantità di dati sono organizzati attorno a un database condiviso o repository.

- Questo pattern è quindi adatto alle applicazioni nelle quali i dati sono generati da un componente e utilizzati da un altro componente.

- Esempi di questo tipo di sistemi sono i sistemi di comando e controllo, i sistemi di gestione delle informazioni, i sistemi CAD (Computer Aided Design) e gli ambienti di sviluppo interattivo per il software.

Pattern architetturali – Repository

- La seguente figura illustra una situazione in cui può essere usato il pattern repository.



- Lo schema mostra un IDE che include vari strumenti per supportare lo sviluppo guidato da modelli.
- Il repository in questo caso potrebbe essere un ambiente controllato dalle versioni che tiene traccia delle modifiche apportate al software e consente un passaggio alle versioni precedenti.
- Organizzare gli strumenti attorno a un repository è un modo efficiente di condividere grandi quantità di dati.
- Non è necessario trasmettere i dati esplicitamente da un componente all'altro.

Pattern architetturali – Repository

- Tuttavia, i componenti devono operare attorno a un modello dei dati del repository concordato.
- Inevitabilmente, questo si otterrà come un compromesso tra le esigenze specifiche di ciascuno strumento, e potrebbe essere difficile o impossibile integrare nuovi componenti se i loro modelli di dati non sono conformi allo schema concordato.
- In pratica, potrebbe essere difficile distribuire il repository su un numero di macchine.
- Sebbene sia possibile distribuire un repository logicamente centralizzato, questo richiede la manutenzione di più copie di dati. Per mantenere coerenti e aggiornate queste copie, si aggiungono altri overhead al sistema.
- Nell'architettura vista nella figura precedente il repository è passivo e del suo controllo sono responsabili i componenti che lo utilizzano.
- Un approccio alternativo, che è stato derivato per i sistemi di intelligenza artificiale, usa un modello «blackboard» che attiva i componenti quando sono disponibili determinati dati.
- Questo modello è utile quando i dati del repository non sono strutturati.
- La decisione su quale strumento attivare può essere presa solo quanto tutti i dati sono stati analizzati.

Pattern architetturali – Client-Server

- Il pattern client-server illustra una tipica organizzazione a runtime di sistemi distribuiti.
- La sua descrizione è la seguente:

Nome	**Client-Server Pattern**
Descrizione	In un'architettura client-server il sistema è presentato come un insieme di servizi, ciascuno dei quali è fornito da un server separato. I client sono utenti di questi servizi e accedono ai server per utilizzare tali servizi.
Esempio	La figura nelle slide seguenti è un esempio di una libreria di film e video DVD organizzata come un sistema client-server.
Quando si usa	Si usa quando occorre accedere ai dati di un database da più postazioni. Poiché i server possono essere replicati, lo schema può essere utilizzato anche quando il carico su un sistema è variabile.
Vantaggi	Il principale vantaggio di questo modello è che i server possono essere distribuiti in una rete. Le funzionalità più comuni (per esempio, il servizio di stampa) possono essere a disposizione di tutti i client e non devono essere necessariamente implementate da tutti i servizi.
Svantaggi	Ciascun servizio è un punto comune di malfunzionamento, nel senso che è suscettibile di attacchi denial-of-service ed è soggetto a guasti del server. Le prestazioni possono essere imprevedibili in quanto dipendono dalla rete e anche dal sistema. Possono nascere problemi di gestione se i server sono di proprietà di più organizzazioni.

Pattern architetturali – Client-Server

- Un sistema conforme allo schema client-server è organizzato come un insieme di servizi e server associati e di client che accedono e usano tali servizi.

- I principali componenti di questo modello sono:

  - Un insieme di server che offrono servizi ad altri sottosistemi, per esempio servizi di stampa, servizi di gestione dei file e servizi di compilazione dei linguaggi di programmazione.

    - I server sono componenti software e molti server possono essere eseguiti sullo stesso computer.

  - Un insieme di client che richiedono i servizi offerti dai server.

    - Di solito si tratta di varie istanze di programmi client eseguiti contemporaneamente su computer differenti.

  - Una rete che permette ai client di accedere a questi servizi.

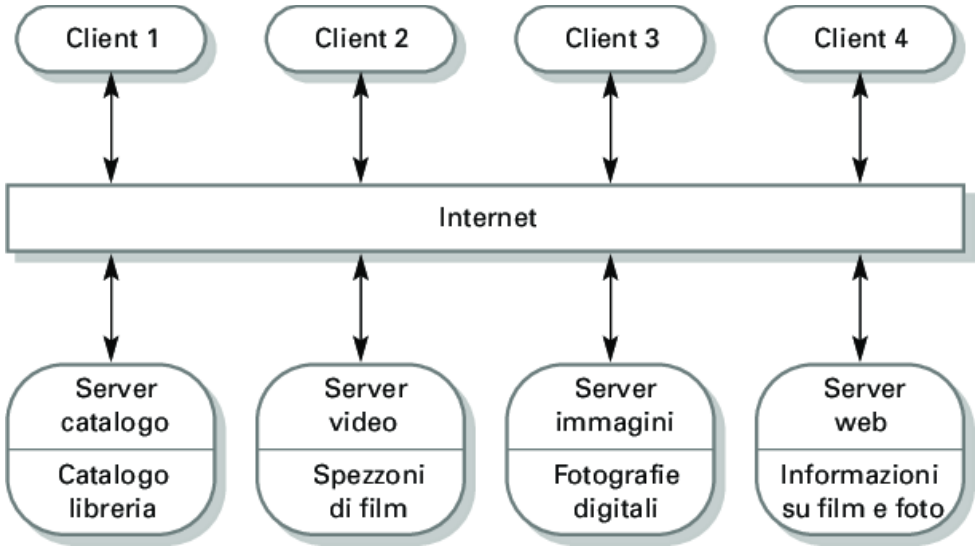
    - I sistemi client-server di solito sono implementati come sistemi distribuiti, collegati attraverso i protocolli Internet.

Pattern architetturali – Client-Server

- Le architetture client-server di solito sono concepite come architetture di sistemi distribuiti, ma il modello logico di servizi indipendenti che sono eseguiti su server separati può essere implementato su un singolo computer.
- Ancora una volta, la separazione e l'indipendenza sono un importante vantaggio.
- I servizi e i server possono essere modificati senza influire su altre parti del sistema.
- I client potrebbero avere bisogno di conoscere il nome dei server disponibili e i servizi che possono fornire.
- I server, invece, non hanno bisogno di conoscere l'identità dei client o il numero di client che stanno utilizzando i loro servizi.
- I client accedono ai servizi offerti da un server attraverso chiamate di procedure remote, usando un protocollo richiesta-risposta (come http), dove un client invia una richiesta a un server e resta in attesa finchè non riceve una risposta dal server.

Pattern architetturali – Client-Server

- La seguente figura mostra l'esempio di un sistema basato sul modello client-server: si tratta di un sistema multiutente, basato sul Web, che fornisce una libreria di film e fotografie.



- In questo sistema diversi server gestiscono e mostrano i diversi tipi di media.

- I frame dei video devono essere trasmessi velocemente e in sincronia, ma con una risoluzione relativamente bassa; possono essere compressi in una unità di memoria in modo che il server dei video possa gestirne la compressione e la decompressione in diversi formati.

- Le immagini statiche, invece, devono essere memorizzate ad alta risoluzione, quindi è opportuno gestirle in un server separato.

Pattern architetturali – Client-Server

- Il catalogo deve essere in grado di gestire una serie di richieste e fornire i collegamenti al sistema informativo del Web che contiene dati sui film e sui videoclip, e ad un sistema di commercio elettronico che ne gestisce la vendita.
- Il programma client è solo un'interfaccia utente integrata per accedere a tali servizi, costruita utilizzando un brower web.
- Il vantaggio più importante del pattern client-server è che si tratta di un'architettura distribuita.
- L'uso più efficiente si realizza nei sistemi in rete con molti processori distribuiti.
- È facile aggiungere un nuovo server e integrarlo con il resto del sistema o aggiornare i server in modo trasparente senza influire su altri parti del sistema.

Pattern architetturali – Pattern Pipe-and-filter

- L'ultimo esempio di pattern architetturale è il pattern pipe-and-filter, che rappresenta l'organizzazione a runtime di un sistema dove le trasformazioni funzionali elaborano i loro input e generano output.

- La sua descrizione è la seguente:

Nome	**Pipe-and-Filter Pattern**
Descrizione	L'elaborazione dei dati del sistema è organizzata in modo che ciascun componente di elaborazione (filtro) è discreto e svolge un particolare tipo di trasformazione dei dati. I dati fluiscono come in un tubo (pipe) da un componente all'altro per essere elaborati.
Esempio	La figura delle prossime slide è un esempio di sistema pipe-and-filter per l'elaborazione di fatture.
Quando si usa	Si usa di solito nelle applicazioni per l'elaborazione dei dati (batch o basata su transazioni), dove gli input vengono elaborati in fasi separate per generare i relativi output.
Vantaggi	È facile da capire e supporta il riutilizzo delle trasformazioni. Lo stile del flusso delle operazioni rispecchia la struttura di molti processi aziendali. La sua evoluzione è semplice perché permette di aggiungere agevolmente le nuove trasformazioni. Può essere implementato come sistema sequenziale o parallelo.
Svantaggi	Il formato di trasferimento dei dati deve essere concordato fra le varie trasformazioni. Ciascuna trasformazione deve essere in grado di leggere gli input e fornire gli output nel formato concordato. Questo aumenta gli overhead del sistema e potrebbe rendere impossibile il riutilizzo di quei componenti architetturali che usano strutture di dati non compatibili con il formato concordato.

Pattern architetturali – Pattern Pipe-and-filter

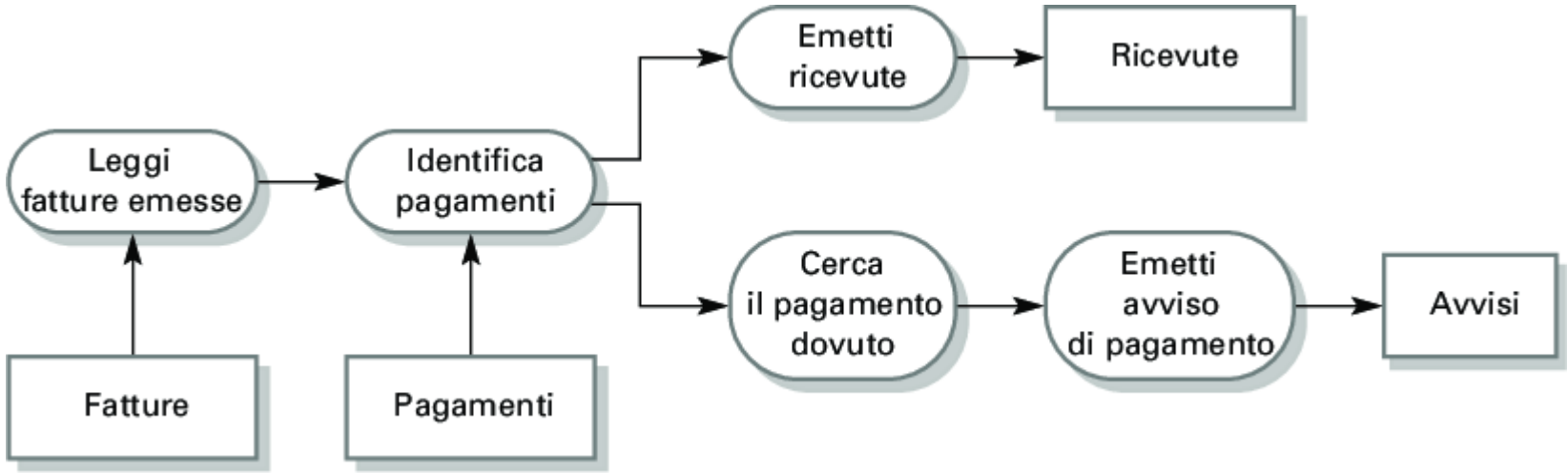
- I dati passano da una trasformazione all'altra attraverso una sequenza.
- Ogni passo di elaborazione viene implementato come una trasformazione: i dati di input passano attraverso queste trasformazioni finchè non vengono convertiti in output.
- Le trasformazioni possono essere eseguite sequenzialmente o in parallelo.
- I dati possono essere elaborati da ogni trasformazione elemento per elemento oppure in blocco.
- Il nome «pipe-and-filter» (letteralmente «tubo e filtro») deriva dalla terminologia utilizzata nei sistemi Unix, dove è possibile collegare i processi tramite «pipe».
- Le pipe passano un flusso di testo da un processo all'altro.
- I sistemi conformi a questo schema possono essere implementati combinando comandi Unix, utilizzando le pipe e le funzionalità di controllo della shell di Unix.
- Il termine «filter» viene utilizzato perché una trasformazione «filtra» i dati che può elaborare dal suo flusso di dati di input.

Pattern architetturali – Pattern Pipe-and-filter

- Varianti di questo schema sono state utilizzate quando i computer erano usati nelle prime elaborazioni automatiche di dati.
- Quando le trasformazioni sono sequenziali con i dati elaborati in blocco, questo schema architetturale prende il nome di modello batch sequenziale.
- Questa è una tipica architettura dei sistemi di elaborazione dei dati, per esempio quelli di fatturazione.
- Anche l'architettura di un sistema integrato può essere organizzata come una pipeline di processi, dove ciascun processo viene eseguito in parallelo agli altri.

Pattern architetturali – Pattern Pipe-and-filter

- Un esempio di questo tipo di architettura di sistema, usato in un'applicazione di elaborazione batch, è illustrato nella seguente figura.
- Un'azienda ha inviato le fatture ai clienti che, dopo una settimana, vengono confrontate con i pagamenti riscossi.
- Per le fatture che sono state pagate viene rilasciata una ricevuta; per quelle non pagate entro la scadenza viene emesso un avviso di sollecito.



Pattern architetturali – Pattern Pipe-and-filter

- Il pattern pipe-and-filter è particolarmente adatto ai sistemi di elaborazione batch e ai sistemi integrati, dove l'interazione con gli utenti è limitata.

- I sistemi interattivi sono difficili da scrivere utilizzando il pattern pipe-and-filter perché è necessario avere un flusso di dati da elaborare.

- Mentre gli input e gli output di testo possono essere modellati in questo modo, le interfacce grafiche utente hanno formati di I/O più complessi e una tecnica di controllo che si basa su eventi, come i click del mouse o le selezioni dei menù.

- Tutto questo è difficile da implementare come un flusso sequenziale che è conforme al pattern pipe-and-filter.

Che cos'è un design pattern

- I design pattern sono soluzioni tipiche ai problemi che si verificano comunemente nella progettazione di software.
- Sono come progetti pre-costruiti che si possono personalizzare per risolvere un problema di progettazione ricorrente nel codice.
- Non è possibile trovare un pattern e copiarlo semplicemente nel proprio programma, come si può fare con funzioni o biblioteche già pronte all'uso.
- Il pattern non è un pezzo di codice specifico, ma un concetto generale per risolvere un particolare problema.
- Possiamo seguire i dettagli del pattern e implementare una soluzione che si adatta alla realtà del nostro programma.
- I pattern sono spesso confusi con gli algoritmi, perché entrambi i concetti descrivono soluzioni tipiche ad alcuni problemi noti.
- Mentre un algoritmo definisce sempre un insieme chiaro di azioni che possono raggiungere un certo obiettivo, un pattern è una descrizione di più alto livello di una soluzione.
- Il codice dello stesso pattern applicato a due programmi diversi può essere diverso.

Che cos'è un design pattern

- Un'analogia con un algoritmo è una ricetta di cucina: entrambi hanno passi chiari per raggiungere un obiettivo.
- D'altra parte, un pattern è più simile a un progetto: si può vedere quale sia il risultato e le sue caratteristiche, ma l'esatto ordine di implementazione dipende da noi.
- La maggior parte dei pattern sono descritti in modo molto formale, in modo che le persone possano riprodurli in molti contesti.
- Le sezioni che di solito sono presenti in una descrizione di un pattern sono le seguenti:
  - L'intento del pattern: descrive brevemente sia il problema che la soluzione.
  - La motivazione: spiega ulteriormente il problema e la soluzione che il pattern rende possibile.
  - La struttura delle classi: mostra ogni parte del pattern e come le varie parti sono tra loro collegate.
  - L'esempio di codice in uno dei linguaggi di programmazione più diffusi: rende più facile comprendere l'idea alla base del pattern.
- Alcuni cataloghi di pattern elencano altri dettagli utili, come l'applicabilità del pattern, le fasi di implementazione e le relazioni con altri pattern.

Che cos'è un design pattern – La storia dei pattern

- I design pattern non sono concetti oscuri e sofisticati, al contrario.
- I pattern sono soluzioni tipiche ai problemi comuni della programmazione orientata agli oggetti.
- Quando una soluzione viene ripetuta più e più volte in vari progetti, qualcuno alla fine le dà un nome e descrive la soluzione nel dettaglio.
- Questo è fondamentalmente il modo in cui viene scoperto un pattern.
- Il concetto di pattern è stato descritto per la prima volta da Christopher Alexander in A Pattern Language: Towns, Buildings, Constructions.
- Il libro descrive un "linguaggio" per la progettazione dell'ambiente urbano.
- Le unità di questo linguaggio sono pattern.
- Possono descrivere quanto dovrebbero essere alte le finestre, quanti piani dovrebbe avere un edificio, quanto dovrebbero essere ampie le aree verdi in un quartiere, e così via.

Che cos'è un design pattern – La storia dei pattern

- L'idea è stata ripresa da quattro autori: Erich Gamma, John Vlissides, Ralph Johnson e Richard Helm.
- Nel 1994 hanno pubblicato Design Patterns: Elements of Reusable Object-Oriented Software, in cui hanno applicato il concetto di design pattern alla programmazione.
- Il libro presentava 23 pattern che risolvevano vari problemi della programmazione orientata agli oggetti ed è diventato un best-seller molto rapidamente.
- A causa del suo lungo nome, la gente cominciò a chiamarlo "il libro della gang of four" che fu presto abbreviato in semplice "il libro della GoF".
- Da allora, sono state scoperte decine di altri pattern orientati agli oggetti.
- L' «approccio basato sui pattern» è diventato molto popolare in altri campi di programmazione, per cui ora esistono anche molti altri pattern al di fuori della programmazione orientata agli oggetti.

Che cos'è un design pattern – Perché i pattern

- Si potrebbe riuscire a lavorare come programmatore per molti anni senza conoscere un solo pattern. Molte persone lo fanno.
- Anche in questo caso, però, potremmo aver implementato alcuni pattern senza nemmeno saperlo.
- Allora perché dovremmo passare del tempo ad impararli?
  - I design pattern sono un toolkit di soluzioni provate e collaudate a problemi comuni nella progettazione di software.
    - Anche se non si incontrano mai questi problemi, conoscere i pattern è comunque utile perché insegna a risolvere ogni tipo di problema utilizzando i principi della programmazione orientata agli oggetti.
  - I design pattern definiscono un linguaggio comune che noi e i nostri compagni di team possiamo usare per comunicare in modo più efficiente.
    - Potremmo dire: "Oh, basta usare un Singleton per questo" e tutti capiranno l'idea alla base del nostro suggerimento. Non c'è bisogno di spiegare cos'è un singleton se si conosce il pattern e il suo nome.

Che cos'è un design pattern – Problemi dei pattern

- Nel tempo i pattern anche avuto non solo approvazioni ma anche svariate critiche.
- Diamo un'occhiata alle argomentazioni più tipiche contro l'uso dei pattern:
  - I pattern sono collanti per un linguaggio di programmazione debole:
    - Di solito la necessità dei pattern nasce quando le persone scelgono un linguaggio di programmazione o una tecnologia che manca del necessario livello di astrazione.
    - In questo caso, i pattern diventano un collante che conferisce al linguaggio le necessarie super-abilità.
    - Per esempio, il pattern Strategy può essere implementato con una semplice funzione anonima (lambda) nella maggior parte dei moderni linguaggi di programmazione.
  - I pattern sono soluzioni inefficienti:
    - I pattern cercano di sistematizzare gli approcci già ampiamente utilizzati.
    - Questa unificazione è vista da molti come un dogma, e questi implementano i pattern «alla lettera», senza adattarli al contesto del loro progetto.

Che cos'è un design pattern – Problemi dei pattern

- I pattern vengono utilizzati anche laddove il loro utilizzo non sarebbe giustificato:
  - Se tutto quello che hai è un martello, tutto sembra un chiodo.
  - Questo è il problema che ossessiona molti programmatori che hanno appena familiarizzato con i pattern. Avendo imparato a conoscere i pattern, cercano di applicarli ovunque, anche in situazioni in cui il codice più semplice andrebbe benissimo.

Che cos'è un design pattern – Utilizzo

- I design pattern possono essere applicati sia nella fase di disegno di un nuovo sistema sia per rivedere un sistema esistente applicando il refactoring al codice.
- L'approccio corretto deve essere orientato alla ricerca delle soluzioni più semplici tra quelle che effettivamente risolvono le problematiche incontrate.
- È importante non sovradimensionare inutilmente il sistema in fase di progettazione introducendo una complessità che si può rivelare in seguito assai dannosa in termini di manutenibilità.
- La scelta se applicare o meno uno o più pattern deriva spesso dalla conoscenza che un architetto o uno sviluppatore hanno di essi.
- Conoscere i pattern sulla carta non basta per poter operare scelte di utilizzo consapevoli.
- L'implementazione di un pattern non necessariamente segue lo schema di base.
- Questo deve far riflettere sul reale ruolo che i pattern ricoprono e sul modo con cui devono essere utilizzati.

Che cos'è un design pattern – Il cluster dei pattern GoF

- Tra i vari design pattern noti in letteratura, i pattern GoF formano, senza dubbio, un cluster fondamentale.
- I 23 pattern che compongono questo cluster sono organizzati in tre categorie distinte e tra loro complementari:
  - Pattern creazionali, che riguardano la creazione di istanze.
  - Pattern strutturali, che si riferiscono alla composizione di classi e oggetti.
  - Pattern comportamentali, che si occupano delle modalità con cui classi e oggetti interagiscono tra loro in relazione alle diverse responsabilità.

Che cos'è un design pattern – Il cluster dei pattern GoF – I pattern creazionali

- I pattern creazionali forniscono un'astrazione del processo di creazione delle istanze delle classi, favorendo l'indipendenza del sistema dalle modalità di creazione e dai tipi concreti effettivamente generati.
- Ci sono due aspetti che caratterizzano i pattern appartenenti a questa categoria:
  - La capacità di mascherare al client la conoscenza degli oggetti concreti creati.
  - La capacità di nascondere le modalità di creazione all'utilizzatore dell'istanza.
- Grazie a queste due caratteristiche ciò che viene creato in generale risulta essere disaccoppiato dal contesto di utilizzo.
- Solo l'oggetto creatore conosce il tipo effettivo dell'istanza.
- Ciò che viene esternamente reso pubblico è unicamente l'interfaccia di riferimento.

Che cos'è un design pattern – Il cluster dei pattern GoF – I pattern creazionali

- I pattern creazionali sono cinque:
  - Abstract Factory: fornisce un'interfaccia per creare famiglie di oggetti correlati o dipendenti senza specificare le classi concrete.
  - Builder: separa la costruzione di un oggetto complesso dalla sua rappresentazione, in modo tale che lo stesso processo di costruzione possa creare rappresentazioni differenti.
  - Factory Method: definisce un'interfaccia per creare un oggetto, ma lascia alle classi derivate di decidere quale classe istanziare. Questo pattern permette a una classe di delegare la creazione di un'istanza alle sue classi derivate.
  - Prototype: specifica il tipo degli oggetti da creare usando un'istanza prototipale e crea i nuovi oggetti a partire da questo prototipo.
  - Singleton: assicura che una classe abbia solamente un'unica istanza e fornisce un entry-point globale ad essa.

Che cos'è un design pattern – Il cluster dei pattern GoF – I pattern strutturali

- I pattern strutturali riguardano la modalità con cui classi e oggetti vengono aggregati allo scopo di formare entità più complesse.
- Possiamo distinguere due tipologie di pattern strutturali:
  - Pattern strutturali basati sulle classi: sfruttano l'ereditarietà multipla per combinare tra loro le interfacce e le implementazioni.
  - Pattern strutturali basati sugli oggetti: descrivono le modalità di composizione di oggetti al fine di estendere in fase di esecuzione le funzionalità di una classe.
- La maggior parte dei pattern strutturali sono basati sugli oggetti.

Che cos'è un design pattern – Il cluster dei pattern GoF – I pattern strutturali

- I pattern strutturali sono sette:
  - Adapter: converte l'interfaccia di una classe in un'altra interfaccia compatibile con il client. Questo pattern consente a classi diverse di collaborare tra loro, cosa che non sarebbe possibile diversamente a causa della incompatibilità delle rispettive interfacce.
  - Bridge: disaccoppia un'astrazione dalla sua implementazione affinché entrambe possano variare in modo indipendente.
  - Composite: compone una serie di oggetti in una struttura ad albero secondo una gerarchia di tipo part-whole (parte-totalità). Questo pattern permette ai client di trattare oggetti singoli o loro raggruppamenti in modo uniforme.
  - Decorator: aggiunge dinamicamente responsabilità addizionali ad un oggetto. Questo pattern fornisce un meccanismo alternativo e flessibile all'ereditarietà per estendere le funzionalità base.
  - Facade: fornisce un'interfaccia unificata a un insieme di interfacce in un sottosistema. Questo pattern definisce un'interfaccia ad un livello più alto che rende il sottosistema più facile da usare, dato che ne maschera la complessità interna.

Che cos'è un design pattern – Il cluster dei pattern GoF – I pattern strutturali

- Flyweight: usa la condivisione per gestire in modo efficiente un numero considerevole di oggetti a granularità fine.

- Proxy: fornisce un surrogato di un oggetto per controllare l'accesso ad esso.

Che cos'è un design pattern – Il cluster dei pattern GoF – I pattern comportamentali

- I pattern comportamentali si riferiscono alla distribuzione delle responsabilità tra oggetti tra loro correlati.
- Essi si focalizzano soprattutto sulle modalità di comunicazione e collaborazione tra oggetti e classi che interagiscono tra di loro.
- La maggior parte di questi pattern fornisce soluzioni per incapsulare le diverse funzionalità in un oggetto specifico con l'intento di delegare ad esso l'esecuzione del codice vero e proprio.
- Questo approccio permette in generale di eliminare le dipendenze dirette tra i vari oggetti coinvolti, limitando l'accoppiamento.
- La distribuzione delle responsabilità porta inevitabilmente a ridurre la genericità di ciascun partecipante dei pattern, aspetto dovuto alla forte specializzazione che caratterizza le diverse classi coinvolte.
- Possiamo distinguere due tipologie di pattern comportamentali:
  - Behavioral class pattern: utilizzano l'ereditarietà per distribuire comportamenti tra diverse classi.
  - Behavioral object pattern: utilizzano la composizione piuttosto che l'ereditarietà.

Che cos'è un design pattern – Il cluster dei pattern GoF – I pattern comportamentali

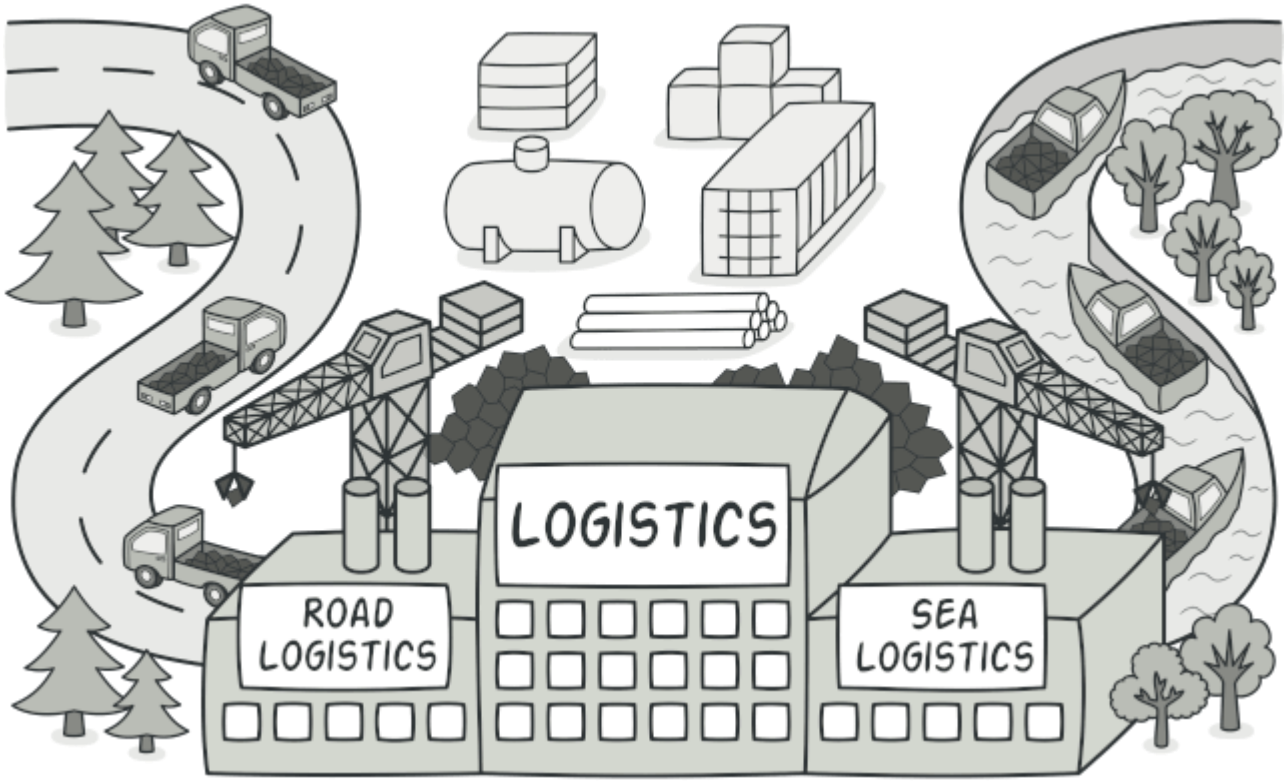
- I pattern comportamentali sono undici:
  - Chain of Responsability: evita di accoppiare il mittente di una richiesta con il suo destinatario dando la possibilità a più di un oggetto di gestire la richiesta. Collega tra loro gli oggetti ricevitori e fa passare la richiesta da un oggetto all'altro fino a destinazione.
  - Command: incapsula una richiesta in un oggetto, rendendo possibile parametrizzare i client con diverse tipologie di richieste, con richieste bufferizzate (queue), con richieste registrate (log) e con richieste annullabili (undo).
  - Interpreter: dato un linguaggio, definisce una rappresentazione della sua grammatica e del relativo interprete, che usa la rappresentazione per interpretare le frasi del linguaggio.
  - Iterator: fornisce un modo per accedere in modo sequenziale agli elementi di una collezione di oggetti senza esporre la sua rappresentazione sottostante.
  - Mediator: definisce un oggetto che incapsula le modalità di interazione di un insieme di oggetti. Questo pattern favorisce un basso accoppiamento, evitando che gli oggetti facciano riferimento l'uno con l'altro esplicitamente, e permette di variare le modalità di interazione in modo indipendente dagli oggetti stessi.

Che cos'è un design pattern – Il cluster dei pattern GoF – I pattern comportamentali

- Memento: senza violare l'incapsulamento, recupera e rende esplicito lo stato interno di un oggetto in modo tale che l'oggetto stesso possa essere riportato allo stato originale in un secondo momento.
- Observer: definisce una dipendenza uno-a-molti fra oggetti in modo tale che, se un oggetto cambia stato, tutti gli oggetti da esso dipendenti vengono notificati e aggiornati automaticamente.
- State: permette ad un oggetto di modificare il suo comportamento quando il suo stato interno cambia.
- Strategy: definisce una famiglia di algoritmi, li incapsula e li rende intercambiabili fra loro. Questo pattern permette di variare gli algoritmi in modo indipendente dal contesto di utilizzo.
- Template Method: definisce lo scheletro di un algoritmo in un metodo di una classe base, delegando alcuni passi alle classi derivate. Questo pattern permette di ridefinire nelle classi derivate alcuni passi dell'algoritmo senza cambiare la struttura dell'algoritmo stesso.
- Visitor: rappresenta un'operazione da svolgersi sugli elementi di una struttura ad oggetti. Questo pattern consente di definire nuove operazioni senza cambiare le classi degli elementi su cui opera.

Pattern Creazionali – Il pattern Factory Method - Intent

- Factory Method è un design pattern creazionale che fornisce un'interfaccia per la creazione di oggetti in una superclasse, ma permette alle sottoclassi di modificare il tipo di oggetti che verranno creati.



Pattern Creazionali – Il pattern Factory Method - Problema

- Immaginiamo di creare un'applicazione per la gestione della logistica.
- La prima versione della nostra applicazione può gestire solo il trasporto con i camion, quindi la maggior parte del nostro codice vive all'interno della classe `Truck`.
- Dopo un po', la nostra app diventa piuttosto popolare.
- Ogni giorno riceviamo decine di richieste da parte di aziende di trasporto marittimo per incorporare la logistica marittima nell'app.



Pattern Creazionali – Il pattern Factory Method - Problema

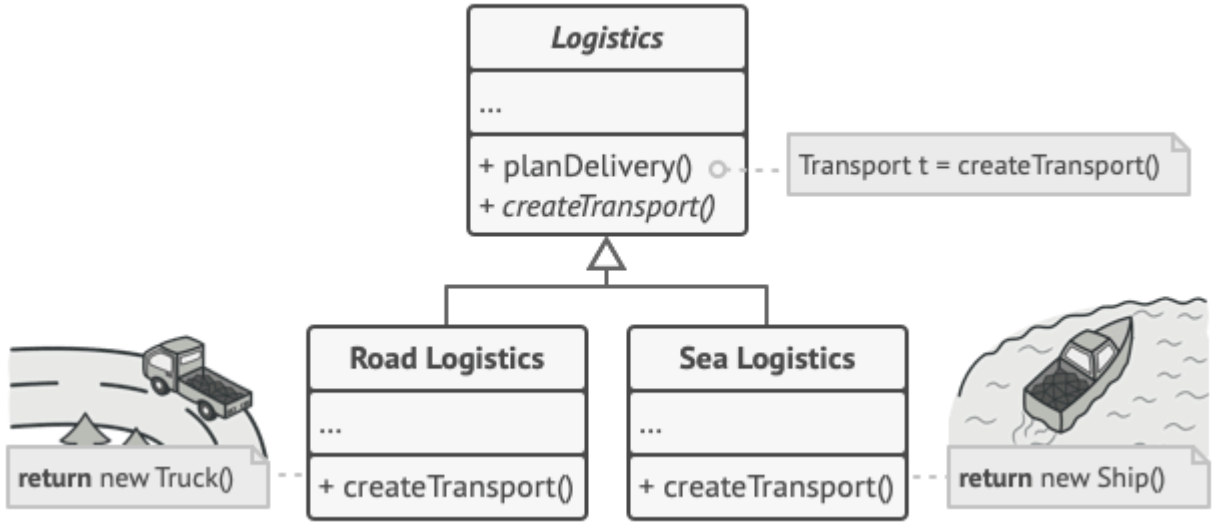
- Sarebbe una bella grande notizia; ma come fare con il codice?
- Al momento, la maggior parte del nostro codice è accoppiato alla classe `Truck`.
- Aggiungere la classe `Ships` nell'app richiederebbe modifiche all'intero codice.
- Inoltre, se in seguito si decide di aggiungere un altro tipo di trasporto all'app, probabilmente sarà necessario apportare di nuovo tutte queste modifiche.
- Di conseguenza, ci ritroveremo con un codice piuttosto malconcio, crivellato di condizioni che cambiano il comportamento dell'app a seconda della classe degli oggetti di trasporto.

Pattern Creazionali – Il pattern Factory Method - Soluzione

- Il pattern Factory Method suggerisce di sostituire le chiamate dirette alla costruzione di oggetti (usando l'operatore `new`) con chiamate ad uno speciale metodo *factory*.

- Non allarmiamoci: gli oggetti vengono ancora creati tramite l'operatore `new`, ma quest'ultimo viene chiamato dall'interno del metodo factory.

- Gli oggetti restituiti da un metodo factory sono spesso chiamati "prodotti".



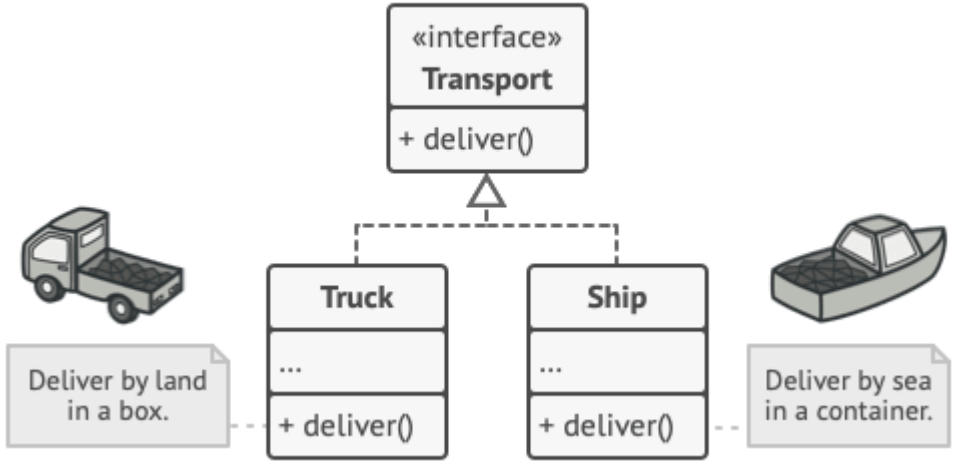
Pattern Creazionali – Il pattern Factory Method - Soluzione

- A prima vista, questo cambiamento può sembrare inutile: abbiamo semplicemente spostato la chiamata del costruttore da una parte all'altra del programma.

- Tuttavia, consideriamo questo: ora è possibile sovrascrivere il metodo factory in una sottoclasse e cambiare la classe dei prodotti che vengono creati dal metodo.

- C'è però una leggera limitazione: le sottoclassi possono restituire diversi tipi di prodotti solo se questi prodotti hanno una classe di base o un'interfaccia comune.

- Inoltre, il metodo factory nella classe base dovrebbe avere il suo tipo di ritorno dichiarato come questa interfaccia.

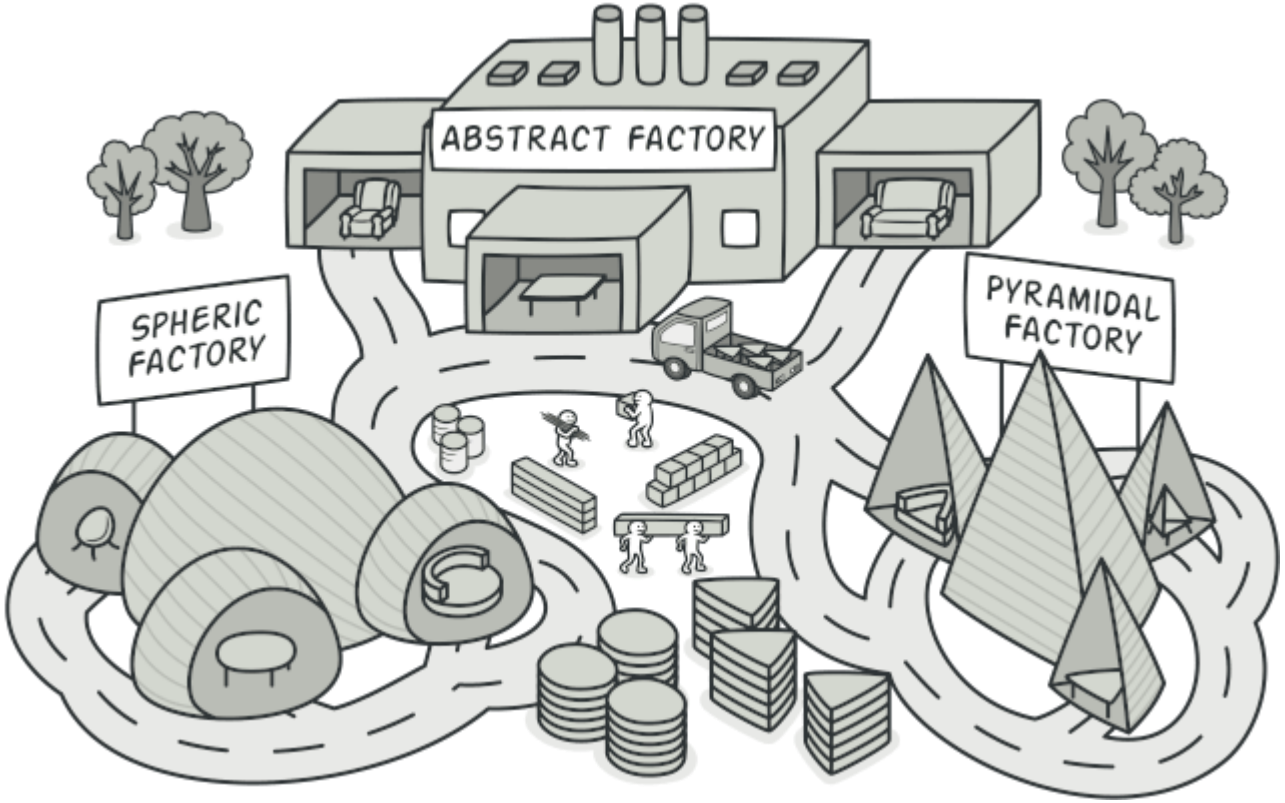


Pattern Creazionali – Il pattern Factory Method - Soluzione

- Il codice che utilizza il metodo factory (spesso chiamato codice client) non vede la differenza tra i prodotti effettivamente restituiti dalle varie sottoclassi.
- Il client tratta tutti i prodotti come `Transport` in modo astratto.
- Il client sa che tutti gli oggetti di trasporto dovrebbero avere il metodo `deliver`, ma come funziona esattamente tale metodo non è importante per il client.

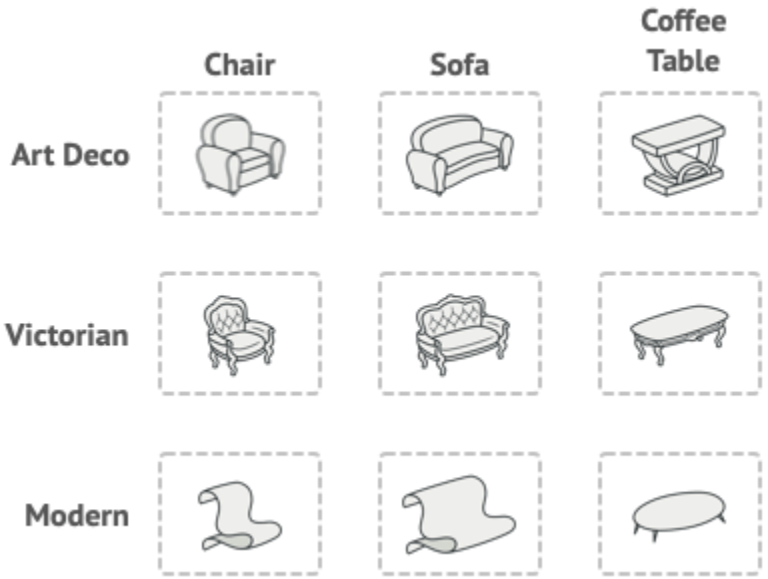
Pattern Creazionali – Il pattern Abstract Factory – Intent

- Abstract Factory è un design pattern creazionale che permette di produrre famiglie di oggetti correlati senza specificare le loro classi concrete.



Pattern Creazionali – Il pattern Abstract Factory – Problema

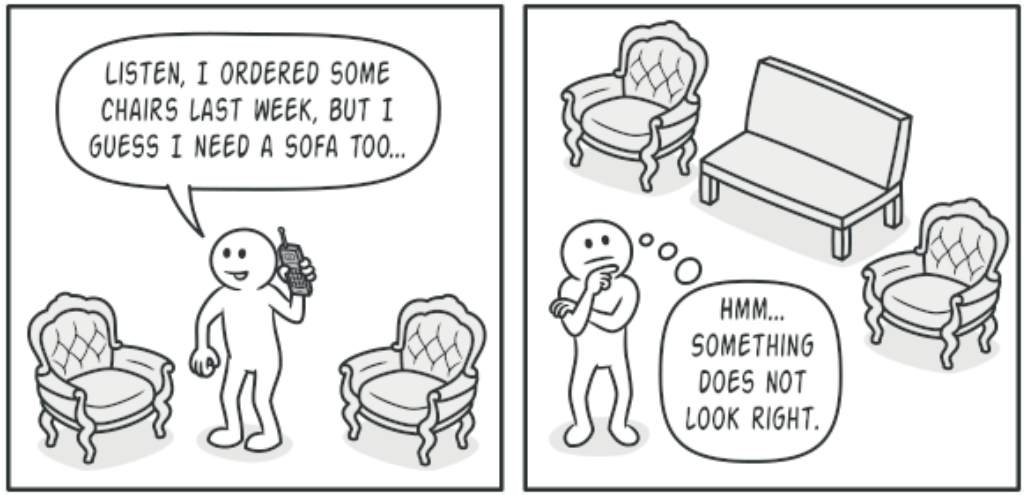
- Immaginiamo di creare un simulatore di negozio di mobili.
- Il nostro codice è costituito da classi che rappresentano:
  - Una famiglia di prodotti correlati, diciamo: `Chair` + `Sofa` + `CoffeeTable`.
  - Diverse varianti di questa famiglia; ad esempio, i prodotti `Chair` + `Sofa` + `CoffeeTable` sono disponibili in queste varianti: `Modern`, `Victorian`, `ArtDeco`.



Pattern Creazionali – Il pattern Abstract Factory – Problema

- È necessario un modo per creare singoli oggetti d'arredo in modo che si abbinino ad altri oggetti della stessa famiglia.

- I clienti si arrabbiano quando ricevono mobili non coordinati.

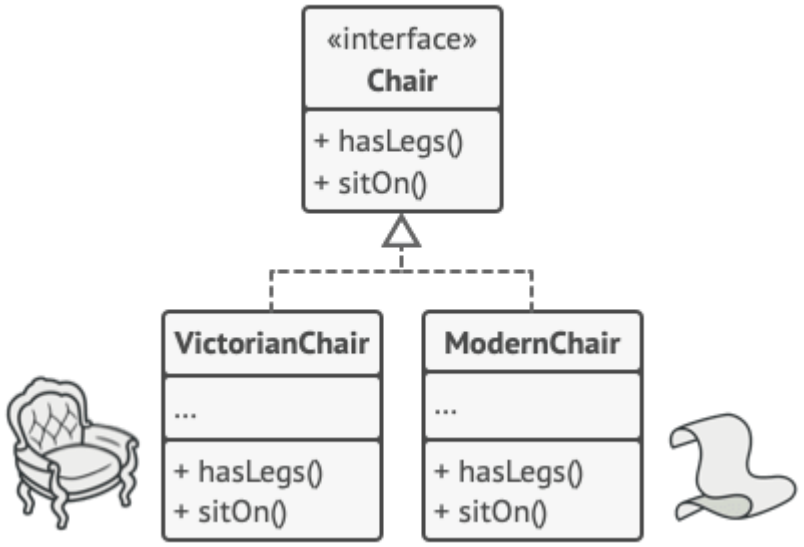


- Inoltre, non si vuole cambiare il codice esistente quando si aggiungono nuovi prodotti o famiglie di prodotti al programma.

- I fornitori di mobili aggiornano i loro cataloghi molto spesso, e non si vuole cambiare il codice di base ogni volta che ciò accade.

Pattern Creazionali – Il pattern Abstract Factory – Soluzione

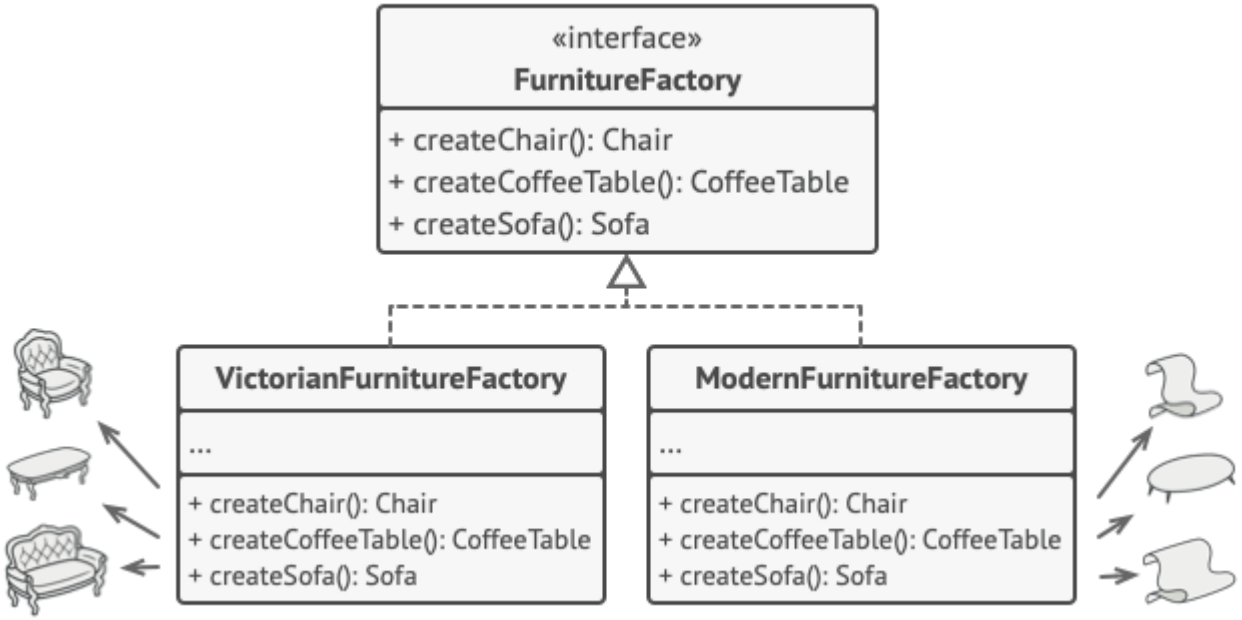
- La prima cosa che lo schema di *Abstract Factory* suggerisce è di dichiarare esplicitamente le interfacce per ogni prodotto distinto della famiglia di prodotti (ad esempio, chair, sofa o coffee table).
- Poi è possibile fare in modo che tutte le varianti dei prodotti seguano quelle interfacce.
- Per esempio, tutte le varianti di chair possono implementare l'interfaccia `Chair`; tutte le varianti di coffee table possono implementare l'interfaccia `CoffeeTable`, e così via.



Pattern Creazionali – Il pattern Abstract Factory – Soluzione

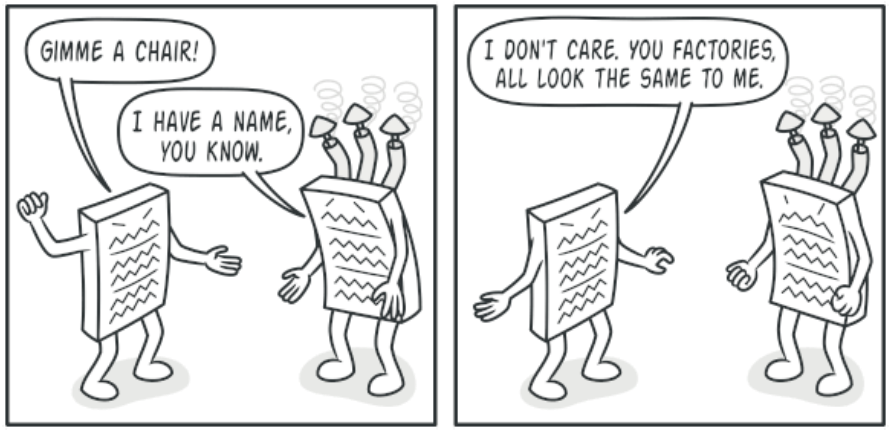
- La prossima mossa è quella di dichiarare l'Abstract Factory - un'interfaccia con un elenco di metodi di creazione per tutti i prodotti che fanno parte della famiglia (ad esempio, `createChair`, `createSofa` e `createCoffeeTable`).

- Questi metodi devono restituire i tipi di prodotti astratti rappresentati dalle interfacce che abbiamo estratto in precedenza: `Chair`, `Sofa`, `CoffeeTable` e così via.



Pattern Creazionali – Il pattern Abstract Factory – Soluzione

- Ora, cosa dire delle varianti di prodotto? Per ogni variante di una famiglia di prodotti, creiamo una classe factory separata basata sull'interfaccia `AbstractFactory`.
- Una factory è una classe che restituisce prodotti di un particolare tipo.
  - Per esempio, la `ModernFurnitureFactory` può creare solo oggetti `ModernChair`, `ModernSofa` e `ModernCoffeeTable`.
- Il codice del client deve operare sia con le factory che con i prodotti attraverso le rispettive interfacce astratte.
- Questo permette di cambiare il tipo di factory che si passa al codice client, così come la variante di prodotto che il codice client riceve, senza rovinare il codice client vero e proprio.

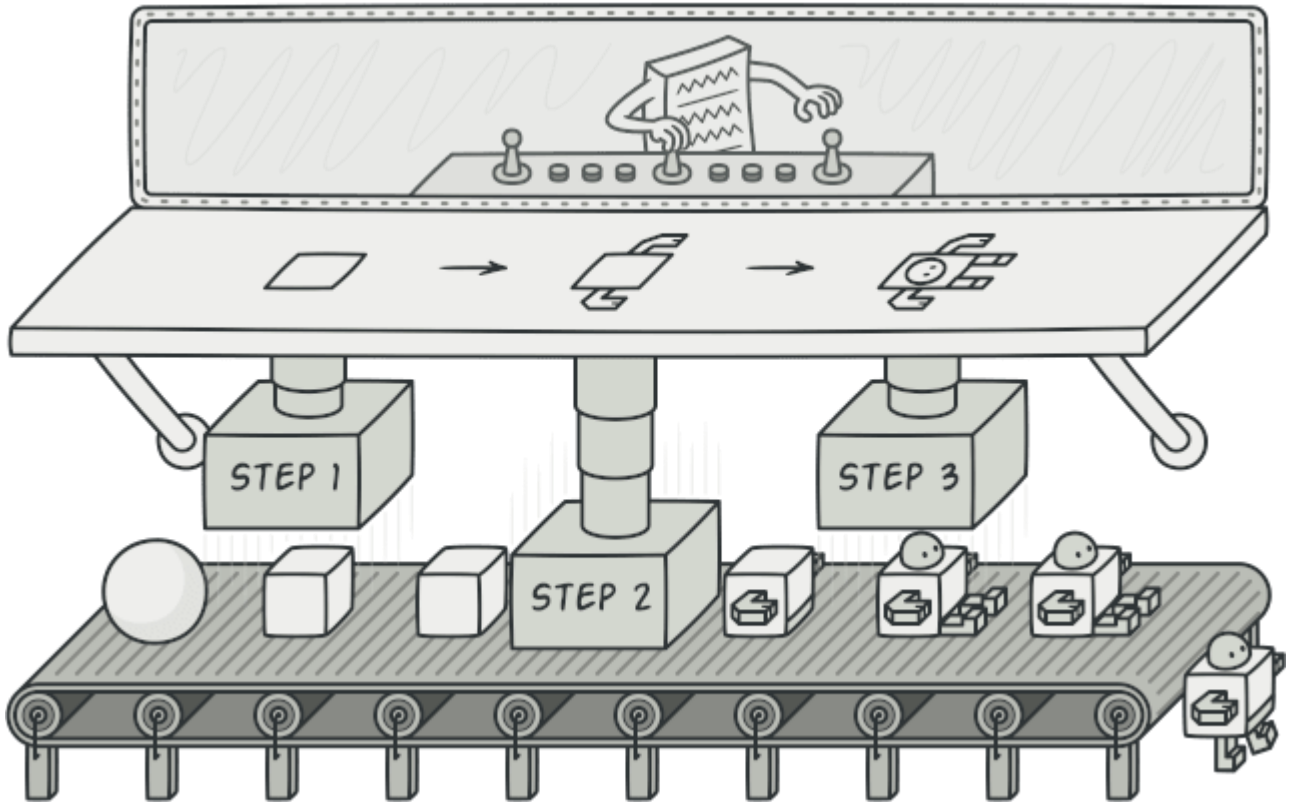


Pattern Creazionali – Il pattern Abstract Factory – Soluzione

- Supponiamo che il client voglia una factory per produrre una sedia.
- Il client non deve essere a conoscenza della classe della factory e non importa che tipo di sedia riceva. Che si tratti di un modello moderno o di una sedia in stile vittoriano, il client deve trattare tutte le sedie allo stesso modo, utilizzando l'interfaccia astratta `Chair`.
- Con questo approccio, l'unica cosa che il client sa della sedia è che implementa in qualche modo il metodo `sitOn`.
- Inoltre, qualunque sia la variante della sedia restituita, essa corrisponderà sempre al tipo di divano o tavolino prodotto dalla stessa fabbrica.
- C'è ancora una cosa da chiarire: se il client vede solo le interfacce astratte, chi crea gli oggetti veri?
- Di solito, l'applicazione crea un oggetto concreto nella fase di inizializzazione. Poco prima dell'inizializzazione, l'applicazione deve selezionare il tipo di factory a seconda della configurazione o delle impostazioni dell'ambiente.

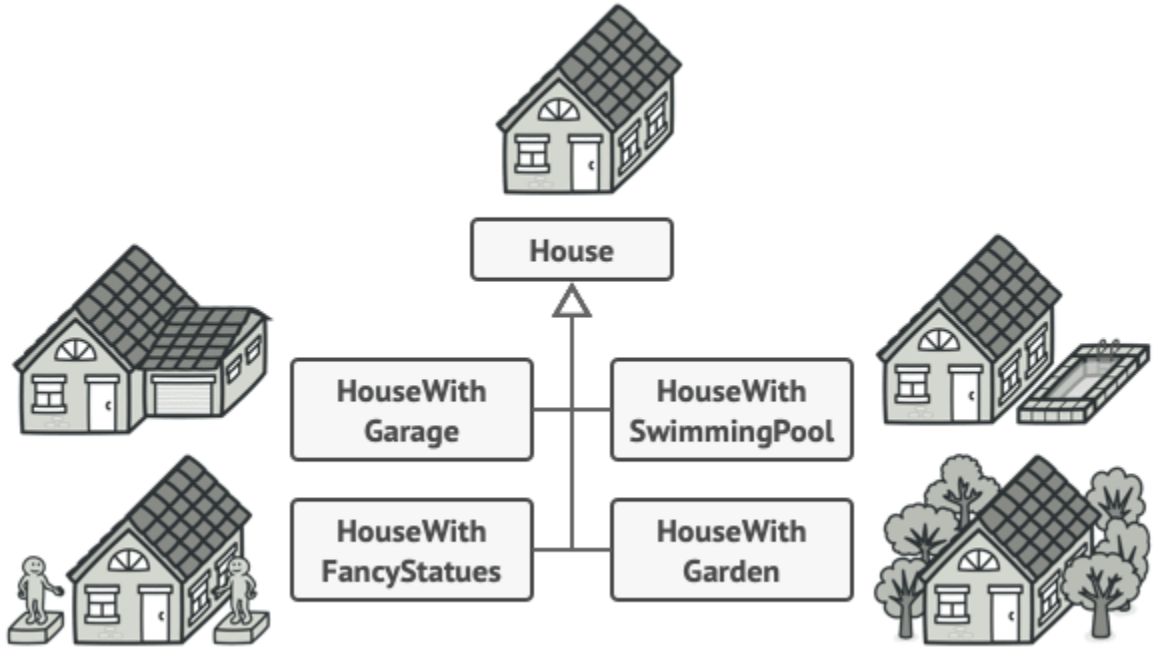
Pattern Creazionali – Il pattern Builder – Intent

- Builder è un design pattern creazionale che permette di costruire oggetti complessi passo dopo passo.
- Il pattern permette di produrre diversi tipi e rappresentazioni di un oggetto utilizzando lo stesso codice per la loro costruzione.



Pattern Creazionali – Il pattern Builder – Problema

- Immaginiamo un oggetto complesso che richiede un'inizializzazione laboriosa e graduale di molti campi e oggetti annidati.
- Tale codice di inizializzazione è di solito sepolto all'interno di un mostruoso costruttore con molti parametri.
- O peggio ancora: sparso in tutto il codice client.

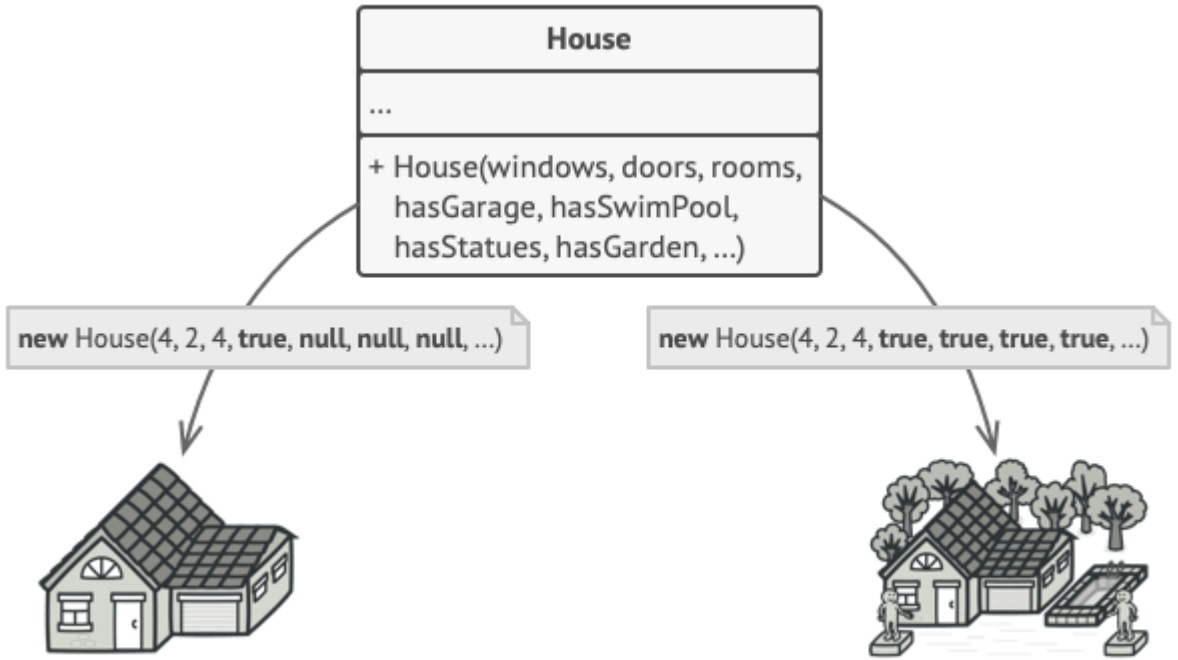


Pattern Creazionali – Il pattern Builder – Problema

- Per esempio, pensiamo a come creare un oggetto `House`.
- Per costruire una casa semplice, è necessario costruire quattro pareti e un pavimento, installare una porta, montare un paio di finestre e costruire un tetto.
- Ma se si vuole una casa più grande, più luminosa, con un cortile e altre comodità (come il riscaldamento, l'impianto idraulico e l'impianto elettrico)?
- La soluzione più semplice è quella di estendere la classe base `House` e creare una serie di sottoclassi per coprire tutte le combinazioni dei parametri.
- Ma alla fine ci si ritroverà con un numero considerevole di sottoclassi.
- Qualsiasi nuovo parametro, come ad esempio lo stile del portico, richiederà di far crescere ancora di più questa gerarchia.
- C'è un altro approccio che non prevede la moltiplicazioni di sottoclassi.
- È possibile creare un costruttore gigante proprio nella classe base `House` con tutti i possibili parametri che controllano l'oggetto casa.

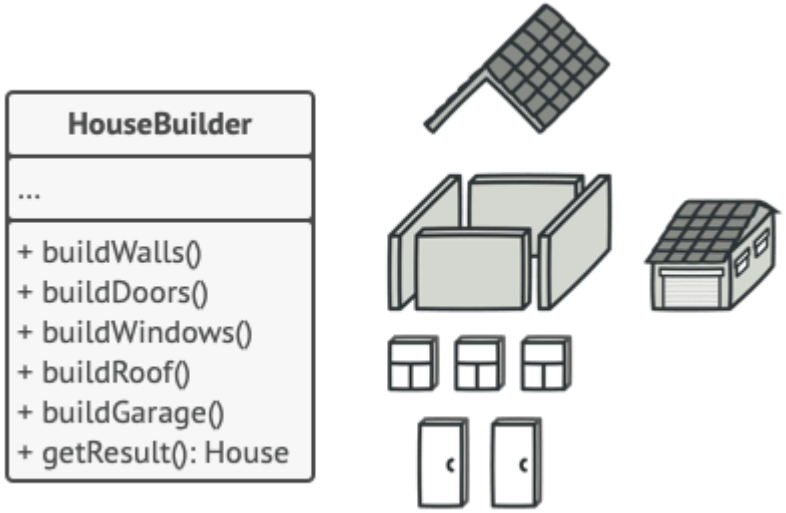
Pattern Creazionali – Il pattern Builder – Problema

- Se da un lato questo approccio elimina la necessità di sottoclassi, dall'altro crea un altro problema.
- Nella maggior parte dei casi la maggior parte dei parametri saranno inutilizzati, rendendo le chiamate del costruttore piuttosto brutte.
- Per esempio, solo una frazione delle case ha una piscina, quindi i parametri relativi alle piscine saranno inutilizzabili nove volte su dieci.



Pattern Creazionali – Il pattern Builder – Soluzione

- Il pattern Builder suggerisce di estrarre il codice di costruzione dell'oggetto dalla propria classe e di spostarlo in oggetti separati chiamati builder.

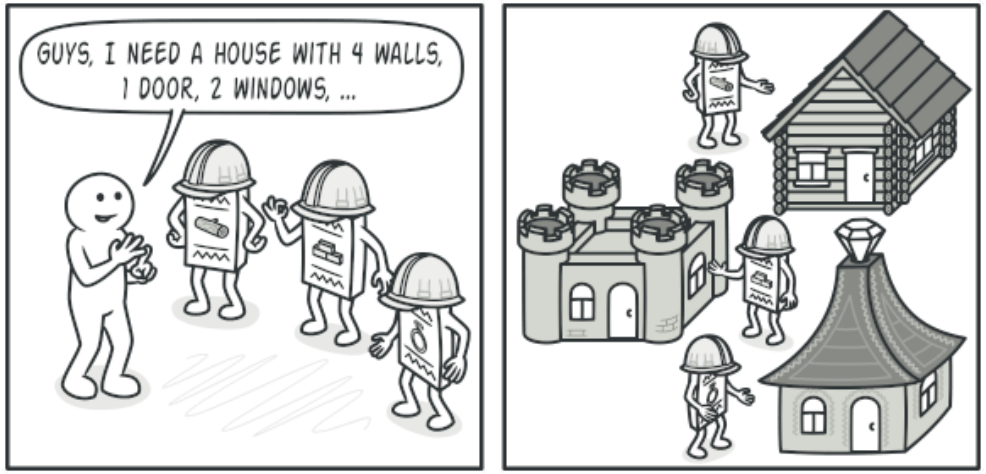


- Il pattern organizza la costruzione dell'oggetto in una serie di passi (`buildWalls`, `buildDoor`, etc.).
- Per creare un oggetto, si esegue una serie di questi passi su un oggetto builder.
- La parte importante è che non è necessario chiamare tutti i passi. Si possono chiamare solo quei passi che sono necessari per produrre una particolare configurazione di un oggetto.

Pattern Creazionali – Il pattern Builder – Soluzione

- Alcune delle fasi della costruzione potrebbero richiedere un'implementazione diversa quando è necessario costruire varie rappresentazioni del prodotto.
- Per esempio, le pareti di una cabina possono essere costruite in legno, ma le mura di un castello devono essere costruite con la pietra.
- In questo caso, è possibile creare diverse classi builder che implementano la stessa serie di fasi di costruzione, ma in modo diverso.
- Poi si possono usare questi builder nel processo di costruzione (cioè un insieme ordinato di chiamate alle fasi di costruzione) per produrre diversi tipi di oggetti.
- Immaginiamo, ad esempio, un builder che costruisce tutto in legno e vetro, un secondo builder che costruisce tutto con pietra e ferro e un terzo builder che usa oro e diamanti.
- Chiamando lo stesso insieme di passi, si ottiene una casa regolare dal primo builder, un piccolo castello dal secondo e una reggia dal terzo.

Pattern Creazionali – Il pattern Builder – Soluzione

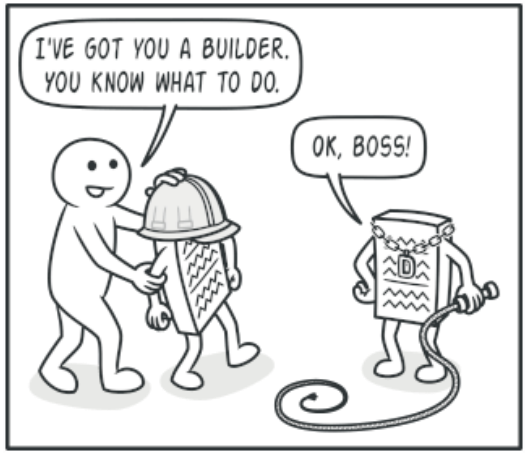


- Tuttavia, questo funzionerebbe solo se il codice client che chiama i passi della costruzione fosse in grado di interagire con i builder utilizzando un'interfaccia comune.

- È possibile andare oltre ed estrarre una serie di chiamate ai passi del costruttore che si utilizzano per costruire un prodotto inserendoli in una classe separata chiamata *director*.

- La classe director definisce l'ordine in cui eseguire le fasi di costruzione, mentre il builder fornisce l'implementazione per tali fasi.

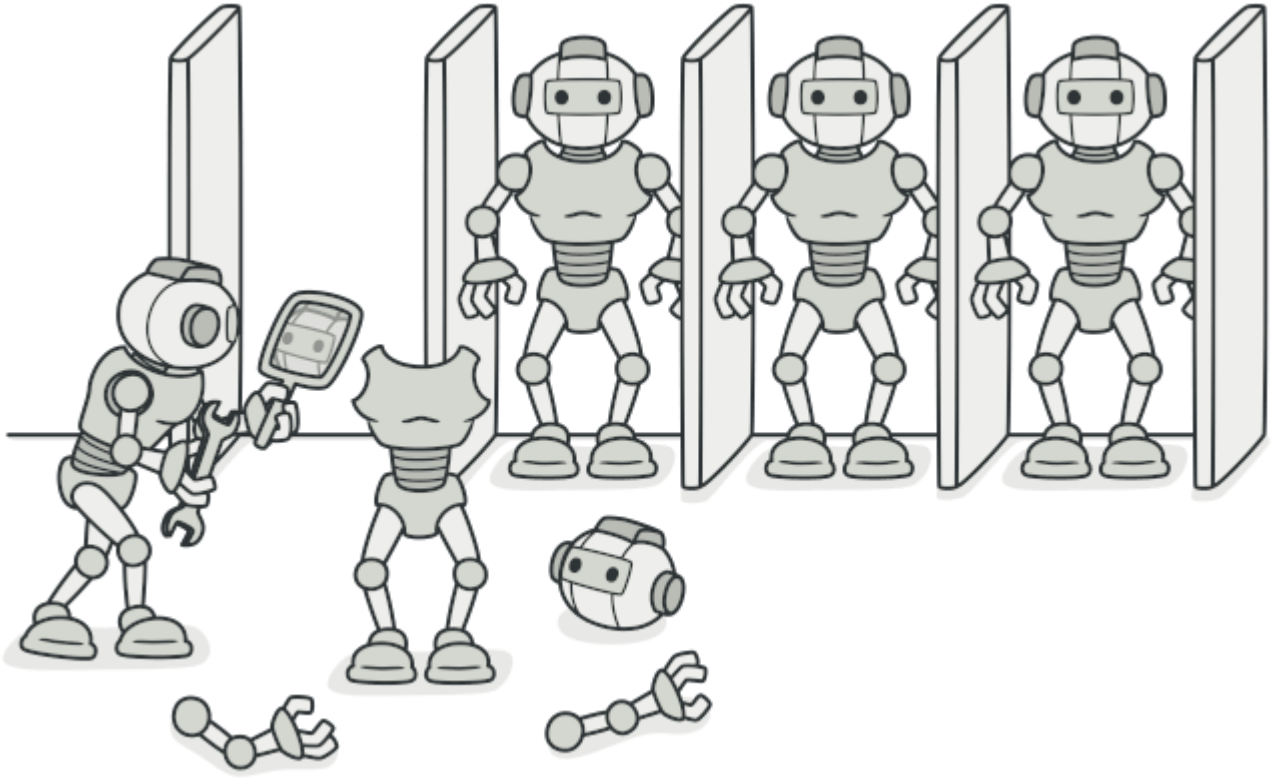
Pattern Creazionali – Il pattern Builder – Soluzione



- Avere una classe director nel nostro programma non è strettamente necessario.
- È sempre possibile chiamare le fasi di costruzione in un ordine specifico direttamente dal codice cliente.
- Tuttavia, la classe director potrebbe essere un buon posto per inserire varie routine di costruzione in modo da poterle riutilizzare nel nostro programma.
- Inoltre, la classe director nasconde completamente i dettagli della costruzione del prodotto al codice client.
- Il client deve solo associare un builder ad un director, lanciare la costruzione con il director e ottenere il risultato dal builder.

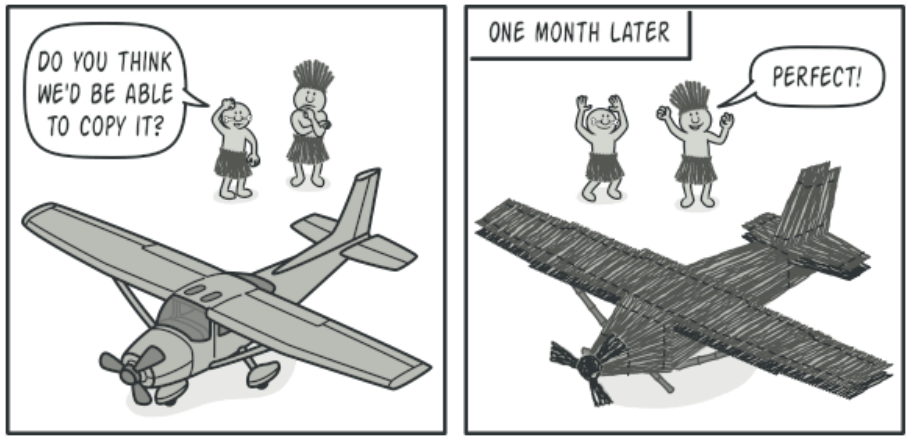
Pattern Creazionali – Il pattern Prototype – Intent

- Prototype è un design pattern creazionale che permette di copiare oggetti esistenti senza rendere il codice dipendente dalle loro classi.



Pattern Creazionali – Il pattern Prototype – Problema

- Supponiamo di avere un oggetto e di voler creare una copia esatta.
- Come lo faremmo?
- Per prima cosa, dobbiamo creare un nuovo oggetto della stessa classe.
- Poi dobbiamo considerare tutti i campi dell'oggetto originale e copiare i loro valori sul nuovo oggetto.
- Bello! Ma c'è un inconveniente.
- Non tutti gli oggetti possono essere copiati in questo modo perché alcuni dei campi dell'oggetto possono essere privati e non visibili dall'esterno dell'oggetto stesso.



Pattern Creazionali – Il pattern Prototype – Problema

- C'è un altro problema con l'approccio diretto.
- Dal momento che si deve conoscere la classe dell'oggetto per creare un duplicato, il codice diventa dipendente da quella classe.
- Se la dipendenza extra non è sufficiente a spaventarci, c'è un altro problema.
- A volte si conosce solo l'interfaccia che l'oggetto segue, ma non la sua classe concreta, quando, per esempio, un parametro in un metodo accetta qualsiasi oggetto che segue una qualche interfaccia.

Pattern Creazionali – Il pattern Prototype – Soluzione

- Il pattern Prototype delega il processo di clonazione agli oggetti reali che vengono clonati.
- Il pattern dichiara un'interfaccia comune per tutti gli oggetti che supportano la clonazione.
- Questa interfaccia permette di clonare un oggetto senza accoppiare il codice alla classe di quell'oggetto.
- Di solito, tale interfaccia contiene solo un singolo metodo `clone`.
- L'implementazione del metodo `clone` è molto simile in tutte le classi.
- Il metodo crea un oggetto della classe corrente e porta tutti i valori dei campi del vecchio oggetto in quello nuovo.
- Si possono anche copiare campi privati perché la maggior parte dei linguaggi di programmazione permette agli oggetti di accedere a campi privati di altri oggetti che appartengono alla stessa classe.
- Un oggetto che supporta la clonazione è chiamato *prototype*.
- Quando i nostri oggetti hanno decine di campi e centinaia di configurazioni possibili, la clonazione potrebbe servire come alternativa alla sottoclasse.

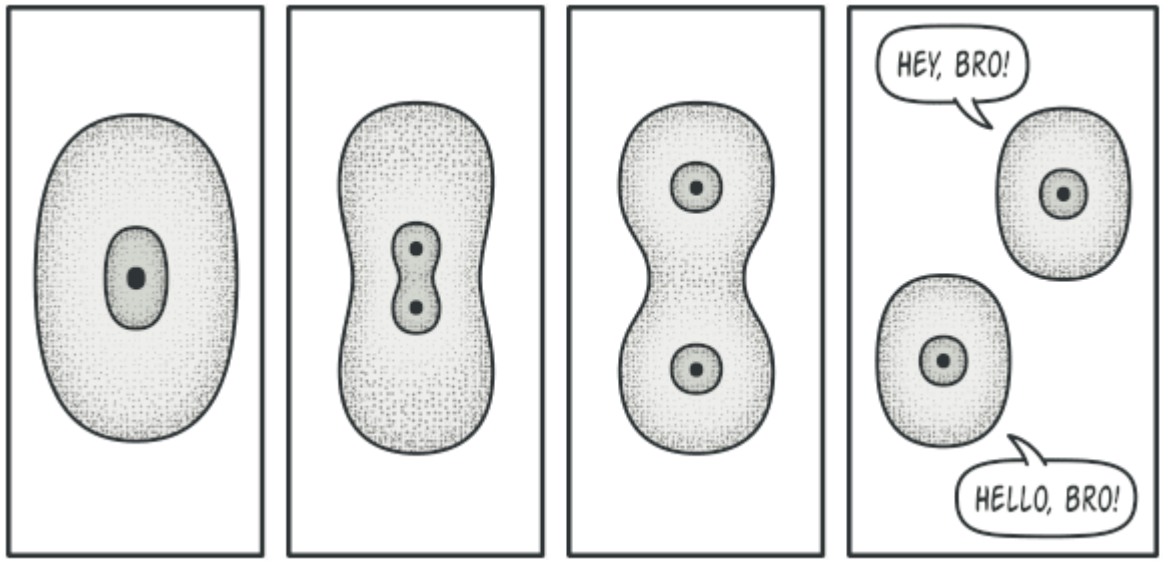
Pattern Creazionali – Il pattern Prototype – Soluzione



- Ecco come funziona: si crea un insieme di oggetti, configurati in vari modi.
- Quando si ha bisogno di un oggetto come quello configurato, basta clonare un prototipo invece di costruire un nuovo oggetto da zero.
- Nella vita reale, i prototipi vengono utilizzati per eseguire vari test prima di iniziare la produzione di massa di un prodotto.

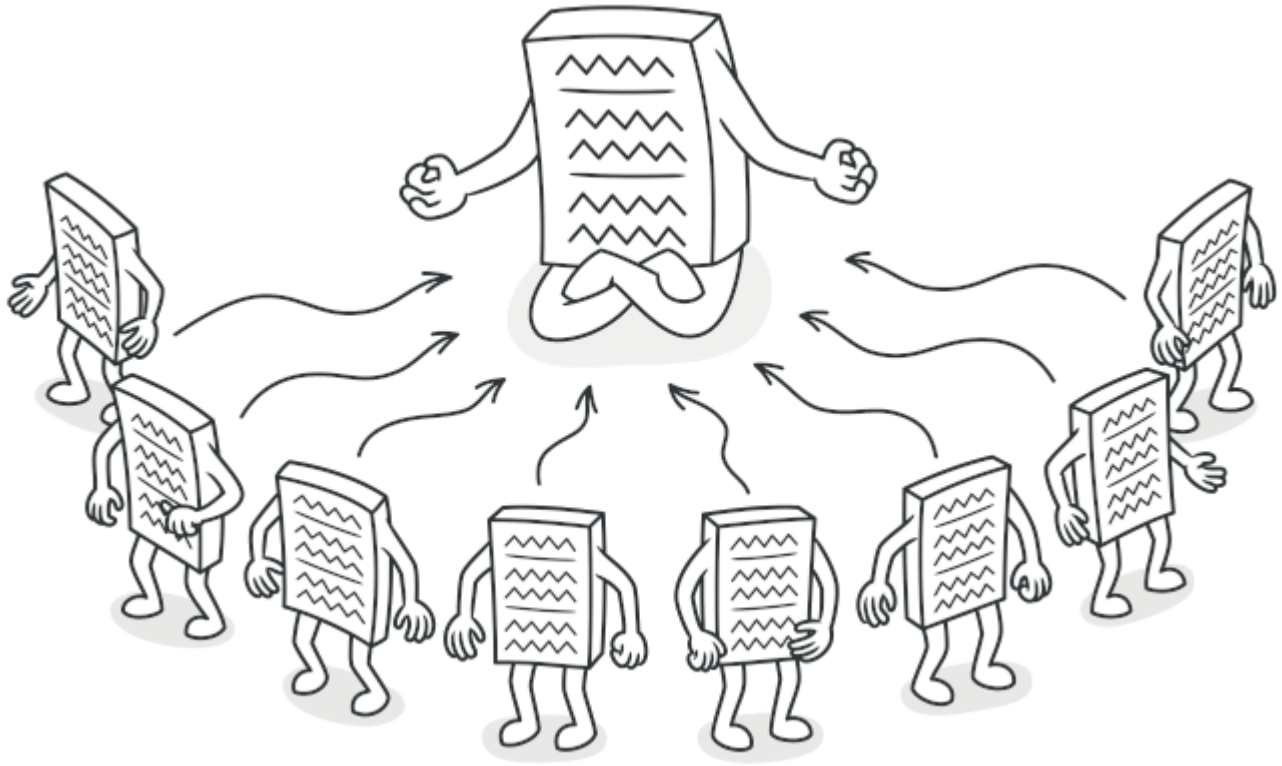
Pattern Creazionali – Il pattern Prototype – Soluzione

- Tuttavia, in questo caso, i prototipi non partecipano ad alcuna produzione effettiva, svolgendo invece un ruolo passivo.
- Poiché i prototipi industriali non si copiano da soli, un'analogia molto più vicina al modello è il processo di divisione delle cellule per mitosi (biologia, ricordate?).
- Dopo la divisione mitosi, si forma una coppia di cellule identiche.
- La cellula originale funge da prototipo e assume un ruolo attivo nella creazione della copia.



Pattern Creazionali – Il pattern Singleton – Intent

- Singleton è un design pattern creazionale che consente di garantire che una classe abbia una sola istanza, fornendo al contempo un punto di accesso globale a questa istanza.



Pattern Creazionali – Il pattern Singleton – Problema

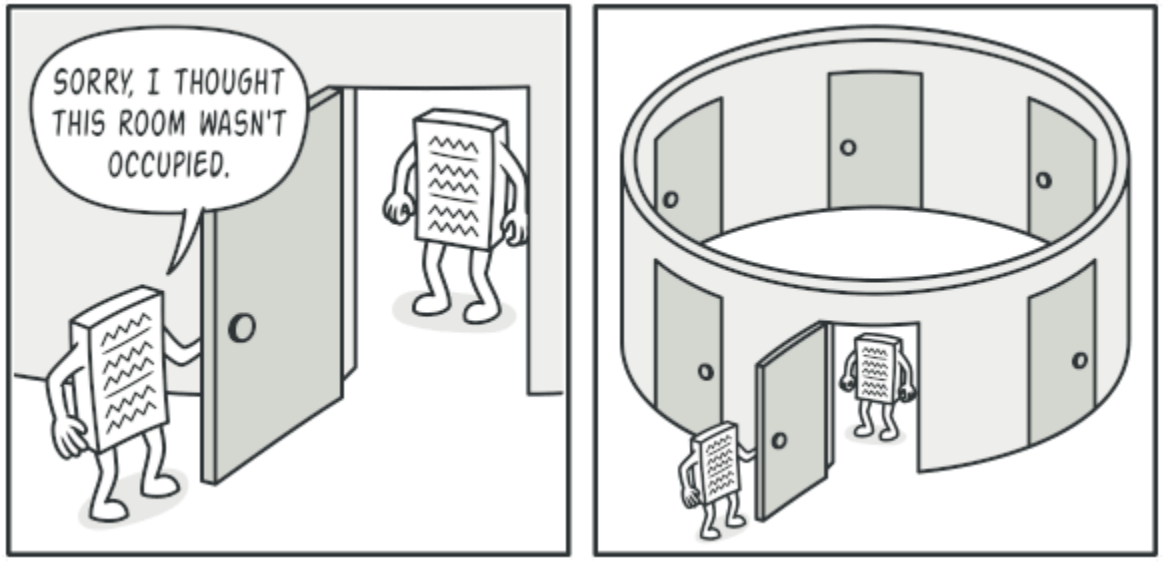
- Il pattern Singleton risolve contemporaneamente due problemi violando, in questo modo, il principio della responsabilità unica.
- Assicurarsi che una classe abbia una sola istanza:
  - Perché qualcuno dovrebbe voler controllare quante istanze ha una classe?
  - La ragione più comune è controllare l'accesso a qualche risorsa condivisa, per esempio un database o un file.
  - Ecco come funziona: immaginiamo di aver creato un oggetto, ma dopo un po' di tempo abbiamo deciso di crearne uno nuovo.
  - Invece di ricevere un nuovo oggetto, otterremo quello già creato.
  - Si noti che questo comportamento è impossibile da implementare con un normale costruttore, poiché una chiamata al costruttore deve sempre restituire un nuovo oggetto per progetto.
- Fornire un punto di accesso globale a tale istanza.
  - Ci sarà capitato più di una volta di avere usato delle variabili globali per memorizzare alcuni oggetti essenziali.

Pattern Creazionali – Il pattern Singleton – Problema

- Tali variabili sono tanto utili quanto insicure, poiché qualsiasi codice può in linea di principio sovrascrivere il loro contenuto e mandare in crash l'applicazione.

- Proprio come una variabile globale, il pattern Singleton consente di accedere ad alcuni oggetti da qualsiasi punto del programma.

- Tuttavia, protegge anche quell'istanza da essere sovrascritta da altro codice.



Pattern Creazionali – Il pattern Singleton – Problema

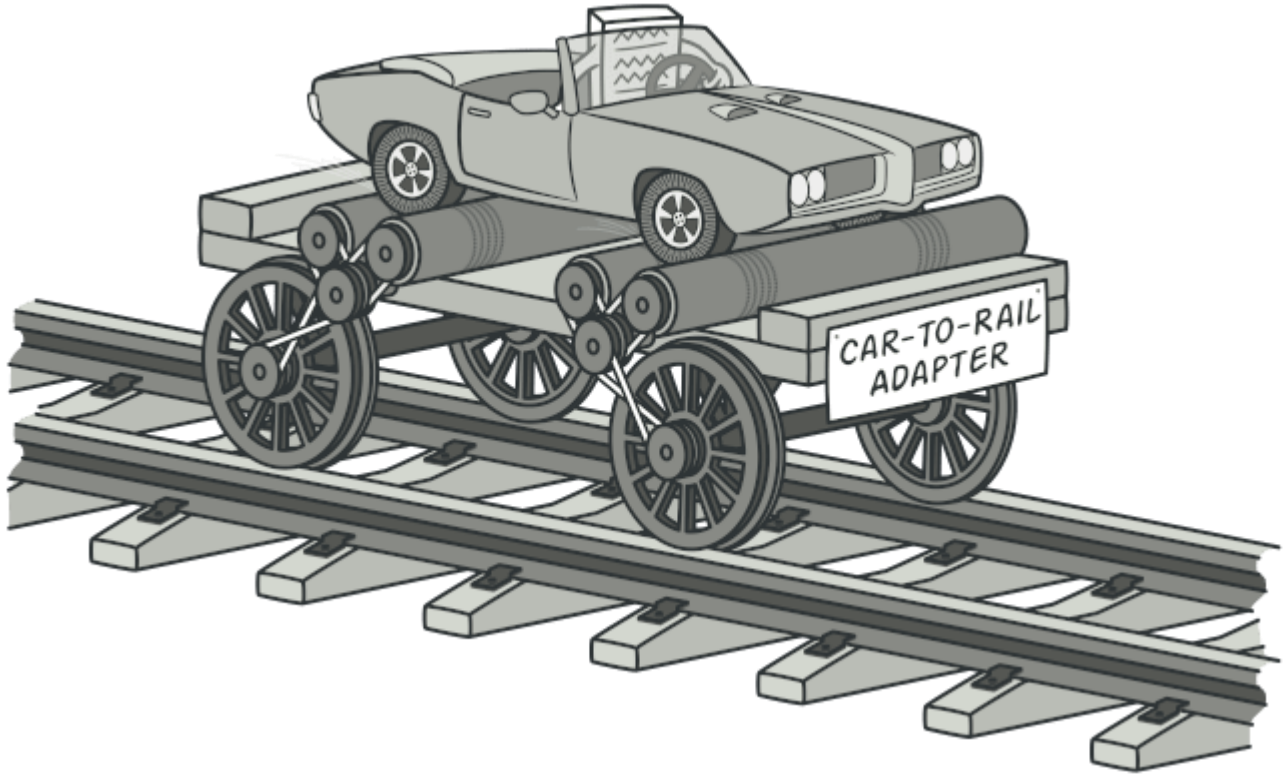
- C'è un altro aspetto interessante di questo problema: non si vuole che il codice che risolve il primo dei due problemi menzionato sopra sia sparso in tutto il programma.
    - È molto meglio averlo all'interno di una classe, specialmente se il resto del nostro codice dipende già da esso.
- Al giorno d'oggi, il pattern Singleton è diventato così popolare che la gente può chiamare qualcosa singleton anche se risolve solo uno dei problemi elencati.

Pattern Creazionali – Il pattern Singleton – Soluzione

- Tutte le implementazioni del pattern Singleton hanno questi due passi in comune:
  - Rendere il costruttore di default privato, per evitare che altri oggetti utilizzino l'operatore `new` con la classe Singleton.
  - Creare un metodo di creazione statica che funga da costruttore.
    - Questo metodo chiama il costruttore privato per creare un oggetto e salvarlo in un campo statico.
    - Tutte le successive chiamate a questo metodo restituiscono l'oggetto in cache.
- Se il codice ha accesso alla classe Singleton, allora è in grado di chiamare il metodo statico di Singleton.
- Così, ogni volta che quel metodo viene chiamato, viene sempre restituito lo stesso oggetto.
- Il governo è un eccellente esempio del modello di Singleton.
- Un paese può avere un solo governo ufficiale. Indipendentemente dall'identità personale degli individui che formano i governi, il titolo, "Il governo di X", è un punto di accesso globale che identifica il gruppo di persone che in quel momento hanno la responsabilità dei ministeri.

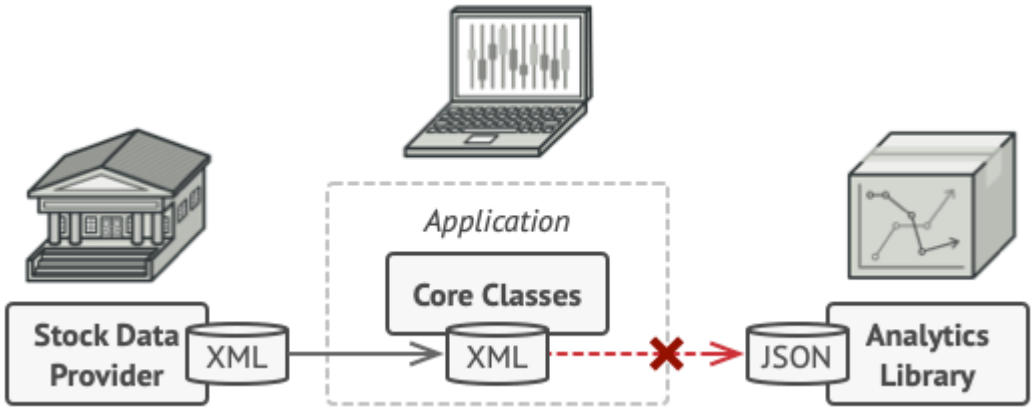
Pattern Strutturali – Il pattern Adapter – Intent

- Adapter è un design pattern strutturale che consente la collaborazione di oggetti con interfacce incompatibili.



Pattern Strutturali – Il pattern Adapter – Problema

- Immaginiamo di creare un'applicazione per il monitoraggio del mercato azionario.
- L'app scarica i dati azionari da più fonti in formato XML e poi visualizza grafici e diagrammi secondo una rappresentazione amichevole per l'utente.
- Ad un certo punto, decidiamo di migliorare l'app integrando una libreria di analisi intelligente di terze parti.
- Ma c'è un problema: la libreria di analisi funziona solo con dati in formato JSON.



Pattern Strutturali – Il pattern Adapter – Problema

- Si potrebbe cambiare la libreria per lavorare con XML.
- Tuttavia, questo potrebbe rendere inutilizzabile un po' di codice esistente che si basa sulla libreria.
- E peggio ancora, si potrebbe non avere del tutto accesso al codice sorgente della libreria, rendendo questo approccio impossibile.

Pattern Strutturali – Il pattern Adapter – Soluzione

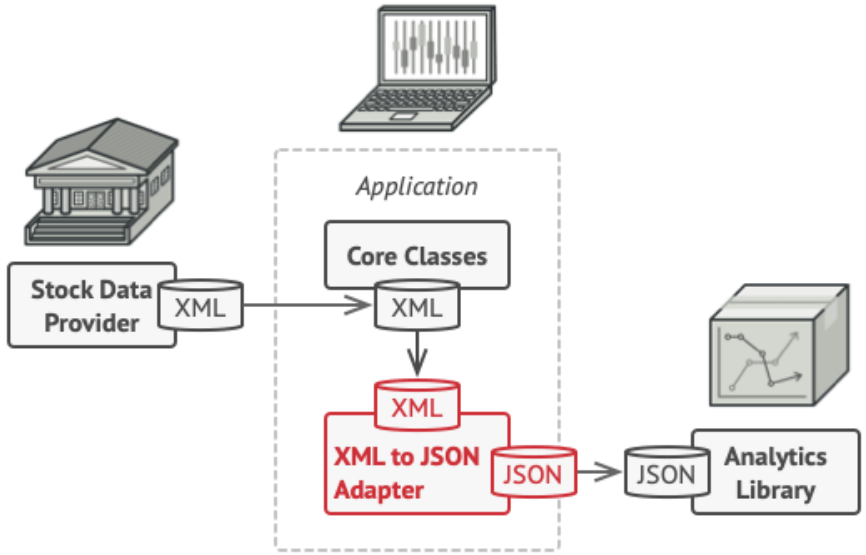
- È possibile creare un adattatore.
- Si tratta di un oggetto speciale che converte l'interfaccia di un oggetto in modo che un altro oggetto possa capirla.
- Un adattatore maschera uno degli oggetti per nascondere la complessità della conversione che avviene dietro le quinte.
- L'oggetto mascherato non è nemmeno consapevole dell'adattatore.
- Per esempio, si può mascherare un oggetto che opera in metri e chilometri con un adattatore che converte tutti i dati in unità imperiali come piedi e miglia.
- Gli adattatori possono non solo convertire i dati in vari formati, ma possono anche aiutare gli oggetti caratterizzati da interfacce differenti a collaborare.

Pattern Strutturali – Il pattern Adapter – Soluzione

- Ecco come funziona il meccansimo:
  - L'adattatore definisce un'interfaccia compatibile con uno degli oggetti esistenti.
  - Utilizzando questa interfaccia, l'oggetto esistente può chiamare in modo sicuro i metodi dell'adattatore.
  - Alla ricezione di una chiamata, l'adattatore passa la richiesta al secondo oggetto, ma in un formato e in un ordine che quest'ultimo si aspetta.
- A volte è anche possibile creare un adattatore bidirezionale in grado di convertire le chiamate in entrambe le direzioni.
- Torniamo alla nostra app per il mercato azionario.
- Per risolvere il dilemma dei formati incompatibili, possiamo creare adattatori XML-to-JSON per ogni classe della libreria di analisi con cui il nostro codice lavora direttamente.
- Poi aggiustiamo il nostro codice per comunicare con la libreria solo attraverso questi adattatori.
- Quando un adattatore riceve una chiamata, traduce i dati XML in arrivo in una struttura JSON e passa la chiamata ai metodi appropriati di un oggetto di analisi mascherato.

Pattern Strutturali – Il pattern Adapter – Soluzione

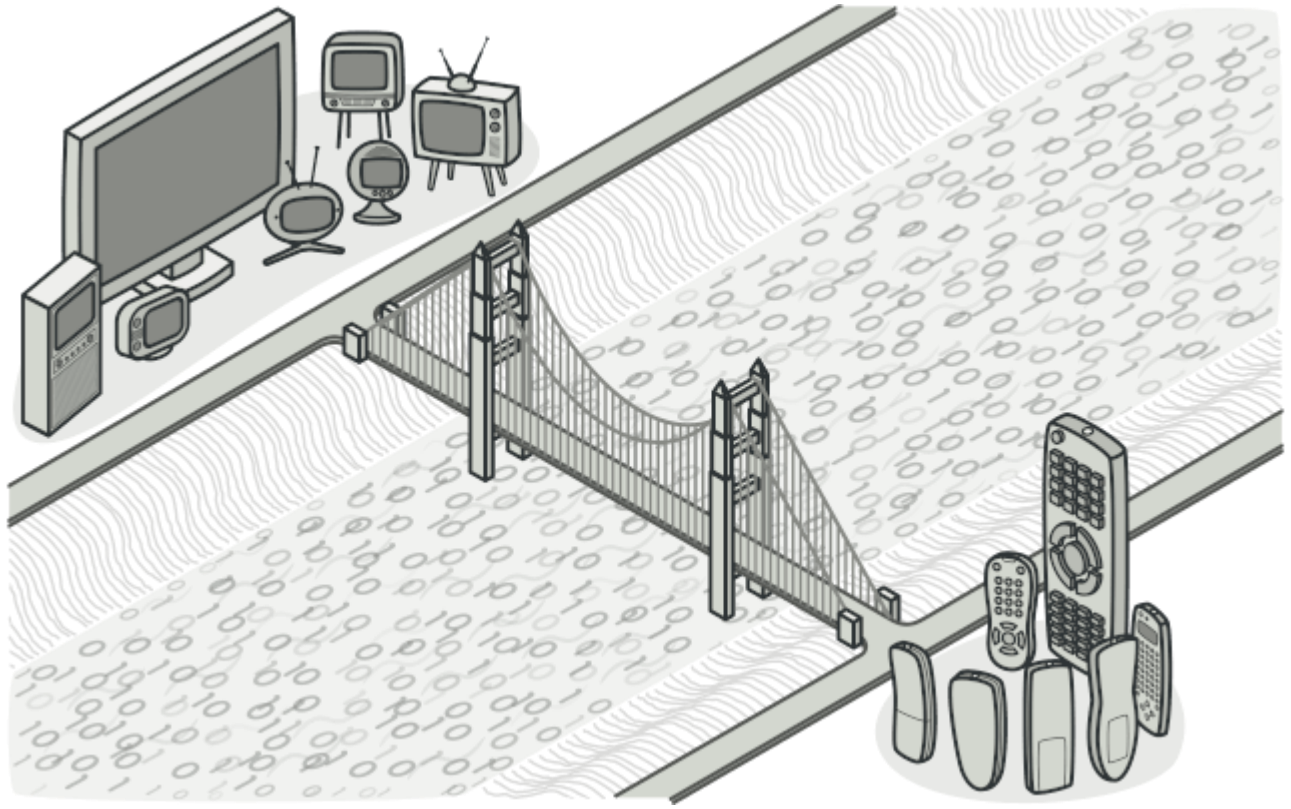
- La seguente figura illustra questo procedimento:



- Gli adattatori non esistono solo in informatica; un esempio pratico sono gli adattatori utilizzati nelle prese elettroniche o nei dispositivi a supporto dei computer.

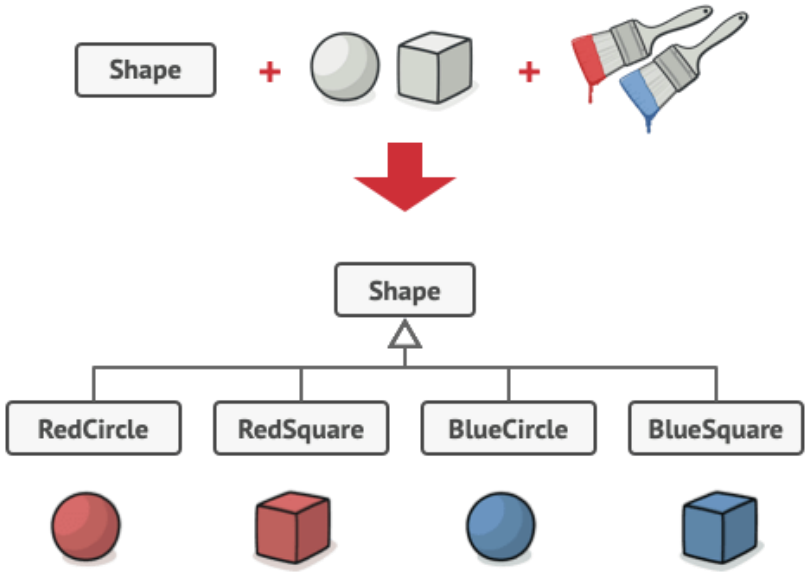
Pattern Strutturali – Il pattern Bridge – Intent

- Bridge è un design pattern strutturale che consente di dividere una grande classe o un insieme di classi strettamente correlate in due distinte gerarchie - astrazione e implementazione - che possono essere sviluppate indipendentemente l'una dall'altra.



Pattern Strutturali – Il pattern Bridge – Problema

- Per capire come funziona questo pattern consideriamo un semplice esempio.
- Supponiamo di avere una classe `Shape` con un paio di sottoclassi: `Circle` e `Square`.
- Vogliamo estendere questa gerarchia di classi per incorporare i colori, quindi prevediamo di creare sottoclassi di Shape che tengano conto dei colori.
- Tuttavia, poiché abbiamo già due sottoclassi di `Shape`, sarà necessario creare quattro combinazioni di sottoclassi come `BlueCircle` e `RedSquare`.

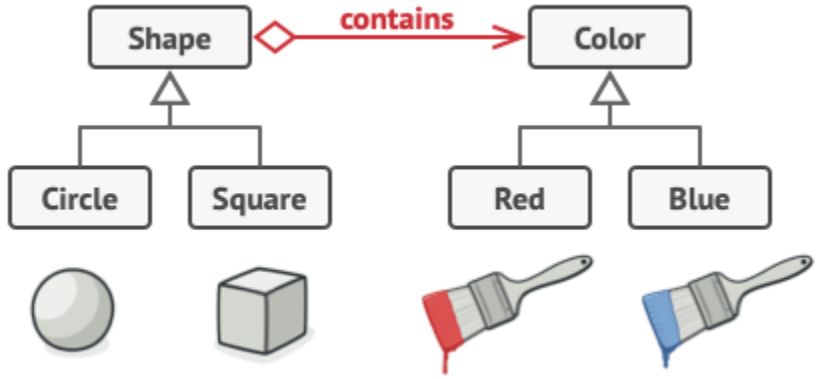


Pattern Strutturali – Il pattern Bridge – Problema

- L'aggiunta di nuovi tipi di forma e colori alla gerarchia farà crescere quest'ultima in modo esponenziale.
- Ad esempio, per aggiungere una forma triangolare sarebbe necessario introdurre due sottoclassi, una per ogni colore.
- E dopo di ciò, l'aggiunta di un nuovo colore richiederebbe la creazione di tre sottoclassi, una per ogni tipo di forma.
- Più si va avanti, peggio diventa.

Pattern Strutturali – Il pattern Bridge – Soluzione

- Questo problema si verifica perché stiamo cercando di estendere le classi relative alle forme lungo due dimensioni indipendenti: per forma e per colore.
- Questo è un problema molto comune con l'eredità delle classi.
- Il pattern *Bridge* tenta di risolvere questo problema passando dall'ereditarietà alla composizione degli oggetti.
- Ciò significa che si estrae una delle dimensioni in una gerarchia di classi separata, in modo che le classi originali facciano riferimento a un oggetto della nuova gerarchia, invece di avere tutti i suoi stati e comportamenti all'interno di una classe.



Pattern Strutturali – Il pattern Bridge – Soluzione

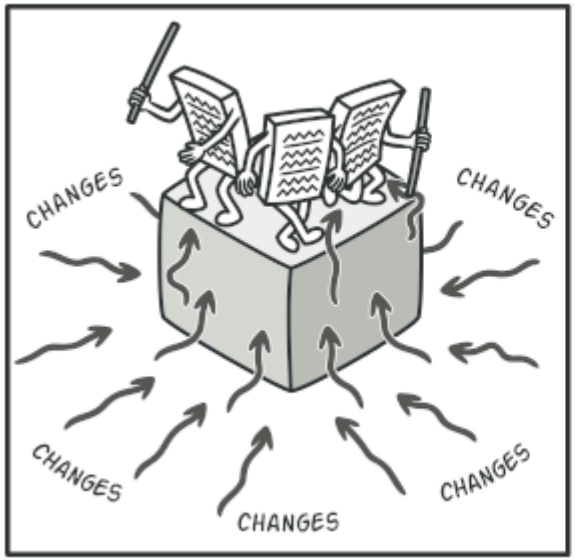
- Seguendo questo approccio, possiamo estrarre il codice relativo al colore in una sua classe con due sottoclassi: `Red` e `Blue`.
- La classe `Shape` ottiene quindi un campo riferimento che punta a uno degli oggetti colore.
- Ora la forma può delegare qualsiasi operazione relativa al colore all'oggetto colore collegato.
- Tale riferimento fungerà da ponte tra le classi `Shape` e `Color`.
- D'ora in poi, l'aggiunta di nuovi colori non richiederà di cambiare la gerarchia delle forme, e viceversa.

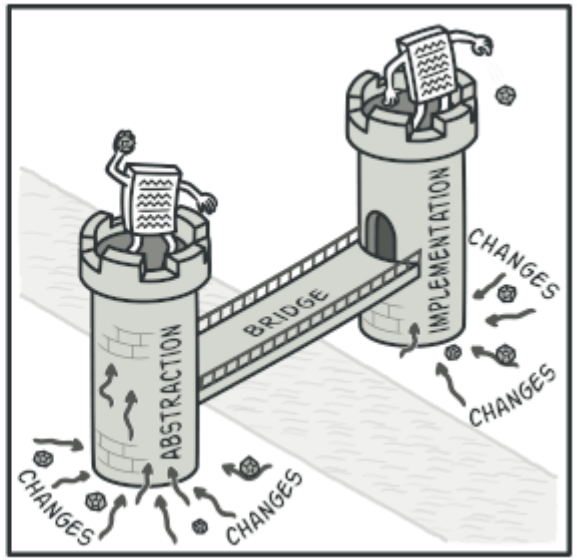
Pattern Strutturali – Il pattern Bridge – Soluzione – Astrazione e implementazione

- Il libro del GoF introduce i termini *Astrazione* e *Implementazione* come parte della definizione del Bridge.
- I termini possono suonare troppo accademici e fanno sembrare il pattern più complicato di quanto non sia in realtà.
- Dopo aver letto il semplice esempio relativo alle forme e ai colori, decifriamo il significato dietro le parole oscure del libro GoF.
- *L'astrazione* (chiamata anche *interfaccia*) è uno strato di controllo ad alto livello per alcune entità.
- Questo strato non dovrebbe fare alcun lavoro reale da solo.
- Esso dovrebbe delegare il lavoro al livello di *implementazione* (chiamato anche *piattaforma*).
- Si noti che non stiamo parlando delle interfacce e delle classi astratte dei linguaggio di programmazione. Queste non sono le stesse cose, anche se, per molti versi, lo ricordano.
- Quando si parla di applicazioni reali, l'astrazione può essere rappresentata da una Graphical User Interface (GUI), e l'implementazione potrebbe essere il codice del sistema operativo sottostante (API) che il livello GUI chiama in risposta alle interazioni dell'utente.

Pattern Strutturali – Il pattern Bridge – Soluzione – Astrazione e implementazione

- In generale, è possibile estendere tale app in due direzioni indipendenti:
  - Avere diverse GUI differenti (ad esempio, su misura per i clienti abituali o gli amministratori).
  - Supportare diverse API differenti (ad esempio, per poter lanciare l'app sotto Windows, Linux e MacOS).
- Nel peggiore dei casi, questa app potrebbe apparire come una gigantesca ciotola di spaghetti, dove centinaia di condizionamenti collegano diversi tipi di GUI con varie API in tutto il codice.





Pattern Strutturali – Il pattern Bridge – Soluzione – Astrazione e implementazione

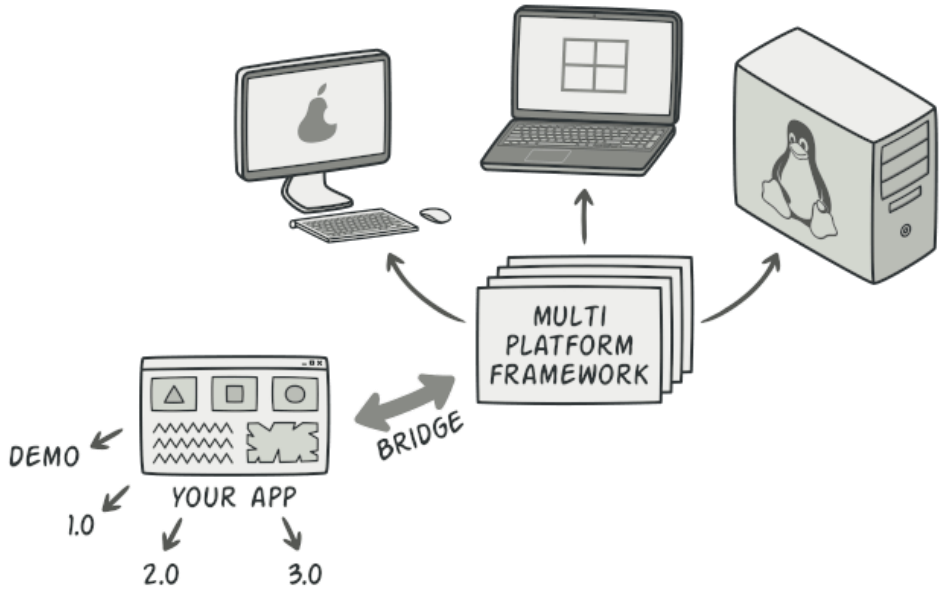
- Si può portare ordine in questo caos estraendo il codice relativo a specifiche combinazioni di interfaccia-piattaforma in classi separate.
- Tuttavia, presto scopriremo che ci sono molte di queste classi.
- La gerarchia delle classi crescerà in modo esponenziale perché l'aggiunta di una nuova GUI o il supporto di una diversa API richiederebbe la creazione di un numero sempre maggiore di classi.
- Cerchiamo di risolvere questo problema con il pattern Bridge.
- Esso suggerisce di dividere le classi in due gerarchie:
  - Astrazione: il livello GUI dell'applicazione.
  - Implementazione: le API dei sistemi operativi.
- L'oggetto relativo all'astrazione controlla l'aspetto dell'applicazione, delegando il lavoro effettivo all'oggetto relativo all'implementazione collegato.

Pattern Strutturali – Il pattern Bridge – Soluzione – Astrazione e implementazione

- Le diverse implementazioni sono intercambiabili fintanto che seguono un'interfaccia comune, permettendo alla stessa GUI di lavorare sotto Windows e Linux.

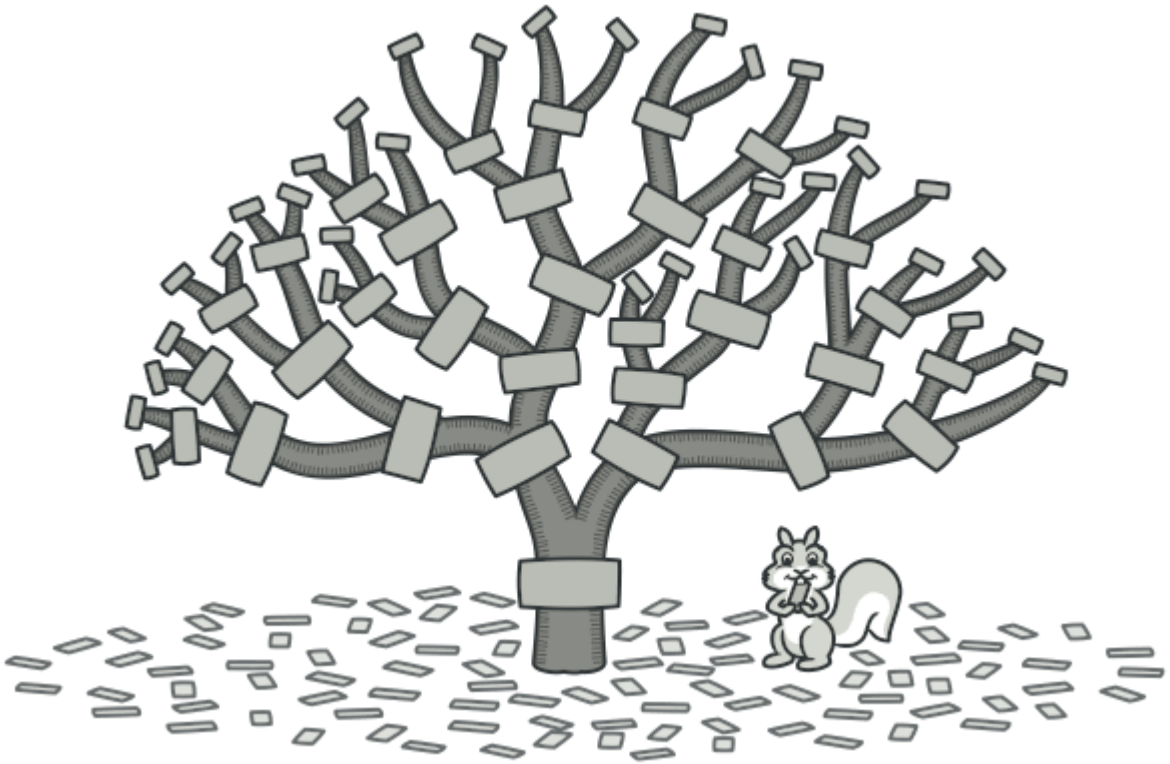
- Di conseguenza, è possibile modificare le classi GUI senza toccare le classi relative alle API.

- Inoltre, l'aggiunta del supporto per un altro sistema operativo richiede solo la creazione di una sottoclasse nella gerarchia di implementazione.



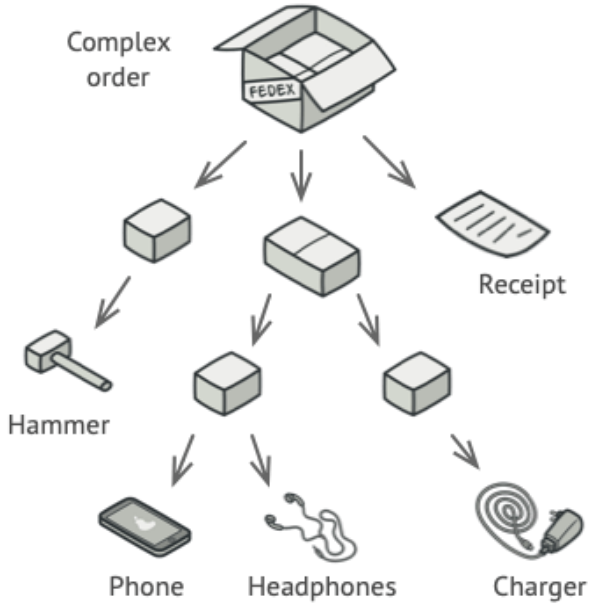
Pattern Strutturali – Il pattern Composite – Intent

- Composite è un design pattern strutturale che consente di comporre gli oggetti in strutture ad albero e poi lavorare con queste strutture come se fossero singoli oggetti.



Pattern Strutturali – Il pattern Composite – Problema

- L'utilizzo del pattern Composite ha senso solo quando il modello di base della nostra app può essere rappresentato come un albero.
- Per esempio, immaginiamo di avere due tipi di oggetti: `Products` e `Boxes`.
- Una `Box` può contenere diversi `Products` così come una serie di `Boxes` più piccole.
- Anche queste `Boxes` piccole possono contenere alcuni `Products` o anche `Boxes` più piccole, e così via.



Pattern Strutturali – Il pattern Composite – Problema

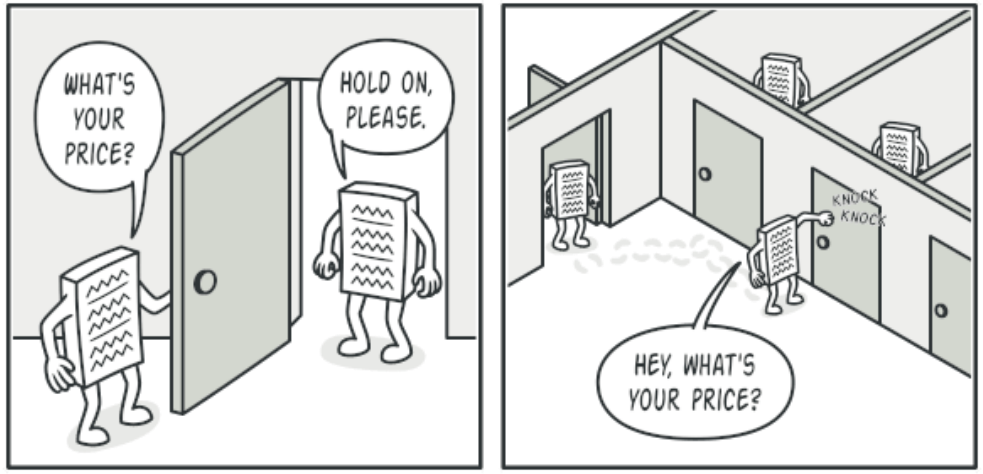
- Supponiamo che decidiamo di creare un sistema di ordinazione che utilizzi queste classi.
- Gli ordini potrebbero contenere prodotti semplici senza alcun involucro, così come scatole piene di prodotti... e altre scatole.
- Come si determina il prezzo totale di un ordine di questo tipo?
- Si potrebbe provare l'approccio diretto: scartare tutte le scatole, esaminare tutti i prodotti e poi calcolare il totale.
- Questo sarebbe fattibile nel mondo reale; ma in un programma non è così semplice come eseguire un ciclo.
- Bisogna conoscere in anticipo le classi di `Products` e `Boxes` che si stanno esaminando, il livello di nidificazione delle scatole e altri dettagli spiacevoli.
- Tutto questo rende l'approccio diretto troppo scomodo o addirittura impossibile.

Pattern Strutturali – Il pattern Composite – Soluzione

- Il pattern Composite suggerisce di lavorare con i `Products` e le `Boxes` attraverso un'interfaccia comune che dichiara un metodo per il calcolo del prezzo totale.
- Come funzionerebbe questo metodo?
- Per un prodotto, semplicemente restituirebbe il prezzo del prodotto.
- Per una scatola, andrebbe ad ogni articolo che la scatola contiene, chiederebbe il suo prezzo e poi restituirebbe un totale per questa scatola.
- Se uno di questi articoli fosse una scatola più piccola, anche quella scatola inizierebbe a esaminare il suo contenuto e così via, fino a calcolare il prezzo di tutti i componenti interni.
- Una scatola potrebbe anche aggiungere qualche costo aggiuntivo al prezzo finale, come il costo dell'imballaggio.
- Il più grande vantaggio di questo approccio è che non è necessario preoccuparsi delle classi di oggetti concreti che compongono l'albero.
- Non è necessario sapere se un oggetto è un prodotto semplice o una scatola sofisticata.

Pattern Strutturali – Il pattern Composite – Soluzione

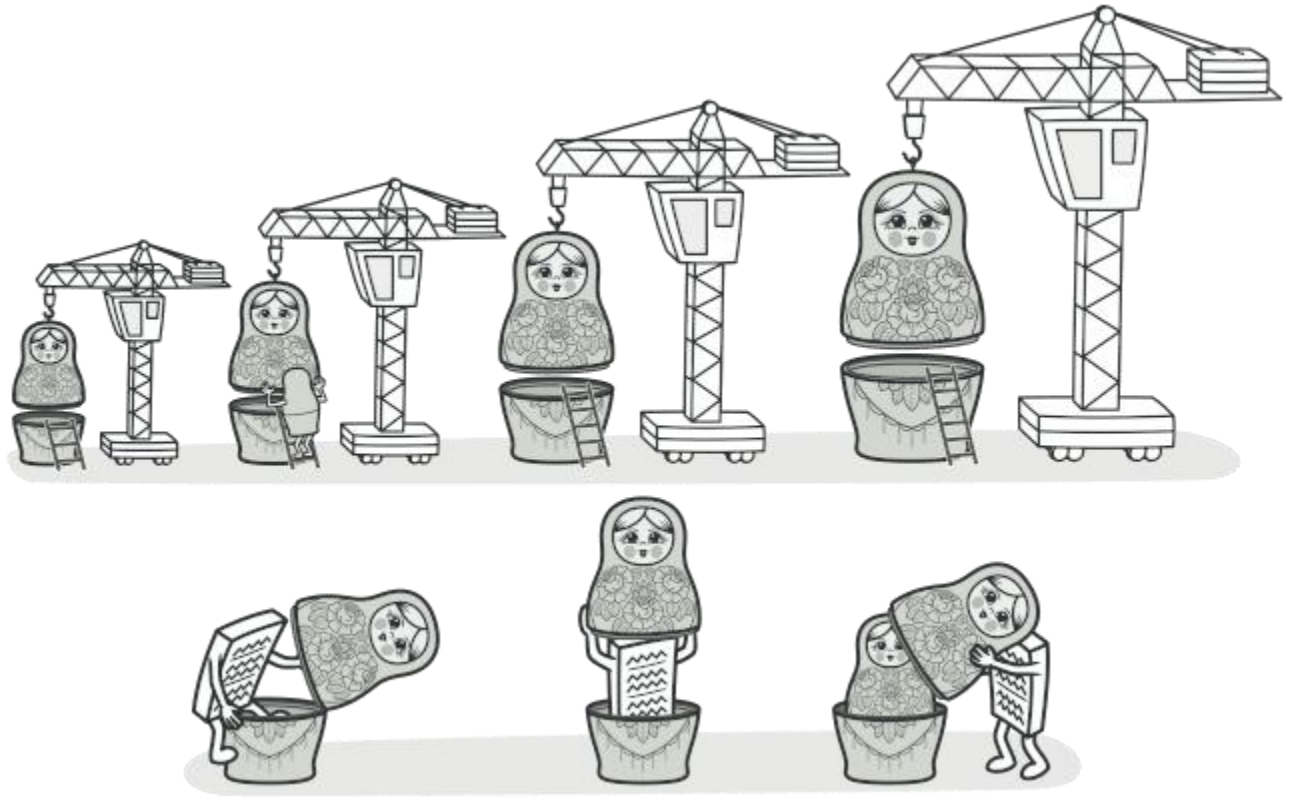
- Possiamo trattarli tutti allo stesso modo attraverso l'interfaccia comune.
- Quando si chiama un metodo, gli oggetti stessi trasmettono la richiesta all'albero.



- Un parallelo con il mondo reale si ha con gli eserciti, che sono strutturati come gerarchie.
- Un esercito è composto da diverse divisioni; una divisione è un insieme di brigate, e una brigata è composta da plotoni, che possono essere suddivisi in squadre. Infine, una squadra è un piccolo gruppo di soldati.
- Gli ordini vengono dati in cima alla gerarchia e trasmessi ad ogni livello fino a quando ogni soldato non sa cosa deve essere fatto.

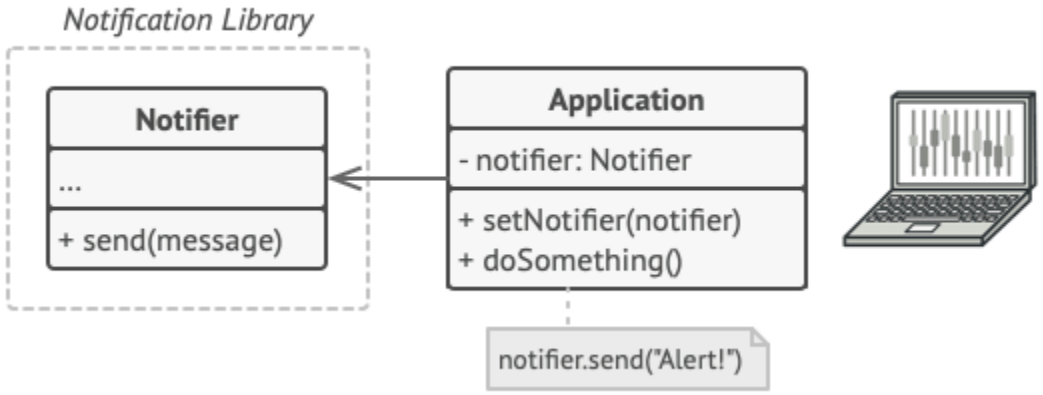
Pattern Strutturali – Il pattern Decorator – Intent

- Decorator è un design pattern strutturale che permette di attaccare nuovi comportamenti agli oggetti posizionando questi oggetti all'interno di speciali oggetti wrapper che contengono i comportamenti.



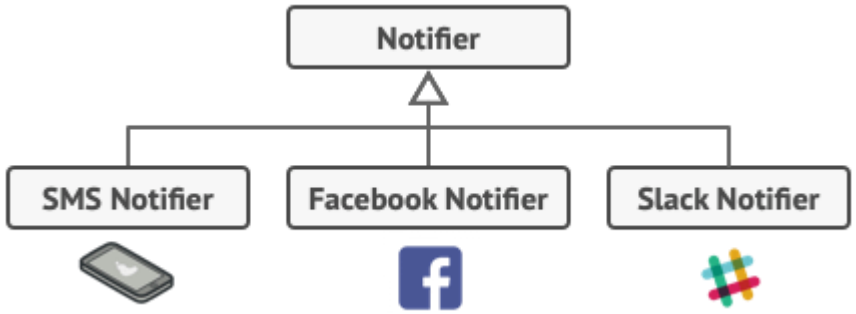
Pattern Strutturali – Il pattern Decorator – Problema

- Immaginiamo di lavorare ad una libreria di notifiche che permette ad altri programmi di notificare ai propri utenti eventi importanti.
- La versione iniziale della libreria era basata sulla classe `Notifier` che aveva solo pochi campi, un costruttore e un singolo metodo `send`.
- Il metodo poteva accettare un argomento messaggio da un client e inviare il messaggio ad una lista di email che venivano passate al notificatore tramite il suo costruttore.
- Un'applicazione di terze parti che fungeva da client doveva creare e configurare l'oggetto notificatore una volta sola, per poi utilizzarlo ogni volta che accadeva qualcosa di importante.



Pattern Strutturali – Il pattern Decorator – Problema

- Ad un certo punto, ci rendiamo conto che gli utenti della libreria si aspettano qualcosa di più di semplici notifiche via e-mail.
- Molti di loro vorrebbero ricevere un SMS su questioni critiche.
- Altri vorrebbero ricevere le notifiche su Facebook e, naturalmente, gli utenti aziendali vorrebbero ricevere le notifiche su Slack.



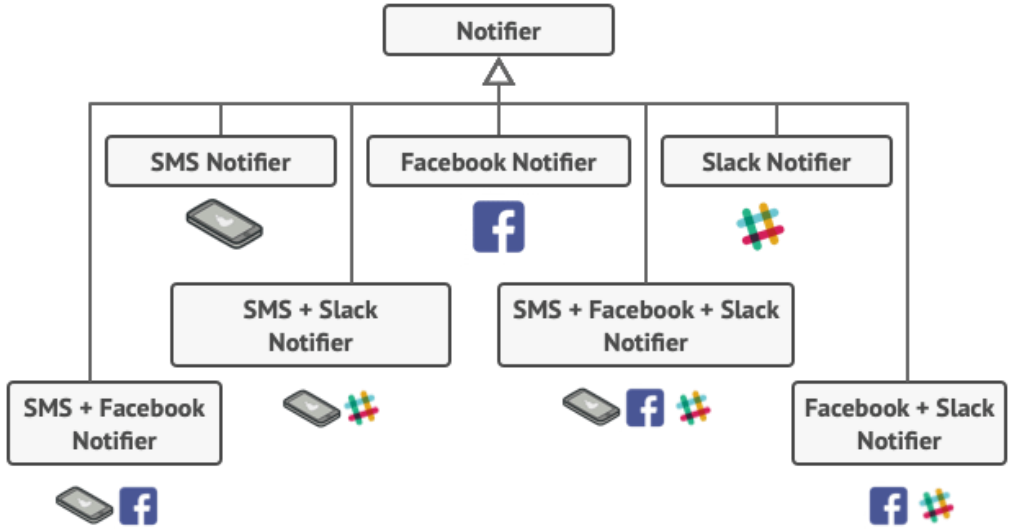
- Quanto può essere difficile?
- Abbiamo esteso la classe `Notifier` e abbiamo inserito i metodi di notifica aggiuntivi in nuove sottoclassi.
- Ora il client avrebbe dovuto istanziare la classe di notifica desiderata e utilizzarla per tutte le ulteriori notifiche.

Pattern Strutturali – Il pattern Decorator – Problema

- Ma poi qualcuno ci ha ragionevolmente chiesto: "Perché non si possono usare più tipi di notifica contemporaneamente? Se la nostra casa è in fiamme, probabilmente vorremmo essere informati attraverso ogni canale".

- Abbiamo cercato di risolvere questo problema creando sottoclassi speciali che combinavano diversi metodi di notifica all'interno di una classe.

- Tuttavia, è diventato subito evidente che questo approccio avrebbe gonfiato a dismisura il codice, non solo il codice della libreria, ma anche il codice del client.



Pattern Strutturali – Il pattern Decorator – Problema

- Dobbiamo trovare un altro modo per strutturare le classi di notifica in modo che il loro numero non infranga accidentalmente qualche record del Guinness dei primati.

Pattern Strutturali – Il pattern Decorator – Soluzione

- L'estensione di una classe è la prima cosa che ci viene in mente quando abbiamo bisogno di modificare il comportamento di un oggetto.
- Tuttavia, l'ereditarietà ha diversi seri problemi di cui bisogna essere consapevoli.
  - L'ereditarietà è statica.
    - Non si può alterare il comportamento di un oggetto esistente a runtime.
    - Si può solo sostituire l'intero oggetto con un altro che è stato creato da una sottoclasse diversa.
  - Le sottoclassi possono avere una sola classe padre.
    - Nella maggior parte dei linguaggi, l'ereditarietà non permette a una classe di ereditare comportamenti di più classi contemporaneamente.

Pattern Strutturali – Il pattern Decorator – Soluzione

- Uno dei modi per superare queste limitazioni è usare l'Aggregazione o la Composizione invece dell'Ereditarietà.
  - Nell'Aggregazione, un oggetto A contiene degli oggetti B; gli oggetti B possono vivere senza A.
  - Nella Composizione, un oggetto A consiste in un insieme di oggetti B; A gestisce il ciclo di vita degli oggetti B; questi ultimi non possono vivere senza A.
- Entrambe le alternative funzionano quasi allo stesso modo: un oggetto ha un riferimento ad un altro e delega ad esso un certo lavoro, mentre con l'ereditarietà, l'oggetto stesso è in grado di fare quel lavoro, ereditando il comportamento dalla sua superclasse.
- Con questo nuovo approccio si può facilmente sostituire l'oggetto "aiutante" collegato con un altro, cambiando il comportamento del contenitore a runtime.
- Un oggetto può usare il comportamento di varie classi, avendo riferimenti a più oggetti e delegando loro ogni tipo di lavoro.
- L'aggregazione/composizione è il principio chiave alla base di molti design pattern, incluso Decorator. Su questa nota, torneremo in seguito.

Pattern Strutturali – Il pattern Decorator – Soluzione

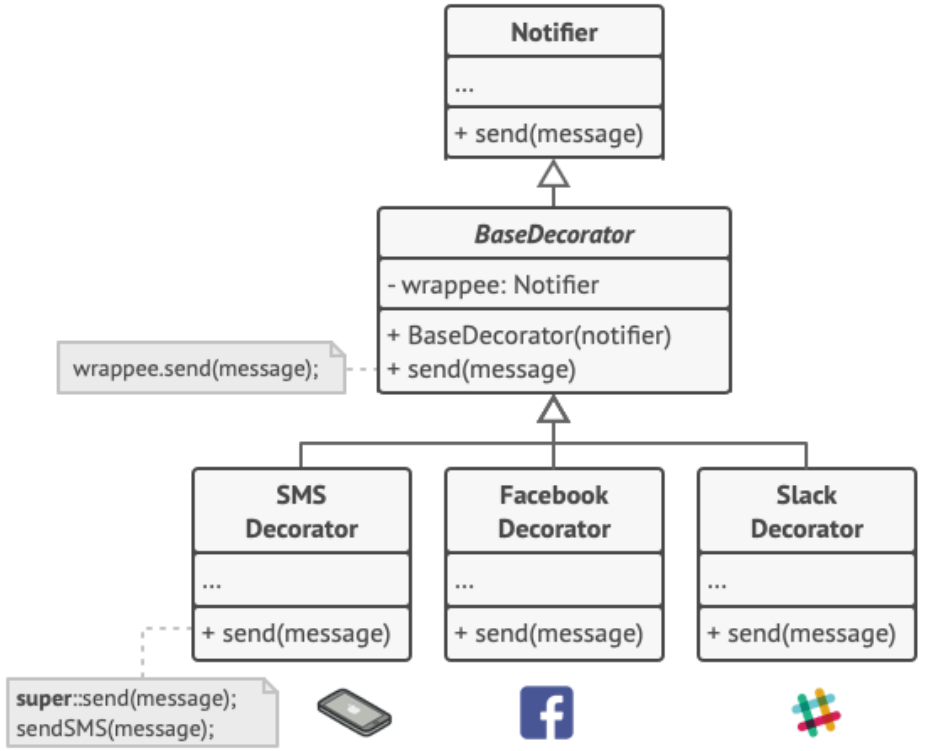


- Wrapper è il nome alternativo per il pattern Decorator che esprime chiaramente l'idea principale del pattern.
- Un "wrapper" è un oggetto che può essere collegato a qualche oggetto «target».
- Il wrapper contiene lo stesso insieme di metodi del target e delega ad esso tutte le richieste che riceve.
- Tuttavia, il wrapper può alterare il risultato facendo qualcosa prima o dopo aver passato la richiesta al target.
- Il wrapper implementa la stessa interfaccia dell'oggetto avvolto.
- Ecco perché dal punto di vista del client questi oggetti sono identici.
- Facciamo in modo che il campo riferimento del wrapper accetti qualsiasi oggetto che segua quell'interfaccia.

Pattern Strutturali – Il pattern Decorator – Soluzione

- Questo ci permetterà di coprire un oggetto in più wrapper, aggiungendovi il comportamento combinato di tutti i wrapper che lo coprono.

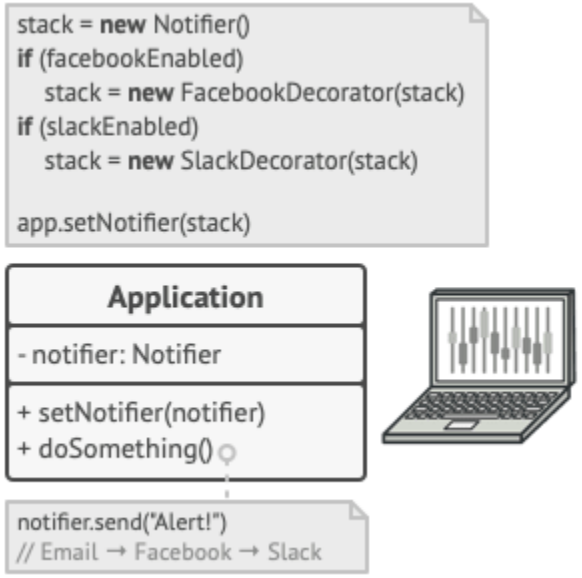
- Nel nostro esempio relativo alle notifiche, lasciamo il semplice comportamento di notifica via email all'interno della classe base `Notifier`, ma trasformiamo tutti gli altri metodi di notifica in decoratori.



Pattern Strutturali – Il pattern Decorator – Soluzione

- Il codice del client dovrebbe avvolgere un oggetto notificatore di base in un insieme di decoratori che corrispondono alle preferenze del cliente.

- Gli oggetti risultanti saranno strutturati come una pila.

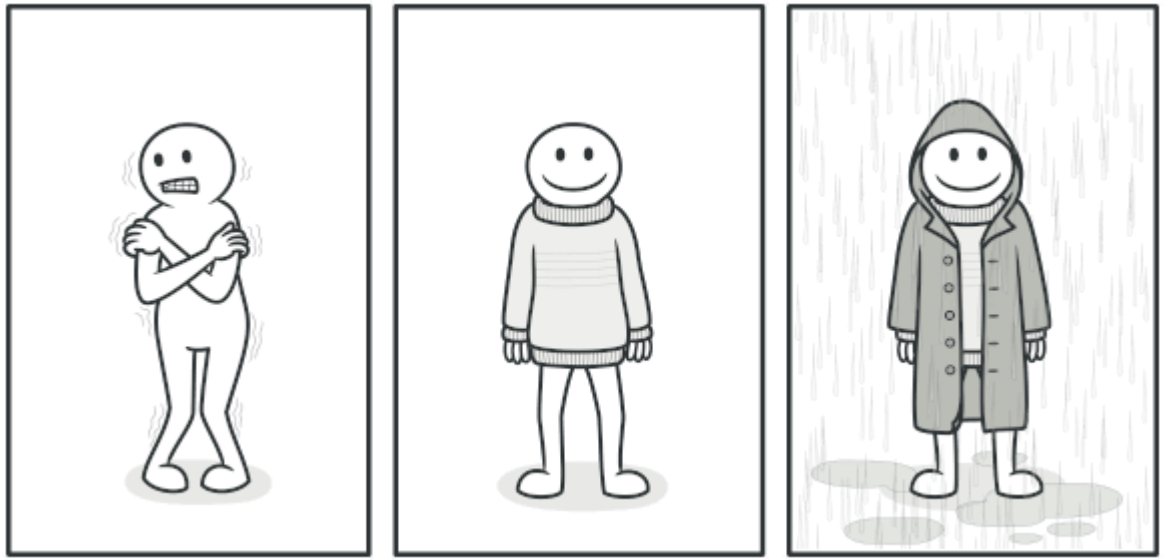


- L'ultimo decoratore della pila sarebbe l'oggetto con cui il client opera effettivamente.

- Poiché tutti i decoratori implementano la stessa interfaccia del notificatore di base, al resto del codice del client non importa se funziona con l'oggetto "puro" notificatore o con quello decorato.

Pattern Strutturali – Il pattern Decorator – Soluzione

- Potremmo applicare lo stesso approccio ad altri comportamenti come la formattazione dei messaggi o la composizione della lista dei destinatari.
- Il client può decorare l'oggetto con qualsiasi decoratore personalizzato, purché segua la stessa interfaccia degli altri.
- Indossare abiti è un esempio di utilizzo dei decoratori.
- Quando abbiamo freddo, ci avvolgiamo in un maglione.

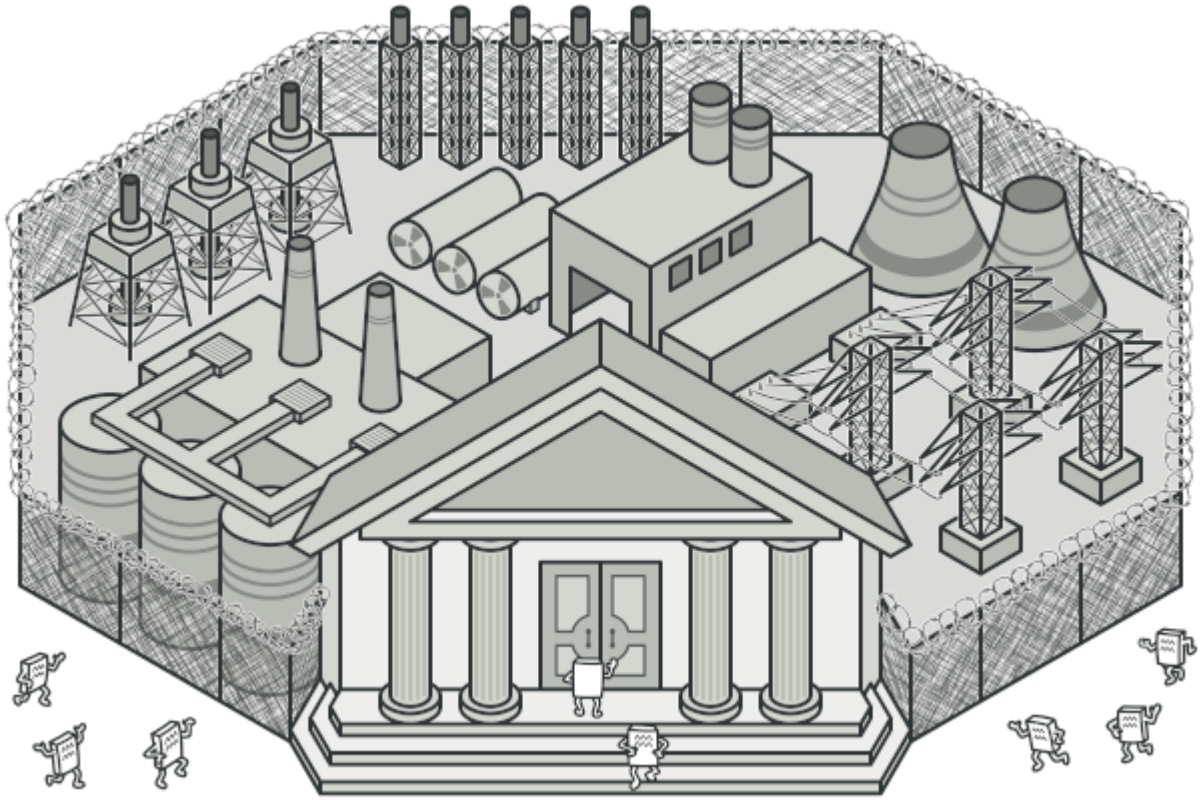


Pattern Strutturali – Il pattern Decorator – Soluzione

- Se abbiamo ancora freddo con un maglione, possiamo indossare una giacca sopra.
- Se piove, possiamo indossare un impermeabile.
- Tutti questi indumenti «estendono» il nostro comportamento di base ma non fanno parte di noi, e possiamo facilmente toglierci qualsiasi capo di vestiario quando non ne abbiamo bisogno.

Pattern Strutturali – Il pattern Facade – Intent

- Facade è un design pattern strutturale che fornisce un'interfaccia semplificata per una libreria, un framework o qualsiasi altro insieme complesso di classi.



Pattern Strutturali – Il pattern Facade – Problema

- Immaginiamo di dover far funzionare il nostro codice con un ampio insieme di oggetti che appartengono ad una sofisticata libreria o a un framework.

- Normalmente, è necessario inizializzare tutti questi oggetti, tenere traccia delle dipendenze, eseguire i metodi nell'ordine corretto e così via.

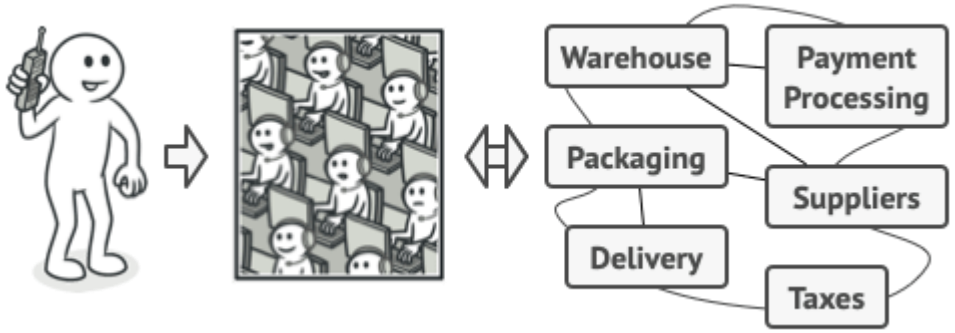
- Di conseguenza, la logica di business delle nostre classi diventerebbe strettamente legata ai dettagli di implementazione delle classi di terze parti, rendendola difficile da comprendere e mantenere.

Pattern Strutturali – Il pattern Facade – Soluzione

- Una facade è una classe che fornisce una semplice interfaccia ad un sottosistema complesso che contiene molte parti mobili.
- Una facade può fornire funzionalità limitate rispetto a quelle a disposizione se si operasse direttamente con il sottosistema.
- Tuttavia, essa include solo quelle caratteristiche che interessano veramente ai client.
- Avere una facciata è utile quando si ha bisogno di integrare la propria applicazione con una sofisticata libreria che ha dozzine di caratteristiche, ma delle cui funzionalità è necessaria solo una piccola parte.
- Ad esempio, un'app che carica brevi video divertenti con i gatti sui social media potrebbe potenzialmente utilizzare una libreria di conversione video professionale.
- Tuttavia, tutto ciò di cui ha veramente bisogno è una classe con il singolo metodo `encode(filename, format)`.
- Dopo aver creato una classe di questo tipo e averla collegata alla libreria di conversione video, avremo la nostra prima facade.

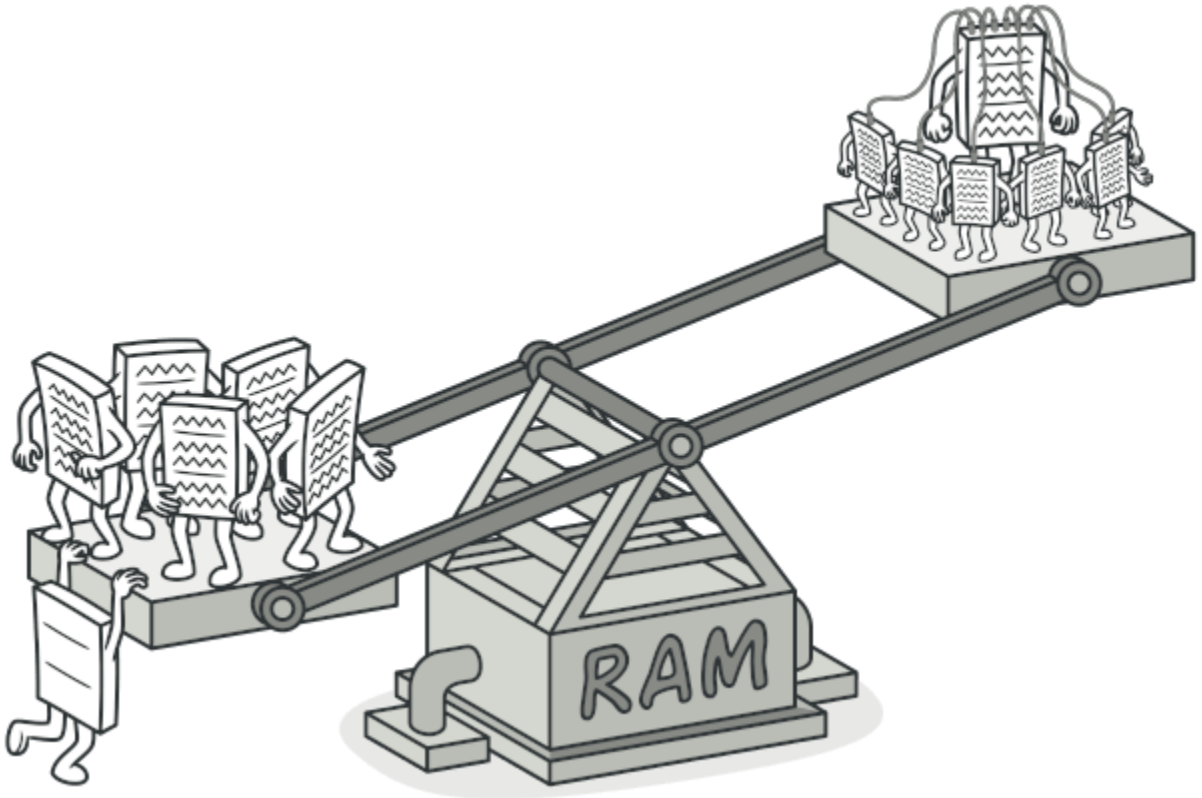
Pattern Strutturali – Il pattern Facade – Soluzione

- Consideriamo la seguente analogia presa dalla vita reale.
- Quando chiamiamo un negozio per effettuare un ordine telefonico, un operatore è la nostra facciata verso tutti i servizi e i reparti del negozio.
- L'operatore ci fornisce una semplice interfaccia vocale al sistema di ordinazione, ai gateway di pagamento e a vari servizi di consegna.



Pattern Strutturali – Il pattern Flyweight – Intent

- Flyweight è un design pattern strutturale che consente di inserire più oggetti nella quantità di RAM disponibile condividendo parti comuni dello stato tra più oggetti invece di mantenere tutti i dati in ogni oggetto.

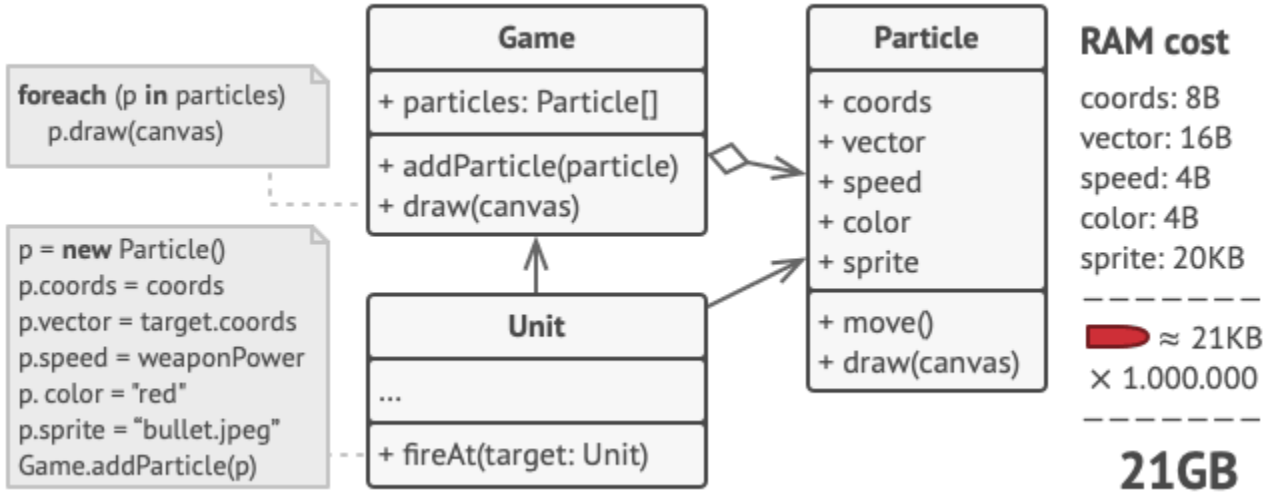


Pattern Strutturali – Il pattern Flyweight – Problema

- Supponiamo che, per divertirci dopo una lunga giornata di lavoro, abbiamo deciso di creare un semplice videogioco: i giocatori si sarebbero spostati su una mappa e si sarebbero sparati a vicenda.
- Abbiamo scelto di implementare un insieme di particelle realistico e di farne una caratteristica distintiva del gioco.
- Vaste quantità di proiettili, missili e schegge di esplosioni dovrebbero volare su tutta la mappa e regalare al giocatore un'esperienza emozionante.
- Al suo completamento, abbiamo effettuato il commit, costruito il gioco e inviato ad un nostro amico per un test dal vivo.
- Anche se il gioco funzionava perfettamente sulla nostra macchina, il nostro amico non è stato in grado di giocare a lungo.
- Sul suo computer, il gioco continuava ad andare in crash dopo pochi minuti.
- Dopo aver passato diverse ore a scavare tra i registri di debug, abbiamo scoperto che il gioco è andato in crash a causa di una quantità insufficiente di RAM.
- Si è scoperto che il computer del nostro amico era molto meno potente del nostro computer, ed è per questo che il problema è emerso così rapidamente sulla sua macchina.

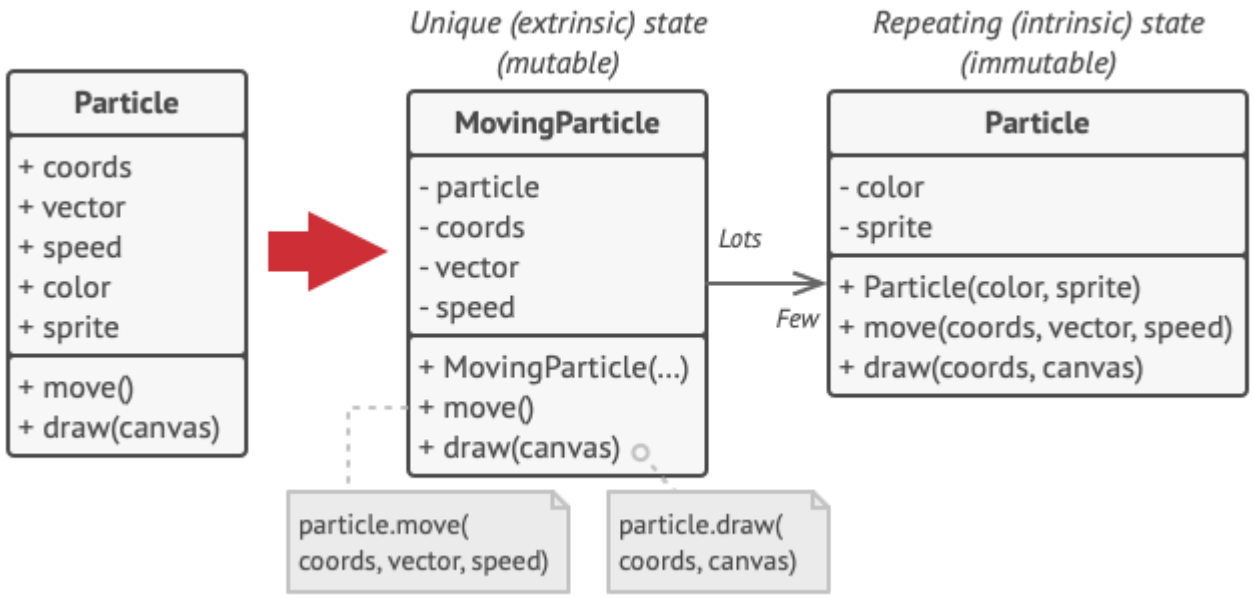
Pattern Strutturali – Il pattern Flyweight – Problema

- Il problema reale era legato al nostro sistema di particelle.
- Ogni particella, come un proiettile, un missile o un pezzo di scheggia era rappresentata da un oggetto separato contenente molti dati.
- Ad un certo punto, quando la carneficina sullo schermo di un giocatore ha raggiunto il suo culmine, le nuove particelle create non entrano più nella RAM rimanente, quindi il programma blocca.



Pattern Strutturali – Il pattern Flyweight – Soluzione

- A un esame più attento della classe `Particle`, possiamo notare che i campi `color` e `sprite` consumano molta più memoria rispetto ad altri campi.
- Quel che è peggio è che questi due campi memorizzano dati quasi identici su tutte le particelle.
- Per esempio, tutti i proiettili hanno lo stesso colore e lo stesso sprite.

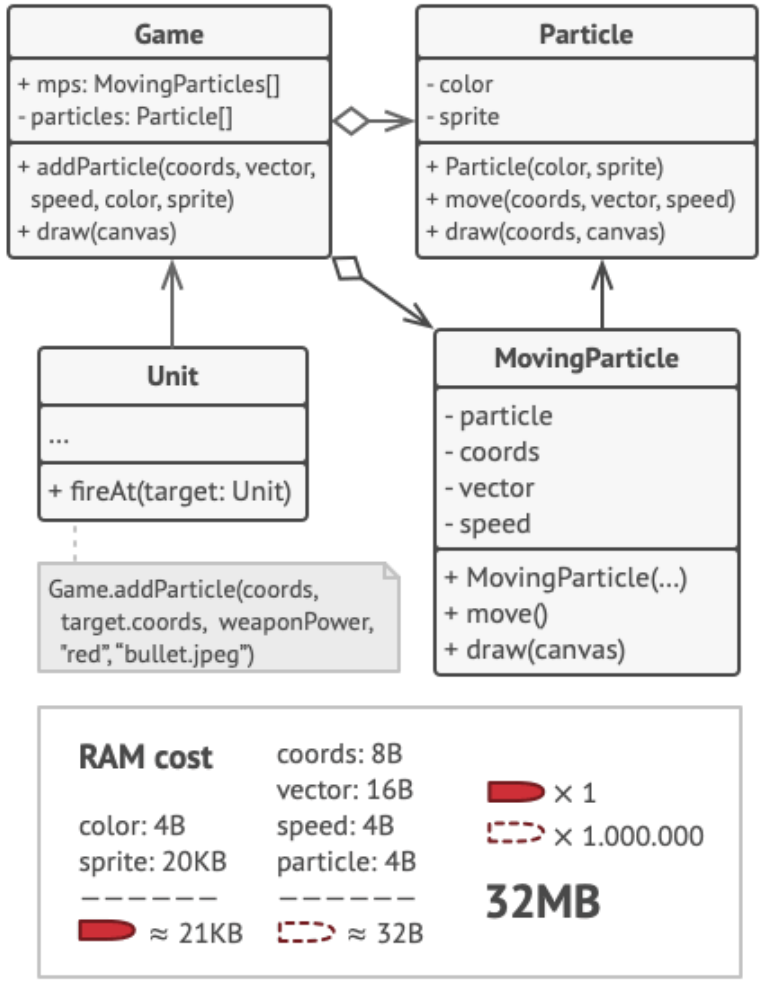


Pattern Strutturali – Il pattern Flyweight – Soluzione

- Altre parti dello stato di una particella, come le coordinate, il vettore di movimento e la velocità, sono uniche per ogni particella.
- Infatti, i valori di questi campi cambiano nel tempo.
- Questi dati rappresentano il contesto sempre mutevole in cui una particella esiste, mentre il colore e lo sprite rimangono costanti per ogni particella.
- Questo dato costante di un oggetto viene solitamente chiamato *stato intrinseco*.
- Esso vive all'interno dell'oggetto; gli altri oggetti possono solo leggerlo, non modificarlo.
- Il resto dello stato dell'oggetto, spesso alterato "dall'esterno" da altri oggetti, è chiamato *stato estrinseco*.
- Il pattern Flyweight suggerisce di non memorizzare più lo stato estrinseco all'interno dell'oggetto.
- Invece, si dovrebbe passare questo stato a metodi specifici che si basano su di esso.
- Solo lo stato intrinseco rimane all'interno dell'oggetto, permettendo di riutilizzarlo in contesti diversi.

Pattern Strutturali – Il pattern Flyweight – Soluzione

- Di conseguenza, avremo bisogno di meno oggetti, poiché si differenziano solo per lo stato intrinseco, che ha molte meno variazioni rispetto all'estrinseco.

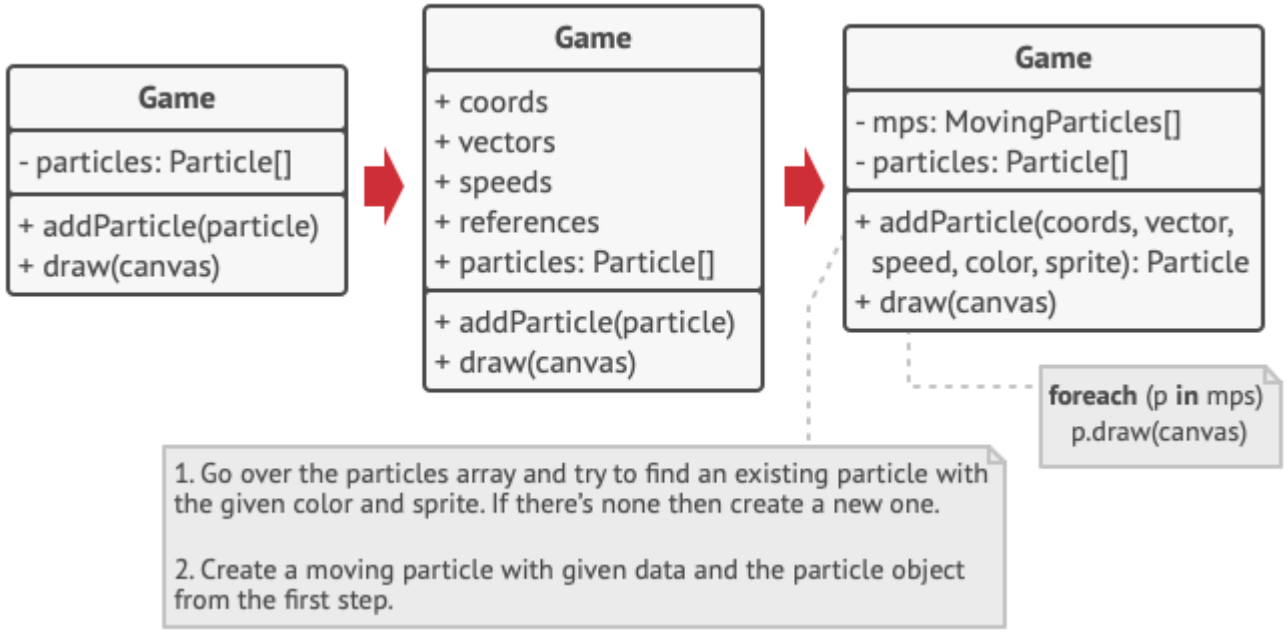


Pattern Strutturali – Il pattern Flyweight – Soluzione

- Torniamo al nostro gioco.
- Supponendo di aver estratto lo stato estrinseco dalla nostra classe che gestisce le particelle, basterebbero solo tre oggetti diversi per rappresentare tutte le particelle del gioco: un proiettile, un missile e un pezzo di scheggia.
- Come probabilmente abbiamo già intuito, un oggetto che memorizza solo lo stato intrinseco si chiama flyweight.
- Dove si sposta lo stato estrinseco? Qualche classe dovrebbe ancora conservarlo, no?
- Nella maggior parte dei casi, viene spostato sull'oggetto contenitore, che aggrega gli oggetti prima di applicare il pattern.
- Nel nostro caso, questo è l'oggetto principale `Game` che memorizza tutte le particelle nel campo `particles`.
- Per spostare lo stato estrinseco in questa classe, è necessario creare diversi campi array per memorizzare le coordinate, i vettori e la velocità di ogni singola particella.
- Ma non è tutto. È necessario un altro array per memorizzare i riferimenti ad un flyweight specifico che rappresenta una particella.

Pattern Strutturali – Il pattern Flyweight – Soluzione

- Il seguente schema rappresenta tutto questo:



- Una soluzione più elegante è quella di creare una classe di contesto separata che memorizzi lo stato estrinseco insieme al riferimento all'oggetto flyweight.

- Questo approccio richiederebbe di avere solo un singolo array nella classe contenitore.

Pattern Strutturali – Il pattern Flyweight – Soluzione

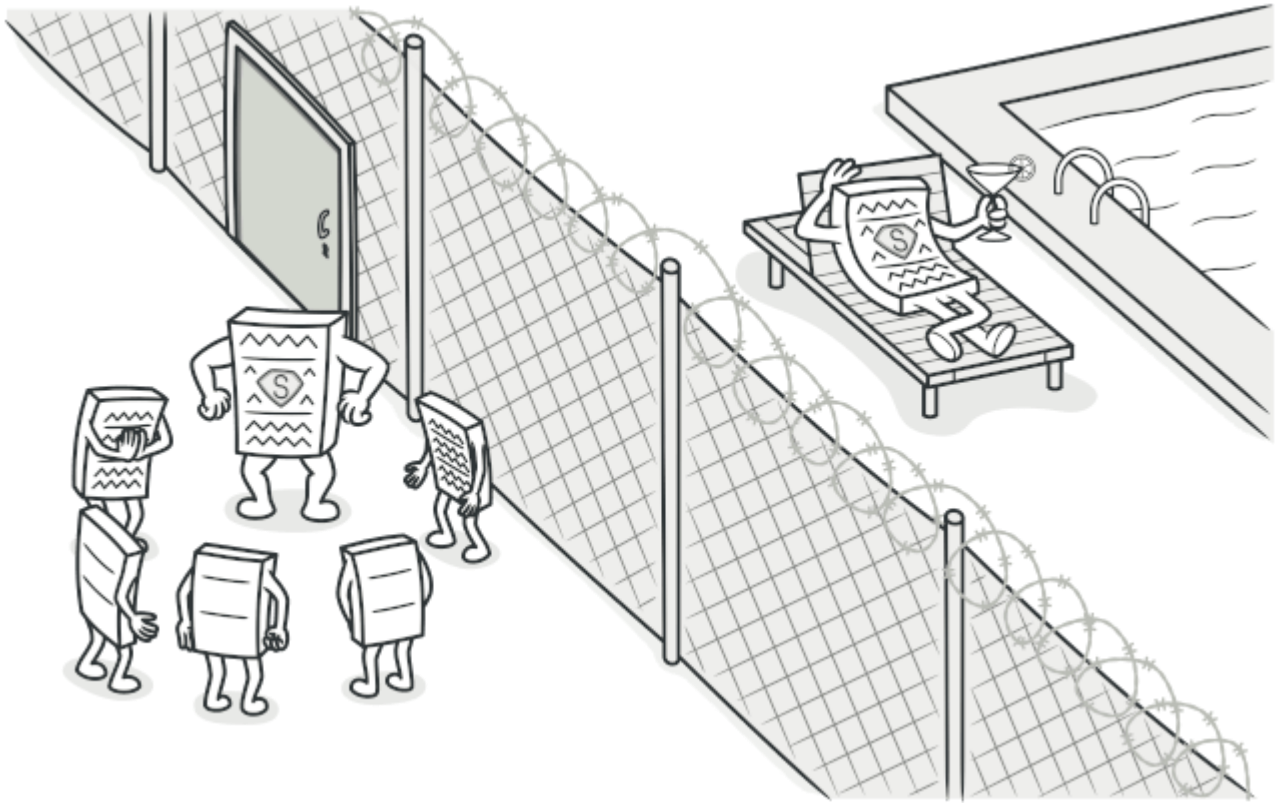
- Riflettiamo, però, un attimo! Non avremo bisogno di avere tanti di questi oggetti contestuali quanti ne avevamo all'inizio?
- Tecnicamente, sì. Ma il fatto è che questi oggetti sono molto più piccoli di prima.
- I campi che consumano più memoria sono stati spostati solo su pochi oggetti flyweight.
- Ora, un migliaio di piccoli oggetti contestuali possono riutilizzare un singolo oggetto flyweight pesante, invece di memorizzare un migliaio di copie dei suoi dati.
- Dal momento che lo stesso oggetto flyweight può essere utilizzato in contesti differenti, è necessario assicurarsi che il suo stato non possa essere modificato.
- Un flyweight dovrebbe inizializzare il suo stato solo una volta, attraverso i parametri del costruttore.
- Non dovrebbe esporre alcun setter o campo pubblico ad altri oggetti.
- Per un accesso più comodo ai vari flyweight, è possibile creare un factory method che gestisce un pool di oggetti flyweight esistenti.

Pattern Strutturali – Il pattern Flyweight – Soluzione

- Il metodo accetta lo stato intrinseco del flyweight desiderato da parte di un cliente, cerca un oggetto flyweight esistente che corrisponda a questo stato e, se lo trova, lo restituisce.
- In caso contrario, crea un nuovo flyweight e lo aggiunge al pool.
- Ci sono diverse opzioni in cui questo metodo potrebbe essere collocato.
- Il posto più ovvio è un contenitore flyweight.
- In alternativa, si potrebbe creare una nuova factory class.
- Oppure si può rendere il factory method statico e metterlo all'interno di una vera e propria flyweight class.

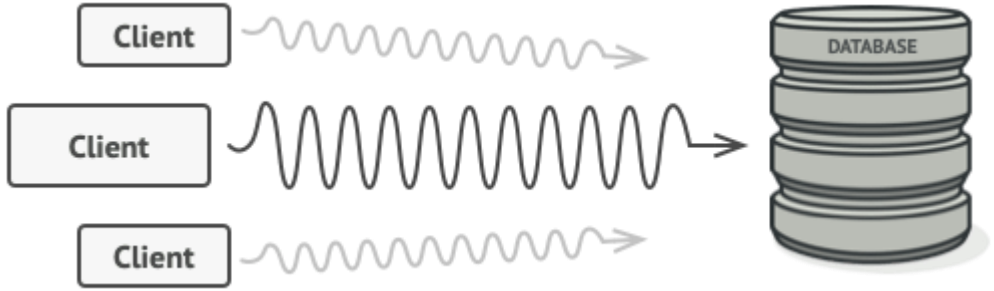
Pattern Strutturali – Il pattern Proxy – Intent

- Proxy è un design pattern strutturale che consente di fornire un sostituto o un segnaposto per un altro oggetto.
- Un proxy controlla l'accesso all'oggetto originale, consentendo di eseguire qualcosa prima o dopo che la richiesta arriva all'oggetto originale.



Pattern Strutturali – Il pattern Proxy – Problema

- Perché vogliamo controllare l'accesso a un oggetto?
- Ecco un esempio: abbiamo un oggetto massivo che consuma una grande quantità di risorse di sistema. Ne abbiamo bisogno di tanto in tanto, ma non sempre.



- Si potrebbe implementare una inizializzazione pigra: in questo caso si creerebbe questo oggetto solo quando è effettivamente necessario.
- Tutti i client dell'oggetto avrebbero bisogno di eseguire del codice di inizializzazione differito, nel momento in cui l'oggetto dovesse servire.
- Sfortunatamente, questo causerebbe probabilmente tantissime duplicazioni di codice.

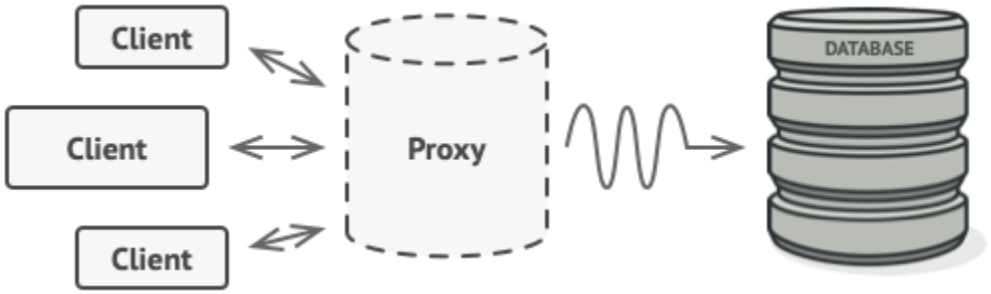
Pattern Strutturali – Il pattern Proxy – Problema

- In un mondo ideale, vorremmo mettere questo codice direttamente nella classe del nostro oggetto, ma non sempre è possibile.

- Ad esempio, la classe può far parte di una libreria di terze parti.

Pattern Strutturali – Il pattern Proxy – Soluzione

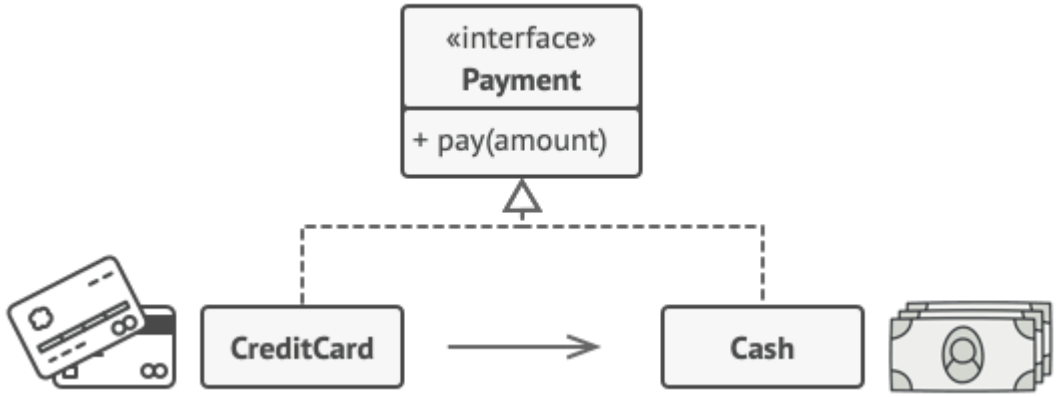
- Il pattern Proxy suggerisce di creare una nuova classe proxy con la stessa interfaccia di un oggetto di servizio originale.
- A questo punto si può aggiornare la nostra applicazione in modo che passi l'oggetto proxy a tutti i client dell'oggetto originale.
- Al ricevimento di una richiesta da parte di un client, il proxy crea un oggetto di servizio reale e delega ad esso tutto il lavoro.



- Ma qual è il vantaggio? Se si ha bisogno di eseguire qualcosa prima o dopo la logica principale della classe, il proxy permette di farlo senza modificare la classe.
- Poiché il proxy implementa la stessa interfaccia della classe originale, può essere passato a qualsiasi client che si aspetta un oggetto di servizio reale.

Pattern Strutturali – Il pattern Proxy – Soluzione

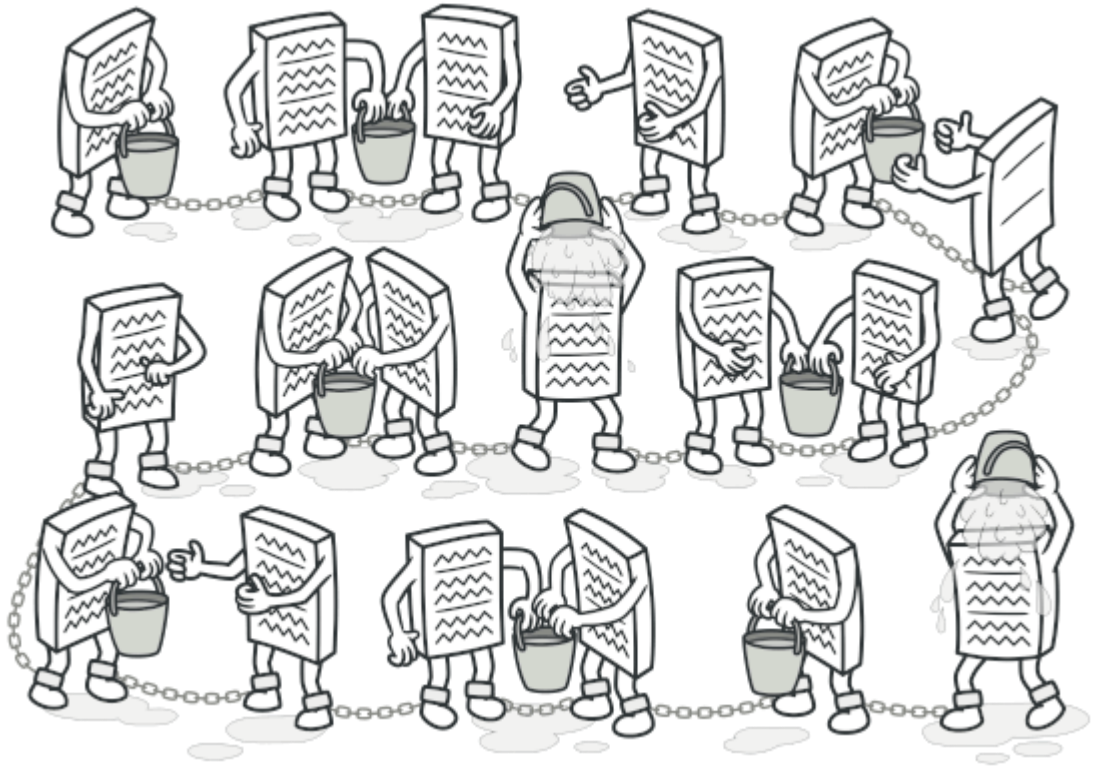
- Vediamo un'analogia relativa al mondo reale: una carta di credito è un proxy per un conto bancario, che è proxy per un pacchetto di contanti.
- Entrambi implementano la stessa interfaccia: possono essere utilizzati per effettuare un pagamento.
- Un consumatore è felice perché così non deve portarsi dietro pacchetti di banconote.
- Il proprietario di un negozio è anche contento perché il ricavo di una transazione viene aggiunto elettronicamente al conto bancario del negozio senza il rischio di perdere il deposito o di essere derubato durante il tragitto verso la banca.



Pattern Comportamentali – Il pattern Chain of Responsibility – Intent

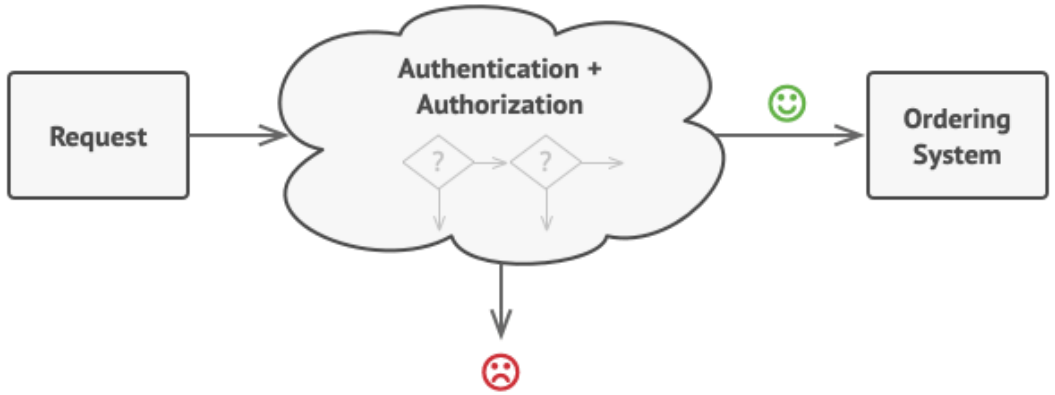
- Chain of Responsibility è un design pattern comportamentale che consente di passare le richieste lungo una catena di gestori.

- Al ricevimento di una richiesta, ogni gestore decide di elaborare la richiesta o di passarla al successivo gestore della catena.



Pattern Comportamentali – Il pattern Chain of Responsibility – Problema

- Immaginiamo di lavorare a un sistema di ordini online.
- Vogliamo limitare l'accesso al sistema in modo che solo gli utenti autenticati possano creare ordini.
- Inoltre, gli utenti che hanno autorizzazioni amministrative devono avere pieno accesso a tutti gli ordini.
- Dopo un po' di pianificazione, ci siamo resi conto che questi controlli devono essere eseguiti in sequenza.
- L'applicazione può provare ad autenticare un utente al sistema ogni volta che riceve una richiesta che contiene le credenziali dell'utente.
- Tuttavia, se tali credenziali non sono corrette e l'autenticazione fallisce, non c'è motivo di procedere con altri controlli.

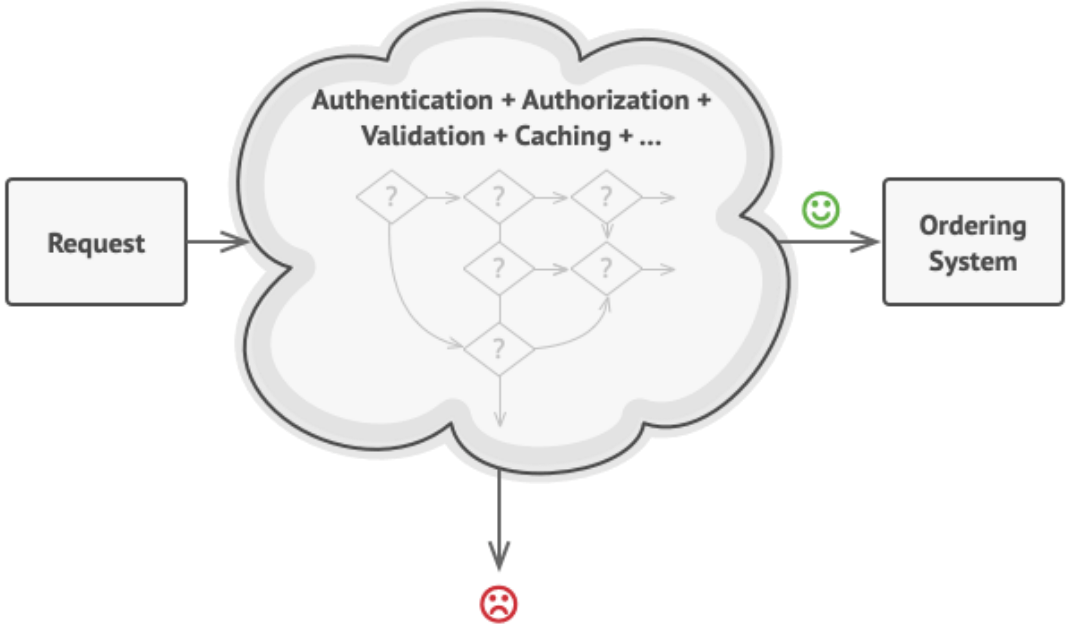


Pattern Comportamentali – Il pattern Chain of Responsibility – Problema

- Durante i mesi successivi, abbiamo implementato diversi altri controlli sequenziali.
  - Uno dei nostri colleghi ha suggerito che non è sicuro passare i dati grezzi direttamente al sistema di ordinazione.
    - Così abbiamo aggiunto un ulteriore passo di convalida per sanificare i dati in una richiesta.
  - In seguito, qualcuno ha notato che il sistema è vulnerabile al cracking delle password mediante forza bruta.
    - Per impedire questo, abbiamo subito aggiunto un controllo che filtra le richieste fallite ripetute provenienti dallo stesso indirizzo IP.
  - Qualcun altro ha suggerito che si potrebbe accelerare il sistema restituendo i risultati della cache quando vi sono richieste ripetute contenenti gli stessi dati.
    - Quindi, abbiamo aggiunto un altro controllo che lascia passare la richiesta al sistema solo se non c'è una risposta adeguata in cache.
- Il codice dei controlli, che già sembrava molto aggrovigliato, è diventato sempre più gonfio man mano che si sono aggiunte nuova funzionalità.

Pattern Comportamentali – Il pattern Chain of Responsibility – Problema

- La modifica di un controllo a volte influiva sugli altri.
- Peggio ancora, quando si cercava di riutilizzare i controlli per proteggere altri componenti del sistema, si doveva duplicare parte del codice, poiché quei componenti richiedevano alcuni dei controlli, ma non tutti.



- Il sistema è diventato molto difficile da comprendere e costoso da mantenere.
- Abbiamo lottato con il codice per un po' di tempo, finché un giorno abbiamo deciso di rifattorizzare il tutto.

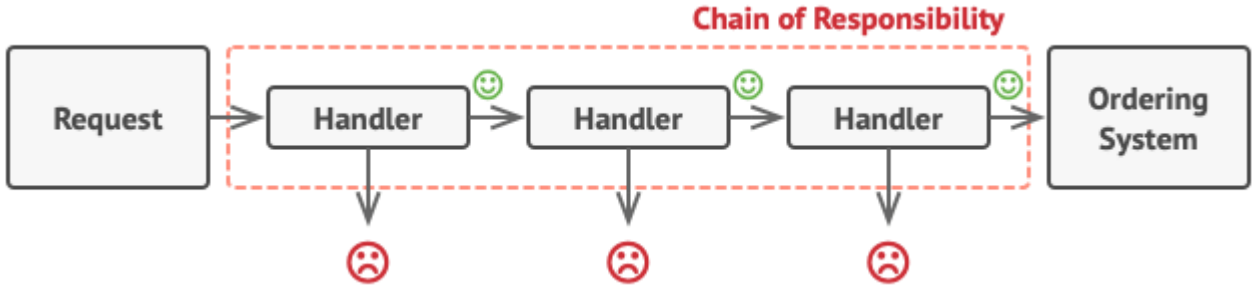
Pattern Comportamentali – Il pattern Chain of Responsibility – Soluzione

- Come molti altri design pattern comportamentali, Chain of Responsibility si basa sulla trasformazione di particolari comportamenti in oggetti standalone chiamati handler.
- Nel nostro caso, ogni controllo dovrebbe essere associato ad una propria classe con un unico metodo che esegue il controllo.
- La richiesta, insieme ai suoi dati, viene passata a questo metodo come argomento.
- Il pattern suggerisce di collegare questi gestori in una catena.
- Ogni gestore collegato ha un campo per memorizzare un riferimento al gestore successivo nella catena.
- Oltre a processare una richiesta, i gestori la passano ulteriormente lungo la catena.
- La richiesta viaggia lungo la catena fino a quando tutti i gestori hanno avuto la possibilità di elaborarla.
- E qui arriva la parte migliore del pattern: un gestore può decidere di non passare la richiesta ulteriormente nella catena e fermare efficacemente qualsiasi ulteriore elaborazione.

Pattern Comportamentali – Il pattern Chain of Responsibility – Soluzione

- Nel nostro esempio con i sistemi di ordini, un gestore esegue l'elaborazione e poi decide se passare la richiesta ulteriormente nella catena.

- Supponendo che la richiesta contenga i dati giusti, tutti i gestori possono eseguire il loro comportamento primario, sia che si tratti di controlli di autenticazione che di cache.



- Tuttavia, c'è un approccio leggermente diverso (ed è un po' più canonico) in cui, al ricevimento di una richiesta, un gestore decide se può elaborarla.

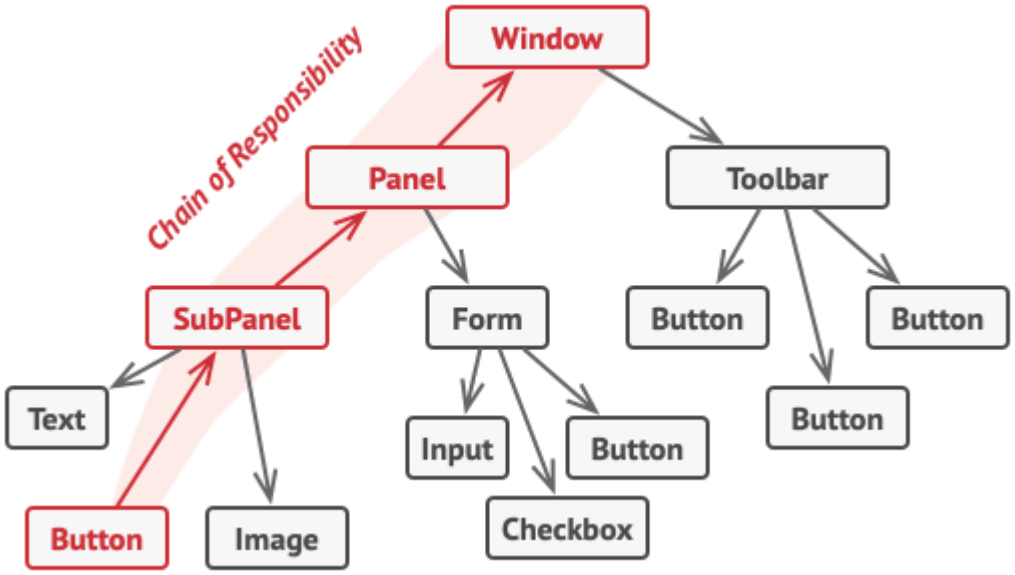
- Se può, non passa più la richiesta.

- Quindi una richiesta viene elaborata o da un solo gestore o da nessuno.

- Questo approccio è molto comune quando si tratta di eventi in pile di elementi all'interno di un'interfaccia utente grafica.

Pattern Comportamentali – Il pattern Chain of Responsibility – Soluzione

- Per esempio, quando un utente fa clic su un pulsante, l'evento si propaga attraverso la catena di elementi GUI che inizia con il pulsante, passa lungo i suoi contenitori (come form o panel), e finisce con la finestra principale dell'applicazione.
- L'evento viene elaborato dal primo elemento della catena che è in grado di gestirlo.
- Questo esempio è interessante anche perché mostra che una catena può sempre essere estratta da un albero di oggetti.

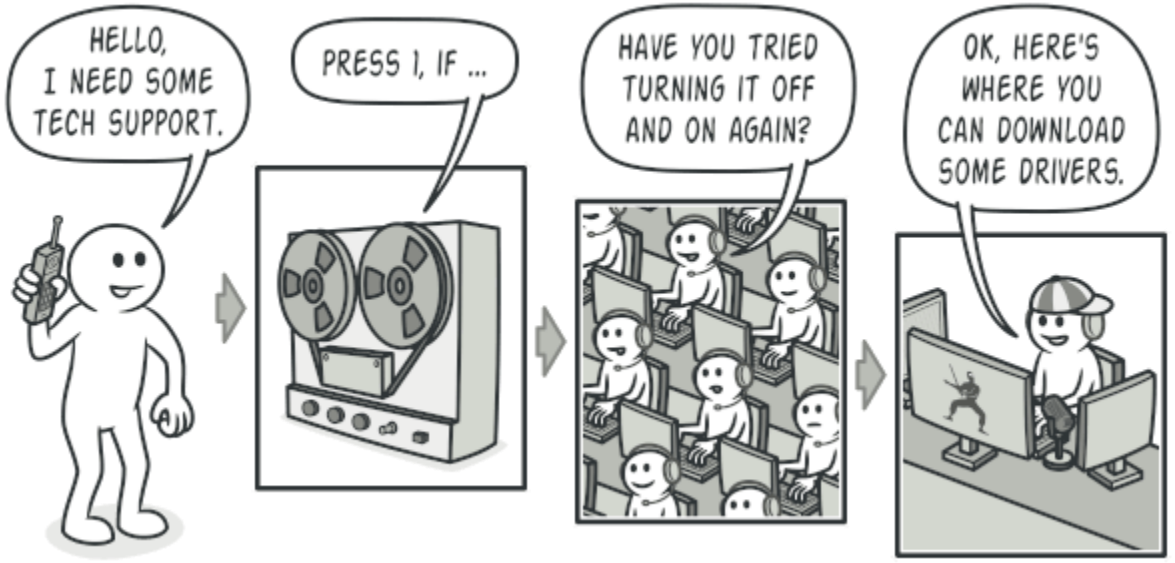


Pattern Comportamentali – Il pattern Chain of Responsibility – Soluzione

- È fondamentale che tutte le classi degli handler implementino la stessa interfaccia.
- Ogni handler concreto dovrebbe preoccuparsi solo di quello seguente che ha il metodo `execute`.
- In questo modo si possono comporre catene a runtime, usando vari gestori senza accoppiare il codice alle loro classi concrete.
- Consideriamo un'analogia proveniente dal mondo reale.
- Abbiamo appena acquistato e installato un nuovo hardware sul nostro computer.
- Dato che siamo smart, il computer ha diversi sistemi operativi installati.
- Cerchiamo di avviarli tutti per vedere se l'hardware è supportato.
- Windows rileva e abilita l'hardware automaticamente.
- Invece, il nostro amato Linux si rifiuta di lavorare con il nuovo hardware.
- Con un piccolo lampo di speranza, decidiamo di chiamare il numero di telefono del supporto tecnico scritto sulla scatola.

Pattern Comportamentali – Il pattern Chain of Responsibility – Soluzione

- La prima cosa che sentiamo è la voce robotica del risponditore automatico.
- Essa suggerisce nove soluzioni popolari a vari problemi, nessuna delle quali è rilevante per il nostro caso.
- Dopo un po', il robot ci collega a un operatore in carne e ossa.



- Purtroppo, l'operatore non è in grado di suggerire nulla di specifico.
- Continua a citare lunghi estratti del manuale, rifiutandosi di ascoltare i nostri commenti.

Pattern Comportamentali – Il pattern Chain of Responsibility – Soluzione

- Dopo aver sentito la frase "avete provato a spegnere e riaccendere il computer?" per la decima volta, chiediamo di essere collegati ad un vero ingegnere.

- Alla fine, l'operatore passa la chiamata a uno degli ingegneri, che probabilmente aveva nostalgia di una chiacchierata umana dal vivo per ore mentre stava seduto nella sua stanza solitaria del server nel buio seminterrato di qualche edificio pieno di uffici.

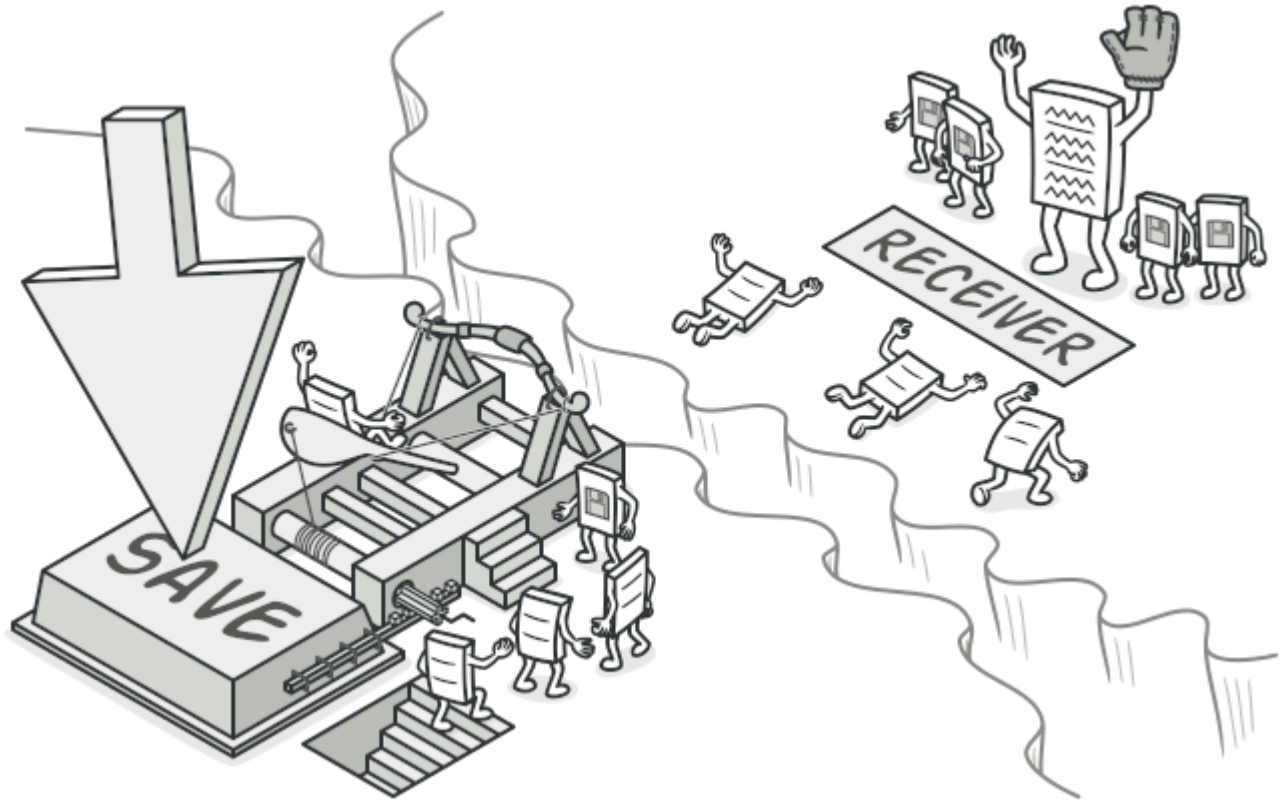
- L'ingegnere ci dice dove scaricare i driver corretti per il nostro nuovo hardware e come installarli su Linux.

- Finalmente, la soluzione! Si finisce la chiamata, scoppiando di gioia.

Pattern Comportamentali – Il pattern Command – Intent

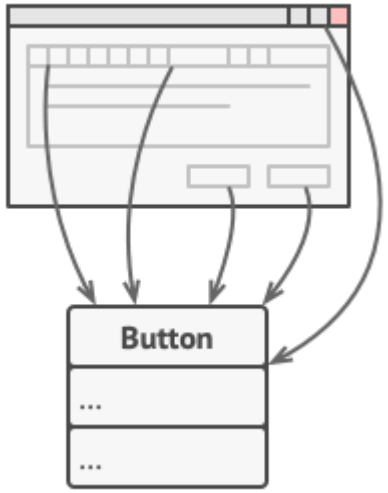
- Command è un design pattern comportamentale che trasforma una richiesta in un oggetto a sé stante che contiene tutte le informazioni sulla richiesta.

- Questa trasformazione consente di parametrizzare i metodi con diverse richieste, ritardare o mettere in coda l'esecuzione di una richiesta e supportare le operazioni annullabili.



Pattern Comportamentali – Il pattern Command – Problema

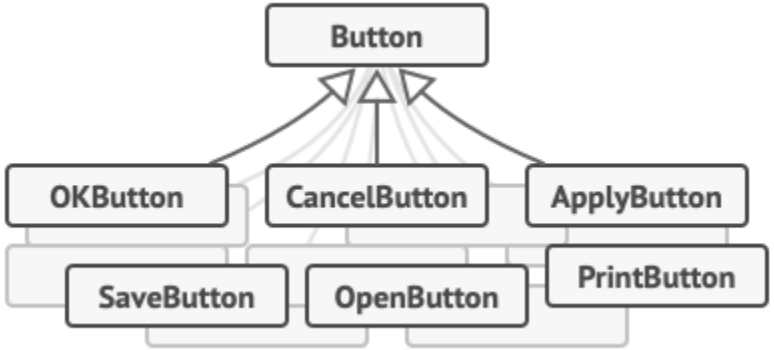
- Immaginiamo di lavorare a un nuovo text editor.
- Il nostro compito attuale è quello di creare una barra degli strumenti con una serie di pulsanti per le varie operazioni dell'editor.
- Abbiamo creato una classe `Button` molto ordinata che può essere utilizzata per i pulsanti della barra degli strumenti, così come per i pulsanti generici in varie finestre di dialogo.



- Anche se tutti questi pulsanti sono simili, si suppone che facciano tutti cose diverse.
- Dove metteremmo il codice per i vari gestori di questi pulsanti?

Pattern Comportamentali – Il pattern Command – Problema

- La soluzione più semplice è quella di creare tonnellate di sottoclassi per ogni punto in cui viene utilizzato il pulsante.
- Queste sottoclassi conterrebbero il codice che dovrebbe essere eseguito a seguito del click sul pulsante.



- In breve tempo, ci rendiamo conto che questo approccio è profondamente difettoso.
- In primo luogo, disponiamo di un numero enorme di sottoclassi, e questo andrebbe bene se non si rischia di rovinare il codice in queste sottoclassi ogni volta che si modifica la classe di base `Button`.
- In parole povere, il nostro codice GUI è diventato goffamente dipendente dal codice volatile della logica di business.

Pattern Comportamentali – Il pattern Command – Problema

- Ed ecco la parte più brutta. Alcune operazioni, come il copia/incolla del testo, dovrebbero essere invocate da più punti.

- Per esempio, un utente potrebbe fare click su un piccolo pulsante "Copia" sulla barra degli strumenti, o copiare qualcosa tramite il menu contestuale, o semplicemente premere `Ctrl+C` sulla tastiera.



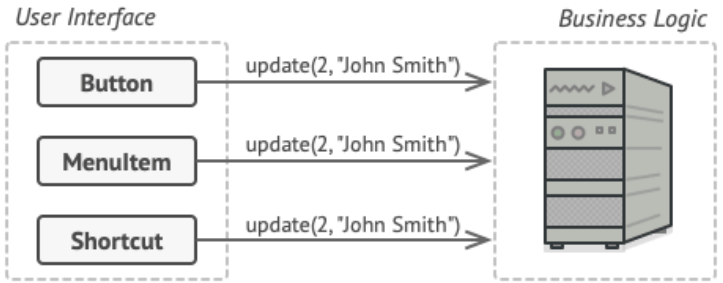
- Inizialmente, quando la nostra applicazione aveva solo la barra degli strumenti, andava bene inserire l'implementazione delle varie operazioni nelle sottoclassi dei pulsanti.

- In altre parole, avere il codice per copiare il testo all'interno della sottoclasse `CopyButton` andava bene.

- Ma poi, quando si implementano menu contestuali, scorciatoie e altre cose, si deve o duplicare il codice dell'operazione in molte classi o rendere i menu dipendenti dai pulsanti, che è un'opzione ancora peggiore.

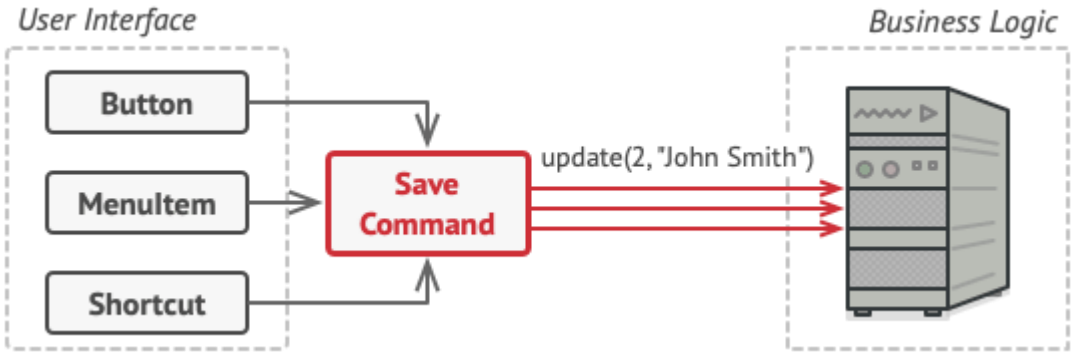
Pattern Comportamentali – Il pattern Command – Soluzione

- Una buona progettazione del software si basa spesso sul *principio della separazione dei problemi*, che di solito si traduce nella strutturazione di un'applicazione in strati.
- L'esempio più comune: un livello per l'interfaccia grafica utente e un altro livello per la logica di business.
- Il livello GUI è responsabile del rendering di una bella immagine sullo schermo, dell'acquisizione di qualsiasi input e della visualizzazione dei risultati di ciò che l'utente e l'app stanno facendo.
- Tuttavia, quando si tratta di fare qualcosa di importante, come calcolare la traiettoria della luna o comporre un rapporto annuale, il livello GUI delega il lavoro al livello sottostante della business logic.
- Nel codice potrebbe apparire così: un oggetto GUI chiama un metodo di un oggetto della logica di business, passando ad esso alcuni argomenti.
- Questo processo è solitamente descritto come un oggetto che invia una richiesta ad un altro.



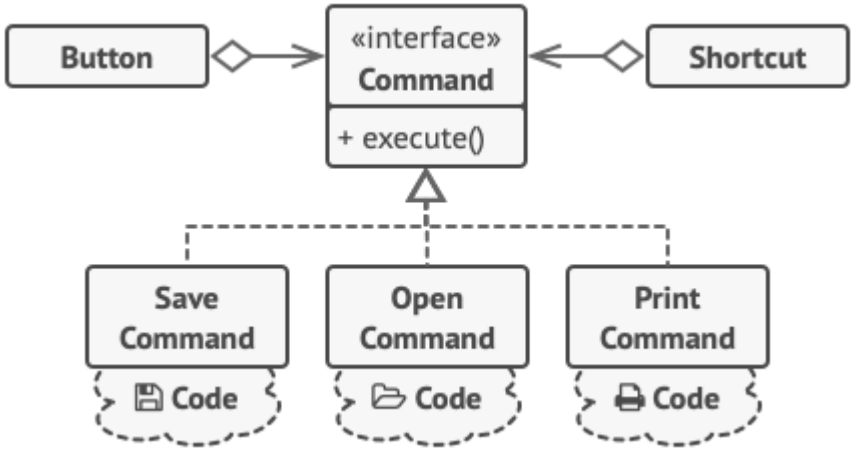
Pattern Comportamentali – Il pattern Command – Soluzione

- Il pattern Command suggerisce che gli oggetti GUI non dovrebbero inviare queste richieste direttamente.
- Invece, si dovrebbero estrarre tutti i dettagli della richiesta, come l'oggetto che si sta chiamando, il nome del metodo e la lista degli argomenti, in una classe *command* separata, con un singolo metodo che innesca questa richiesta.
- Gli oggetti *command* servono come collegamenti tra le varie GUI e gli oggetti della logica di business.
- D'ora in poi, l'oggetto GUI non ha bisogno di sapere quale oggetto della business logic riceverà la richiesta e come verrà elaborata.
- L'oggetto GUI si limita ad attivare il comando, che gestisce tutti i dettagli.



Pattern Comportamentali – Il pattern Command – Soluzione

- Il passo successivo è quello di far sì che i comandi implementino la stessa interfaccia.
- Di solito tale interfaccia ha solo un singolo metodo di esecuzione che non prende parametri.
- Essa permette di utilizzare vari comandi con lo stesso mittente della richiesta, senza accoppiarlo a classi concrete di comandi.
- Come vantaggio di questo modo di procedere, ora è possibile cambiare gli oggetti comando collegati al mittente, cambiando di fatto il comportamento del mittente durante l'esecuzione.



Pattern Comportamentali – Il pattern Command – Soluzione

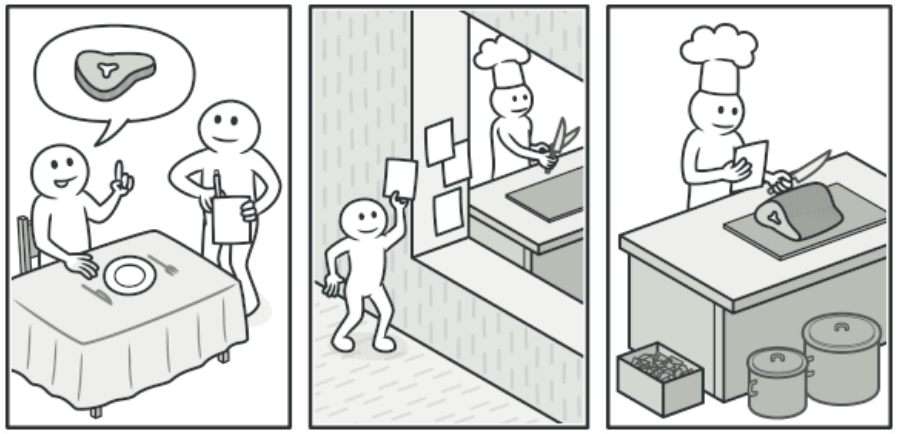
- Potremmo aver notato un pezzo mancante del puzzle, ovvero i parametri della richiesta.
- Un oggetto GUI potrebbe dover fornire all'oggetto del livello di business alcuni parametri.
- Poiché il metodo di esecuzione dei comandi non ha parametri, come passeremmo i dettagli della richiesta al destinatario?
- Il comando dovrebbe essere o preconfigurato con questi dati, o in grado di ottenerli da solo.
- Torniamo al nostro editor di testi.
- Dopo aver applicato il pattern Command, non abbiamo più bisogno di tutte quelle sottoclassi di pulsanti per implementare vari comportamenti di click.
- È sufficiente mettere un singolo campo nella classe base `Button` che memorizza un riferimento a un oggetto comando e far eseguire al pulsante il comando a seguito di un click.
- Si potranno così implementare moltissimi comandi per ogni tipo di applicazione e li si collegheranno con particolari pulsanti, a seconda del comportamento previsto per i pulsanti.

Pattern Comportamentali – Il pattern Command – Soluzione

- Altri elementi dell'interfaccia grafica, come menu, scorciatoie o interi dialoghi, possono essere implementati allo stesso modo.
- Saranno collegati ad un comando che viene eseguito quando un utente interagisce con l'elemento GUI.
- Come probabilmente abbiamo già intuito, gli elementi relativi alle stesse operazioni saranno collegati agli stessi comandi, evitando qualsiasi duplicazione di codice.
- Di conseguenza, i comandi diventano un comodo strato intermedio che riduce l'accoppiamento tra lo strato GUI e quello della logica di business.
- E ciò è solo una parte dei vantaggi che il pattern Command può offrire!
- Consideriamo un'analogia con il mondo reale.
- Dopo una lunga passeggiata per la città, si arriva ad un bel ristorante e ci si siede al tavolo vicino alla finestra.
- Un cameriere amichevole si avvicina e prende rapidamente l'ordine, scrivendo su un pezzo di carta.
- Il cameriere va in cucina e attacca l'ordine al muro.

Pattern Comportamentali – Il pattern Command – Soluzione

- Dopo un po', l'ordine arriva allo chef, che lo legge e cucina il pasto di conseguenza.
- Il cuoco mette il pasto su un vassoio insieme all'ordine.
- Il cameriere trova il vassoio, controlla l'ordine per assicurarsi che tutto sia come desiderato dal cliente e porta tutto al tavolo.



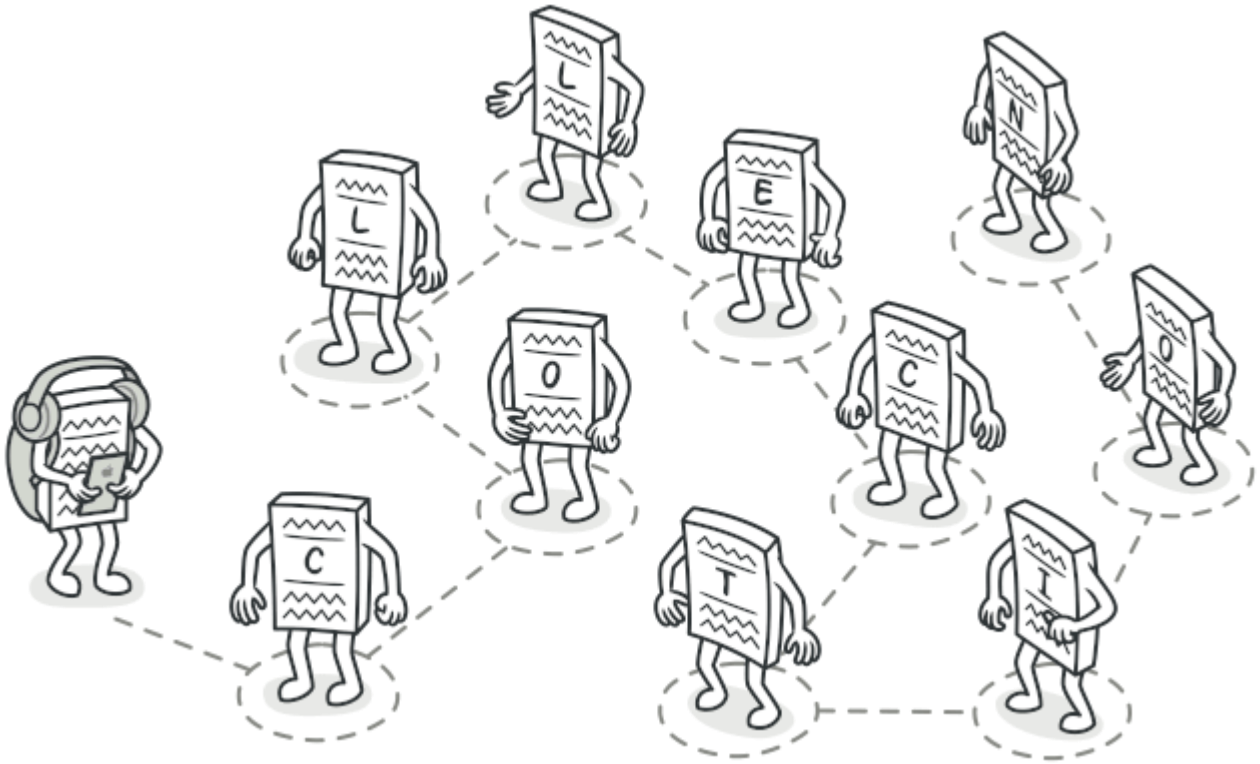
- L'ordine cartaceo serve come comando.
- Rimane in coda fino a quando lo chef non è pronto a servirlo.

Pattern Comportamentali – Il pattern Command – Soluzione

- L'ordine contiene tutte le informazioni necessarie per cucinare il pasto.
- Esso permette allo chef di iniziare subito a cucinare invece di andare in giro a chiarire i dettagli dell'ordine direttamente dal cameriere.

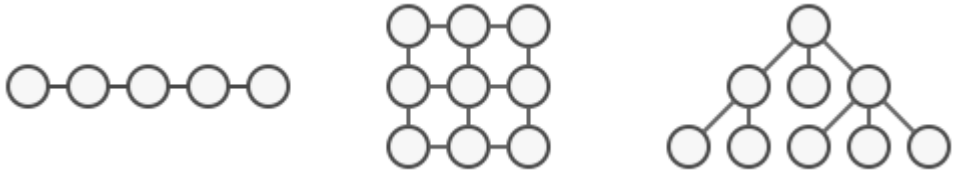
Pattern Comportamentali – Il pattern Iterator – Intent

- Iterator è un design pattern comportamentale che permette di attraversare gli elementi di una collezione senza esporne la rappresentazione sottostante (lista, pila, albero, ecc.).



Pattern Comportamentali – Il pattern Iterator – Problema

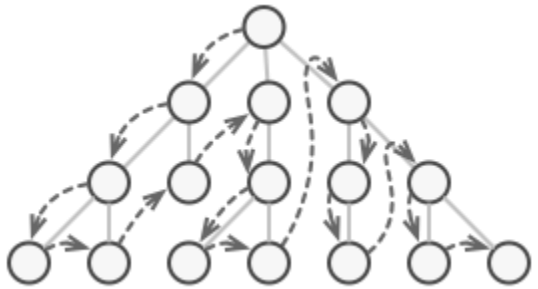
- Le collezioni sono uno dei tipi di dati più utilizzati nella programmazione.
- Tuttavia, una collezione è solo un contenitore per un gruppo di oggetti.

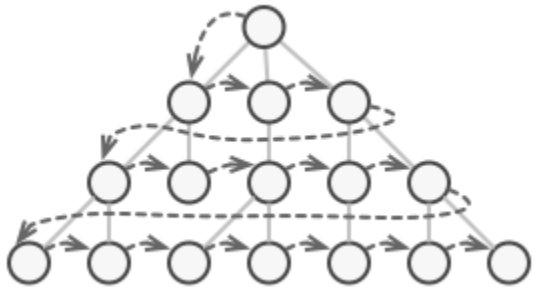


- La maggior parte delle collezioni memorizza i propri elementi in semplici liste.
- Tuttavia, alcune di esse si basano su pile, alberi, grafici e altre strutture di dati complessi.
- Ma indipendentemente da come è strutturata una collezione, essa deve fornire un modo per accedere ai suoi elementi in modo che un altro codice possa usare questi elementi.
- Ci dovrebbe essere un modo per attraversare ogni elemento della collezione senza accedere agli stessi elementi più e più volte.

Pattern Comportamentali – Il pattern Iterator – Problema

- Questo può sembrare un lavoro facile se si dispone di una collezione basata su una lista. Basta fare un loop su tutti gli elementi.
- Ma come si fa ad attraversare in sequenza gli elementi di una struttura dati complessa, come un albero?
- Per esempio, un giorno potremmo essere a nostro agio con l'attraversamento in profondità di un albero.
- Ma il giorno dopo potrebbe essere necessaria un attraversamento in ampiezza.
- E la settimana dopo, potremmo aver bisogno di qualcos'altro, come l'accesso casuale agli elementi dell'albero.



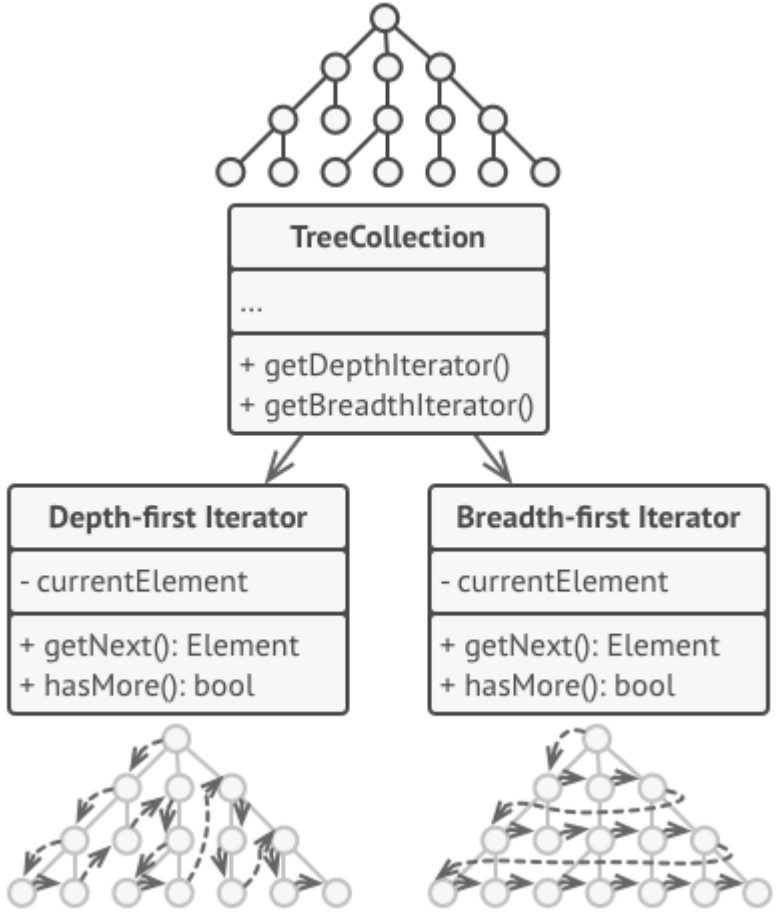


Pattern Comportamentali – Il pattern Iterator – Problema

- L'aggiunta di un numero sempre maggiore di algoritmi di attraversamento alla collezione ne offusca gradualmente la responsabilità primaria, che è l'efficiente archiviazione dei dati.
- Inoltre, alcuni algoritmi potrebbero essere personalizzati per una specifica applicazione, quindi sarebbe strano includerli in una classe di raccolta generica.
- D'altra parte, il codice client che dovrebbe funzionare con varie collezioni potrebbe non curarsi nemmeno di come memorizzano i loro elementi.
- Tuttavia, dato che le collezioni forniscono tutti modi diversi di accedere ai loro elementi, non si ha altra scelta se non quella di accoppiare il codice alle specifiche classi di collezione.

Pattern Comportamentali – Il pattern Iterator – Soluzione

- L'idea principale del pattern Iterator è quella di estrarre il comportamento di attraversamento di una collezione in un oggetto separato chiamato iteratore.

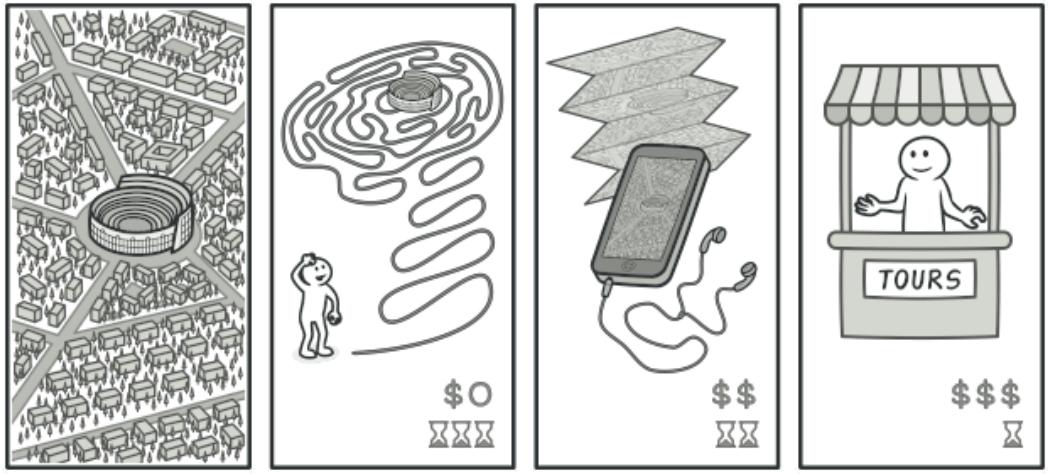


Pattern Comportamentali – Il pattern Iterator – Soluzione

- Oltre ad implementare l'algoritmo stesso, un oggetto iteratore racchiude tutti i dettagli dell'attraversamento, come la posizione corrente e quanti elementi sono rimasti per arrivare alla fine.
- Per questo motivo, più iteratori possono attraversare la stessa collezione allo stesso tempo, indipendentemente l'uno dall'altro.
- Di solito, gli iteratori forniscono un metodo principale per il recupero degli elementi della collezione.
- Il client può continuare a utilizzare questo metodo fino a quando non restituisce nulla, il che significa che l'iteratore ha attraversato tutti gli elementi.
- Tutti gli iteratori devono implementare la stessa interfaccia.
- Questo rende il codice del client compatibile con qualsiasi tipo di collezione o con qualsiasi algoritmo di attraversamento, purché ci sia un iteratore appropriato.
- Se si ha bisogno di un modo speciale per attraversare una collezione, basta creare una nuova classe iteratore, senza dover cambiare la collezione o il client.

Pattern Comportamentali – Il pattern Iterator – Soluzione

- Consideriamo un'analogia derivata dal mondo reale.
- Abbiamo in programma di visitare Roma per qualche giorno e di visitare tutti i suoi principali luoghi e attrazioni.
- Ma una volta lì, potremmo perdere un sacco di tempo a camminare a vuoto, incapaci di trovare anche il Colosseo.
- D'altra parte, potremmo acquistare un'applicazione di guida virtuale per il nostro smartphone e usarla per la navigazione.
- È intelligente e poco costosa, e si potrebbe soggiornare in alcuni luoghi interessanti per tutto il tempo che si desidera.



Pattern Comportamentali – Il pattern Iterator – Soluzione

- Una terza alternativa è che si potrebbe spendere una parte del budget del viaggio e richiedere il supporto di una guida locale che conosca la città come il palmo della sua mano.

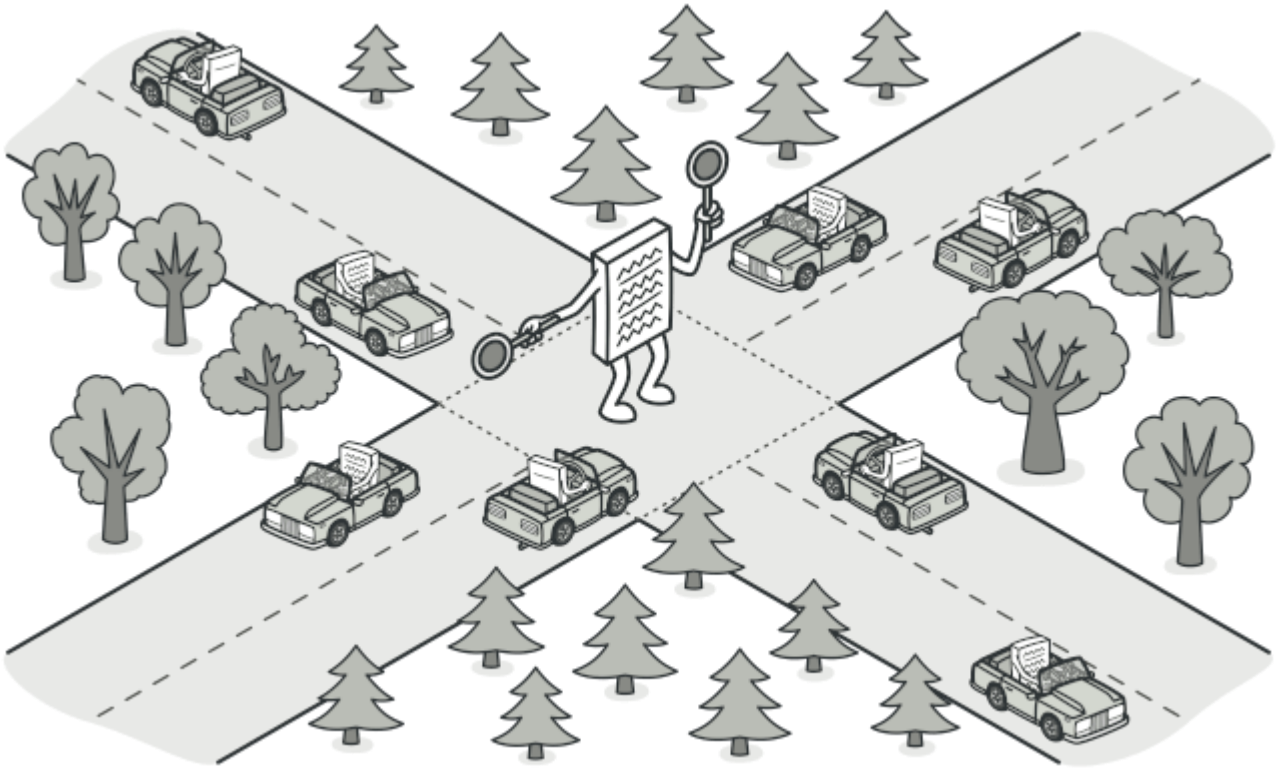
- La guida sarebbe in grado di adattare il tour ai nostri gusti, di mostrarci ogni attrazione e di raccontarci tante storie emozionanti.

- Sarà ancora più divertente, ma, ahimè, anche più costoso.

- Tutte queste opzioni - le direzioni casuali che nascono nella nostra testa, il navigatore dello smartphone o la guida umana - agiscono come iteratori sulla vasta collezione di attrazioni e monumenti di Roma.

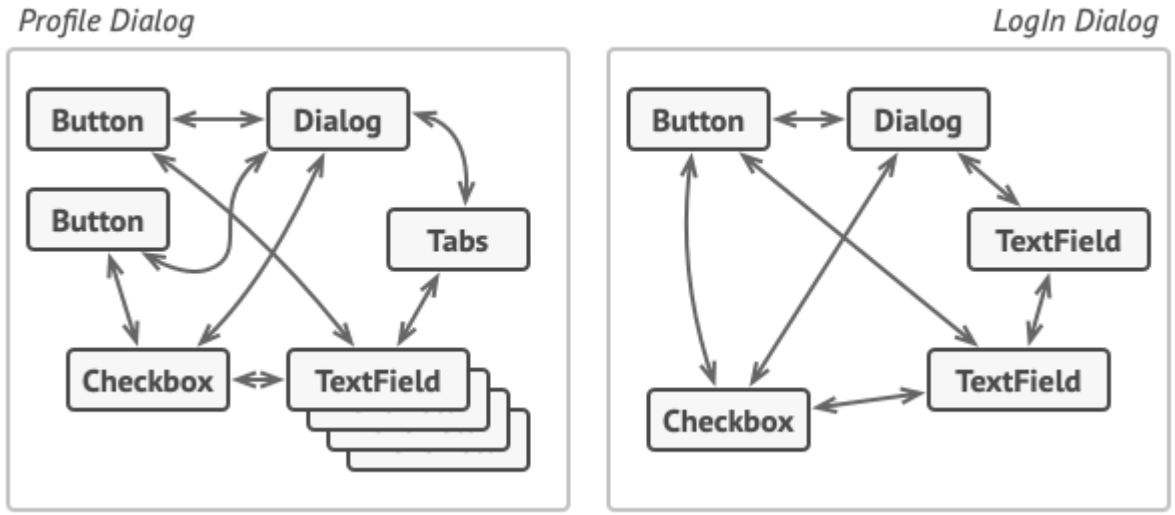
Pattern Comportamentali – Il pattern Mediator – Intent

- Mediator è un design pattern comportamentale che consente di ridurre le dipendenze caotiche tra gli oggetti.
- Il pattern limita le comunicazioni dirette tra gli oggetti e li costringe a collaborare solo attraverso un oggetto mediatore.



Pattern Comportamentali – Il pattern Mediator – Problema

- Supponiamo di avere una finestra di dialogo per la creazione e la modifica dei profili dei clienti.
- Essa consiste di controlli di vario tipo come campi di testo, checkbox, pulsanti, ecc.



- Alcuni elementi della form possono interagire con altri.
- Per esempio, selezionando la checkbox "Ho un cane" può apparire un campo di testo nascosto per l'inserimento del nome del cane.
- Un altro esempio è il pulsante di invio che deve convalidare i valori di tutti i campi prima di salvare i dati.

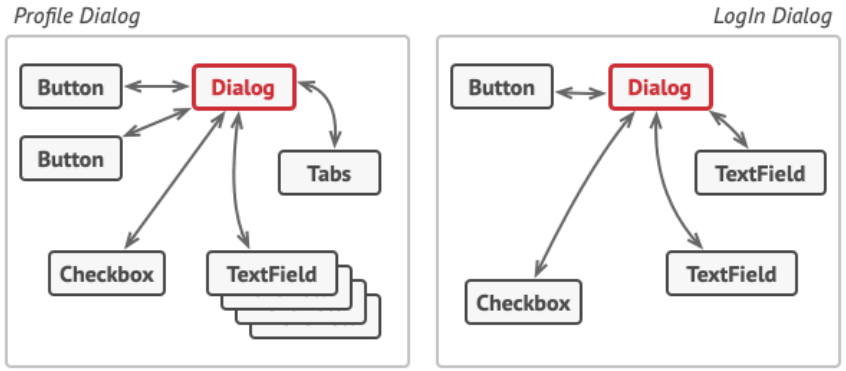
Pattern Comportamentali – Il pattern Mediator – Problema



- Avendo questa logica implementata direttamente all'interno del codice degli elementi della form, si rende molto più difficile riutilizzare le classi di questi elementi in altre form dell'applicazione.
- Per esempio, non sarà possibile usare quella classe checkbox all'interno di un altro form, perché è accoppiata al campo di testo del cane.
- Si possono usare tutte le classi coinvolte nel form del profilo, oppure nessuna.

Pattern Comportamentali – Il pattern Mediator – Soluzione

- Il pattern Mediator suggerisce di interrompere ogni comunicazione diretta tra i componenti che si desidera rendere indipendenti l'uno dall'altro.
- Questi componenti devono invece collaborare indirettamente, chiamando uno speciale oggetto mediatore che reindirizza le chiamate ai componenti appropriati.
- Di conseguenza, i componenti dipendono solo da una singola classe mediatore, invece di essere accoppiati a decine di loro colleghi.
- Nel nostro esempio con il form di editing del profilo, la classe di dialogo stessa può fungere da mediatore.
- Molto probabilmente, la classe di dialogo è già a conoscenza di tutti i suoi sottoelementi, quindi non sarà nemmeno necessario introdurre nuove dipendenze in questa classe.



Pattern Comportamentali – Il pattern Mediator – Soluzione

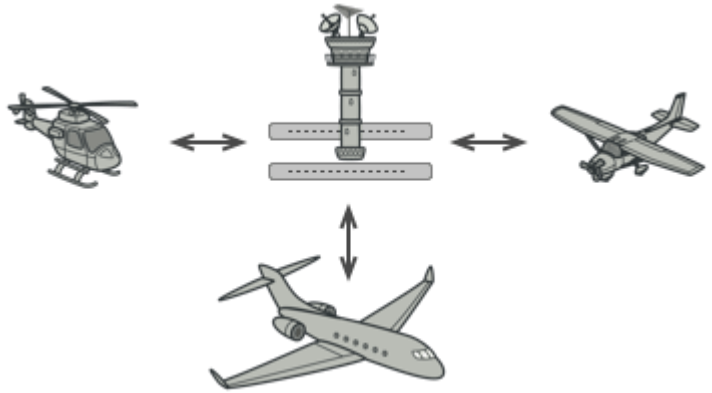
- Il cambiamento più significativo riguarda gli effettivi elementi della form.
- Consideriamo il pulsante di invio.
- In precedenza, ogni volta che un utente cliccava il pulsante, doveva convalidare i valori di tutti i singoli elementi del modulo.
- Ora il suo unico compito è quello di notificare alla finestra di dialogo il click.
- Al ricevimento di questa notifica, la finestra di dialogo stessa esegue le convalide o passa il compito ai singoli elementi.
- Così, invece di essere legato a una dozzina di elementi del modulo, il pulsante dipende solo dalla classe della finestra di dialogo.
- È possibile andare oltre e rendere la dipendenza ancora più allentata estraendo l'interfaccia comune per tutti i tipi di finestre di dialogo.
- L'interfaccia dichiarerebbe il metodo di notifica che tutti gli elementi del modulo possono utilizzare per notificare alla finestra di dialogo gli eventi che accadono a tali elementi.

Pattern Comportamentali – Il pattern Mediator – Soluzione

- Così, il nostro pulsante di invio dovrebbe ora essere in grado di lavorare con qualsiasi finestra di dialogo che implementi tale interfaccia.
- In questo modo, il pattern Mediator permette di incapsulare una complessa rete di relazioni tra vari oggetti all'interno di un singolo oggetto mediatore.
- Meno dipendenze ha una classe, più facile diventa modificare, estendere o riutilizzare quella classe.
- Consideriamo un'analogia con il mondo reale.
- I piloti degli aerei che si avvicinano o partono dall'area di controllo dell'aeroporto non comunicano direttamente tra loro.
- Parlano invece con un controllore del traffico aereo, che siede in una torre alta da qualche parte vicino alla pista di atterraggio.
- Senza il controllore del traffico aereo, i piloti dovrebbero essere a conoscenza di ogni aereo nelle vicinanze dell'aeroporto, discutendo le priorità di atterraggio con un comitato di decine di altri piloti.
- Questo probabilmente farebbe salire alle stelle le statistiche sugli incidenti aerei.

Pattern Comportamentali – Il pattern Mediator – Soluzione

- La torre non ha bisogno di controllare l'intero volo.
- Esiste solo per far rispettare i vincoli nell'area del terminal perché il numero di attori coinvolti potrebbe essere schiacciante per un pilota.



Pattern Comportamentali – Il pattern Memento – Intent

- Memento è un design pattern comportamentale che permette di salvare e ripristinare lo stato precedente di un oggetto senza rivelare i dettagli della sua implementazione.

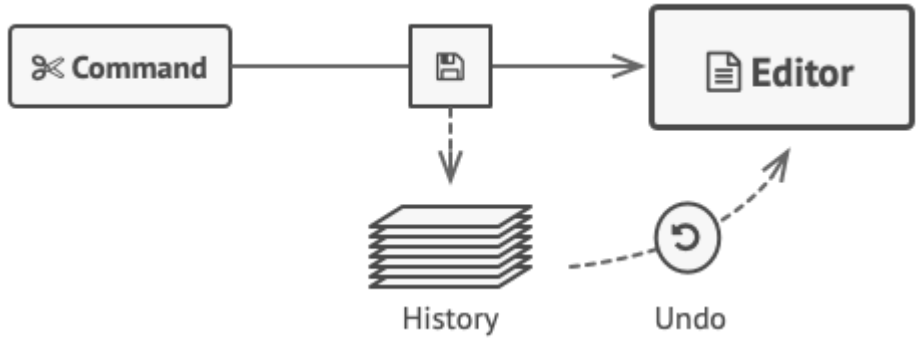


Pattern Comportamentali – Il pattern Memento – Problema

- Immaginiamo di creare un'applicazione per l'editor di testi.
- Oltre alla semplice modifica del testo, il nostro editor può formattare il testo, inserire immagini online, ecc.
- Ad un certo punto, abbiamo deciso di consentire che gli utenti annullino le operazioni effettuate sul testo.
- Questa caratteristica è diventata così comune nel corso degli anni che oggi la gente si aspetta che ogni app la possieda.
- Per l'implementazione, abbiamo scelto di adottare l'approccio diretto.
- Prima di eseguire qualsiasi operazione, l'app registra lo stato di tutti gli oggetti e lo salva da qualche parte nella memoria.
- Più tardi, quando un utente decide di annullare un'azione, l'app recupera l'ultima istantanea della storia e la utilizza per ripristinare lo stato di tutti gli oggetti.
- Pensiamo a tutte queste istantanee dello stato. Come verrebbe prodotta una di esse, esattamente?
- Probabilmente dovremmo esaminare tutti i campi di un oggetto e copiarne i valori in memoria.

Pattern Comportamentali – Il pattern Memento – Problema

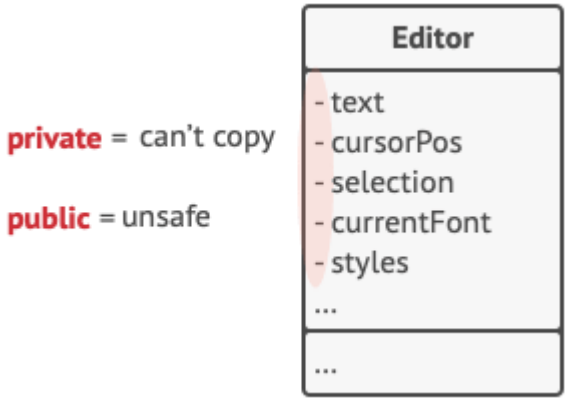
- Tuttavia, questo funzionerebbe solo se l'oggetto avesse restrizioni di accesso abbastanza rilassate ai suoi contenuti.
- Purtroppo, la maggior parte degli oggetti reali non permette agli altri di sbirciarci dentro così facilmente, nascondendo tutti i dati significativi in campi privati.



- Ignoriamo per ora questo problema e supponiamo che i nostri oggetti si comportino come gli hippy, preferendo relazioni aperte e mantenendo pubblico il loro stato.
- Se da un lato questo approccio risolverebbe il problema immediato e permetterebbe di produrre istantanee degli stati degli oggetti a piacimento, dall'altro ha comunque dei seri problemi.
- In futuro, potremmo decidere di ristrutturare alcune delle classi dell'editor, o di aggiungere o rimuovere alcuni campi.

Pattern Comportamentali – Il pattern Memento – Problema

- Sembra facile, ma questo richiederebbe anche il cambiamento delle classi responsabili della copia dello stato degli oggetti interessati.



- Ma c'è di più. Consideriamo lo snapshot corrente dello stato dell'editor. Quali dati contiene?
- Come minimo, deve contenere il testo vero e proprio, le coordinate del cursore, la posizione corrente dello scorrimento, ecc.
- Per fare uno snapshot, è necessario raccogliere questi valori e metterli in una specie di contenitore.
- Molto probabilmente, si andrà a memorizzare molti di questi oggetti container all'interno di qualche lista che rappresenterebbe la storia.

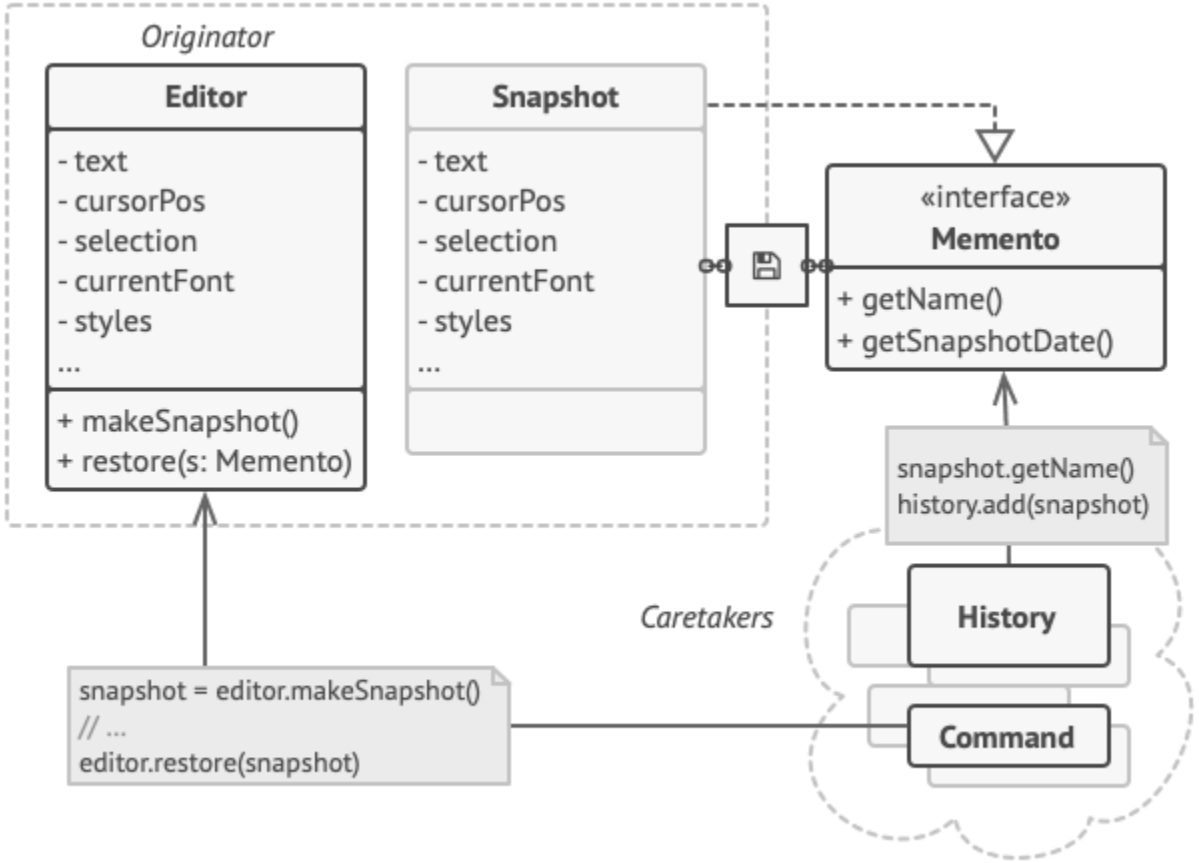
Pattern Comportamentali – Il pattern Memento – Problema

- Quindi i contenitori finirebbero probabilmente per essere oggetti di una classe.
- La classe non avrebbe quasi nessun metodo, ma molti campi che rispecchiano lo stato dell'editor.
- Per consentire ad altri oggetti di scrivere e leggere i dati da e verso uno snapshot, è probabilmente necessario rendere pubblici i suoi campi.
- Questo esporrebbe tutti gli stati dell'editor, privati o meno.
- Altre classi diventerebbero dipendenti da ogni piccolo cambiamento della classe dello snapshot, che altrimenti avverrebbe all'interno di campi e metodi privati senza influenzare le classi esterne.
- Sembra che siamo arrivati a un vicolo cieco: o si espongono tutti i dettagli interni delle classi, rendendole troppo fragili, o si limita l'accesso al loro stato, rendendo impossibile la produzione di snapshot.
- C'è un altro modo per implementare l'«undo»?

Pattern Comportamentali – Il pattern Memento – Soluzione

- Tutti i problemi che abbiamo appena considerato sono causati dal fatto che viene indebolito il principio di incapsulamento.
- Alcuni oggetti cercano di fare più di quanto dovrebbero.
- Per raccogliere i dati necessari a compiere una certa azione, invadono lo spazio privato di altri oggetti invece di lasciare che questi ultimi compiano l'azione vera e propria.
- Il pattern Memento delega la creazione degli snapshot dello stato all'effettivo proprietario di quello stato, l'oggetto originator.
- Quindi, invece di altri oggetti che cercano di copiare lo stato dell'editor dall'"esterno", la classe editor stessa può creare lo snapshot poiché ha pieno accesso al proprio stato.
- Il pattern suggerisce di memorizzare la copia dello stato dell'oggetto in un oggetto speciale chiamato memento.
- Il contenuto del memento non è accessibile a nessun altro oggetto se non a quello che lo ha prodotto.
- Altri oggetti devono comunicare con i memento utilizzando un'interfaccia limitata che può permettere di recuperare i metadati dello snapshot (tempo di creazione, nome dell'operazione eseguita, ecc.), ma non lo stato dell'oggetto originale contenuto nello snapshot.

Pattern Comportamentali – Il pattern Memento – Soluzione



- Una politica così restrittiva consente di conservare i memento all'interno di altri oggetti, solitamente chiamati caretaker (custodi).

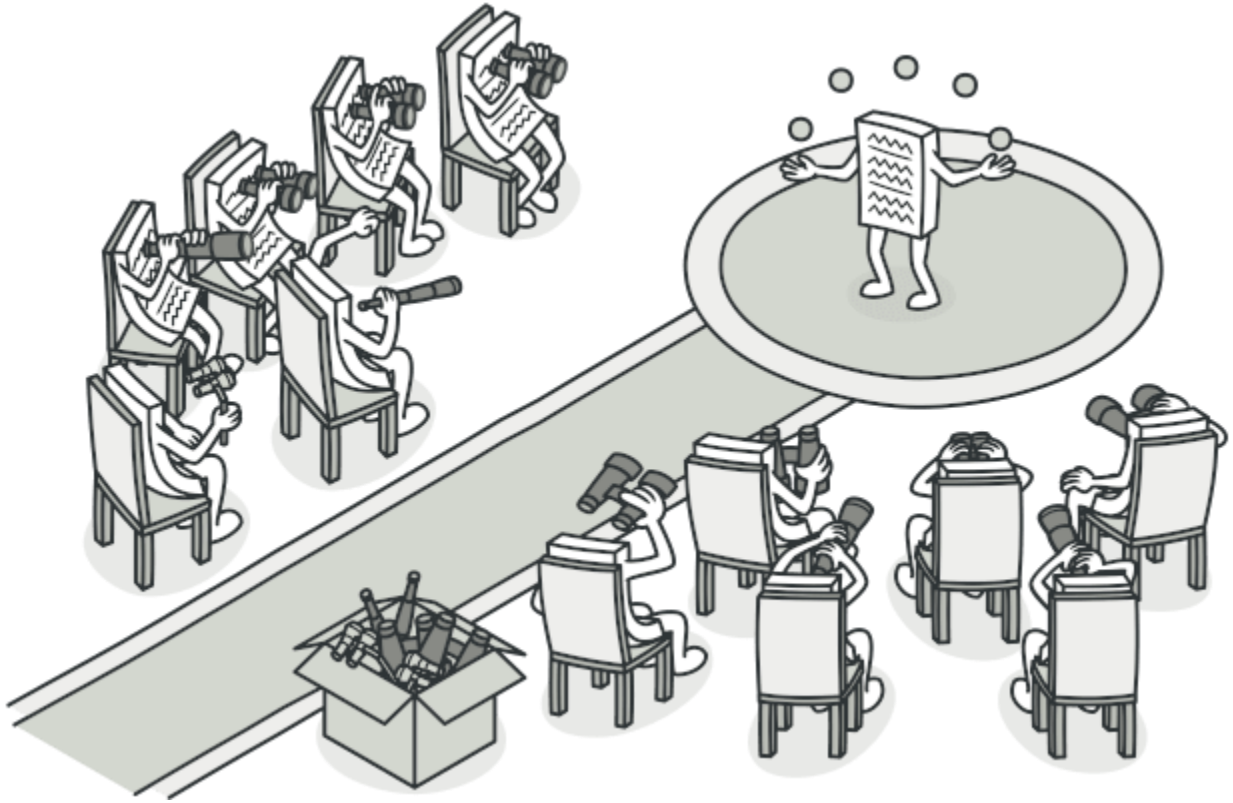
- Poiché il caretaker opera con il memento solo attraverso l'interfaccia limitata, non è in grado di manomettere lo stato memorizzato all'interno del memento.

Pattern Comportamentali – Il pattern Memento – Soluzione

- Allo stesso tempo, l'originator ha accesso a tutti i campi all'interno del memento, il che gli permette di ripristinare lo stato precedente a suo piacimento.
- Nel nostro esempio di editor di testo, possiamo creare una classe history separata per fare da caretaker.
- Una pila di memento memorizzati all'interno del caretaker crescerà ogni volta che l'editor sta per eseguire un'operazione.
- Si potrebbe anche rendere questa pila all'interno dell'interfaccia utente dell'applicazione, visualizzando ad un utente la cronologia delle operazioni eseguite in precedenza.
- Quando un utente attiva l'undo, la history preleva il memento più recente dalla pila e lo ritrasmette all'editor, richiedendo un roll-back.
- Poiché l'editor ha pieno accesso al memento, cambia il proprio stato con i valori presi dal memento.

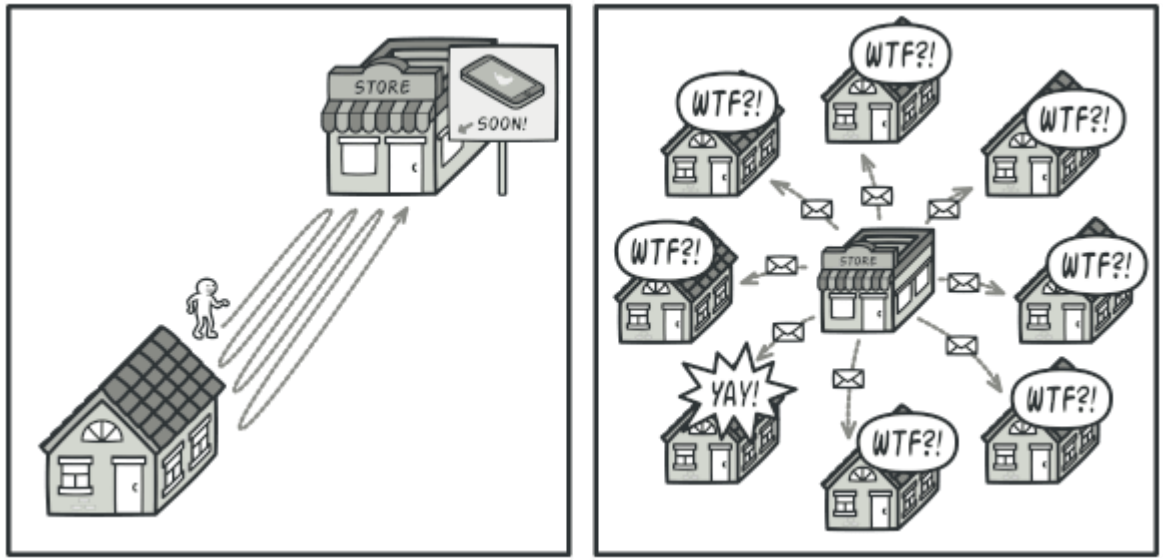
Pattern Comportamentali – Il pattern Observer – Intent

- Observer è un design pattern comportamentale che permette di definire un meccanismo di sottoscrizione per notificare a più oggetti gli eventi che accadono all'oggetto che stanno osservando.



Pattern Comportamentali – Il pattern Observer – Problema

- Immaginiamo di avere due tipi di oggetti: un `Customer` e uno `Store`.
- Il cliente è molto interessato a una particolare marca di prodotto (supponiamo ad un nuovo modello di iPhone) che dovrebbe essere disponibile nel negozio molto presto.
- Il cliente potrebbe visitare il negozio ogni giorno e verificare la disponibilità del prodotto.
- Ma mentre il prodotto è ancora in viaggio, la maggior parte di questi viaggi sarebbe inutile.

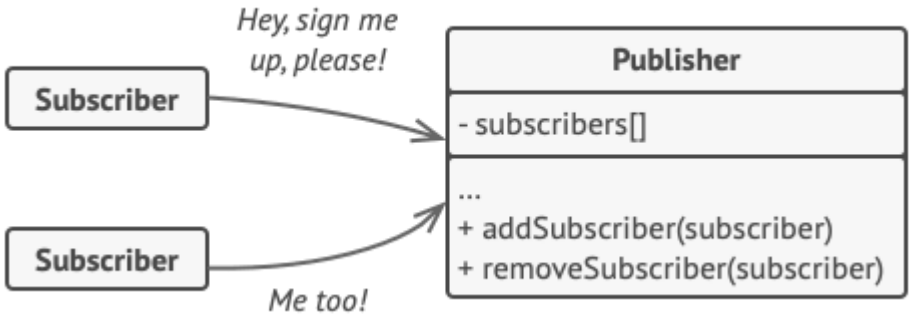


Pattern Comportamentali – Il pattern Observer – Problema

- In alternativa, il negozio potrebbe inviare tonnellate di e-mail (che potrebbero essere considerate spam) a tutti i clienti ogni volta che un nuovo prodotto diventa disponibile.
- In questo modo alcuni clienti sarebbero risparmiati da viaggi infiniti verso il negozio.
- Questo modo di procedere, però, sconvolgerebbe altri clienti che non sono interessati a nuovi prodotti.
- Sembra che ci sia un conflitto. O il cliente perde tempo a controllare la disponibilità del prodotto o il negozio spreca risorse inviando notifiche ai clienti sbagliati.

Pattern Comportamentali – Il pattern Observer – Soluzione

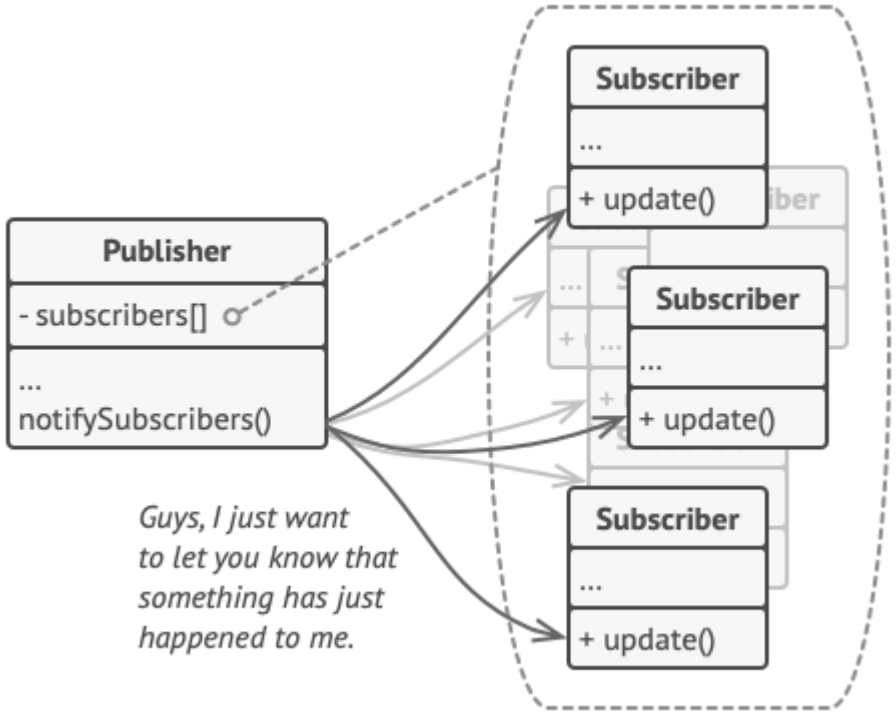
- L'oggetto che ha uno stato interessante viene spesso chiamato *subject*, ma dato che notificherà anche ad altri oggetti i cambiamenti del suo stato, lo chiameremo *publisher*.
- Tutti gli altri oggetti che vogliono seguire le modifiche allo stato dell'editore sono chiamati *subscriber*.
- Il pattern Observer suggerisce di aggiungere un meccanismo di sottoscrizione alla classe del publisher in modo che i singoli oggetti possano iscriversi o cancellarsi da un flusso di eventi provenienti da quel publisher.
- Non temete! Non è tutto così complicato come sembra.
- In realtà, questo meccanismo consiste in 1) un campo array per memorizzare una lista di riferimenti agli oggetti subscriber e 2) diversi metodi pubblici che permettono di aggiungere e rimuovere i subscriber da quella lista.



Pattern Comportamentali – Il pattern Observer – Soluzione

- Ora, ogni volta che un evento importante accade al publisher, questo individua i suoi subscriber e chiama il metodo di notifica specifico sui loro oggetti.
- Le applicazioni reali potrebbero avere decine di classi subscriber diverse interessate a tracciare gli eventi della stessa classe publisher.
- Non vorremmo accoppiare il publisher a tutte queste classi.
- Inoltre, potremmo anche non sapere in anticipo se la nostra classe publisher è destinata ad essere usata da altre persone.
- Ecco perché è fondamentale che tutti i subscriber implementino la stessa interfaccia e che il publisher comunichi con loro solo attraverso tale interfaccia.
- Questa interfaccia dovrebbe dichiarare il metodo di notifica insieme ad un insieme di parametri che il publisher può usare per passare alcuni dati contestuali insieme alla notifica.

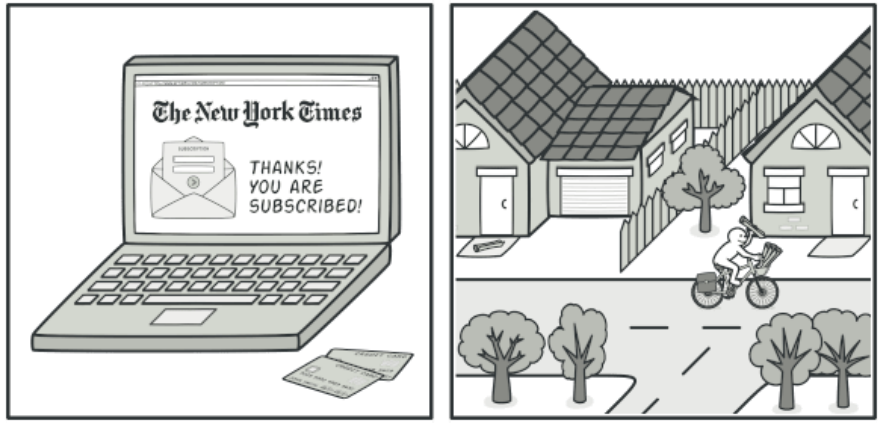
Pattern Comportamentali – Il pattern Observer – Soluzione



- Se la nostra applicazione ha diversi tipi di publisher e vogliamo rendere i nostri subscriber compatibili con tutti i publisher, possiamo andare ancora più oltre e far sì che tutti i publisher seguano la stessa interfaccia.
- Questa interfaccia dovrebbe solo descrivere alcuni metodi di sottoscrizione.
- L'interfaccia permetterebbe ai subscriber di osservare gli stati dei publisher senza doversi accoppiare alle loro classi concrete.

Pattern Comportamentali – Il pattern Observer – Soluzione

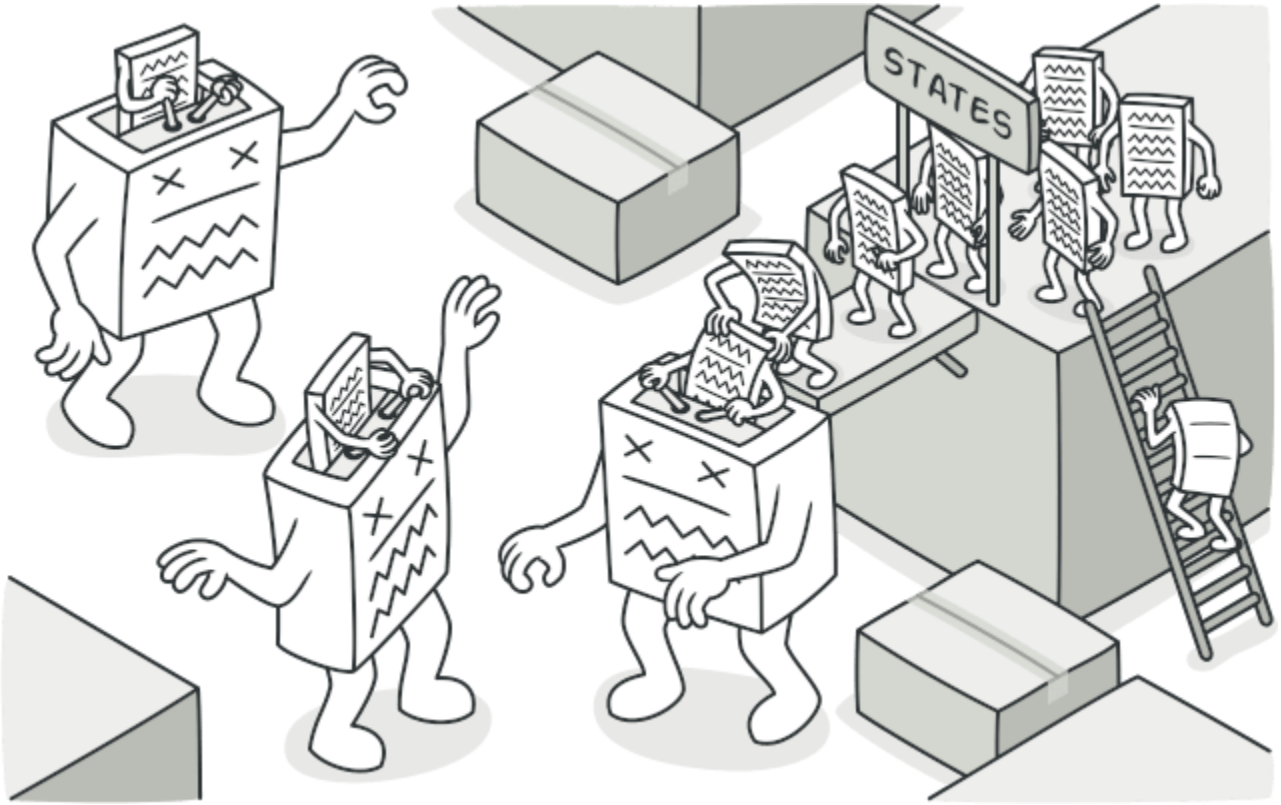
- Consideriamo un'analogia dal mondo reale.
- Se siamo abbonati a un giornale o a una rivista, non abbiamo più bisogno di andare in negozio per verificare se il prossimo numero è disponibile.
- Al contrario, l'editore invia i nuovi numeri direttamente alla nostra casella di posta subito dopo la pubblicazione o addirittura in anticipo.
- L'editore mantiene una lista di abbonati e sa a quali riviste sono interessati.
- Gli abbonati possono cancellare l'abbonamento in qualsiasi momento quando non vogliono più ricevere riviste da parte dell'editore.



Pattern Comportamentali – Il pattern State – Intent

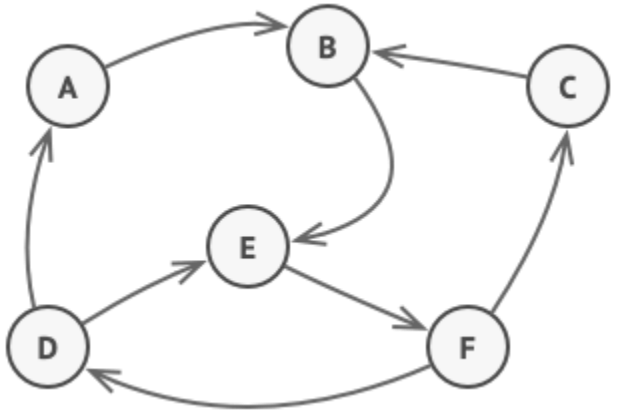
- State è un design pattern comportamentale che permette ad un oggetto di modificare il suo comportamento quando il suo stato interno cambia.

- Sembra che l'oggetto abbia cambiato la sua classe.



Pattern Comportamentali – Il pattern State – Problema

- Il pattern State è strettamente legato al concetto di Macchina a Stati Finiti.



- L'idea principale è che, in ogni momento, c'è un numero finito di stati in cui un programma può trovarsi.
- All'interno di qualsiasi stato unico, il programma si comporta in modo diverso, e il programma può passare da uno stato all'altro istantaneamente.
- Tuttavia, a seconda dello stato corrente, il programma può passare o meno ad un determinato altro stato.
- Queste regole di commutazione, chiamate transizioni, sono anch'esse finite e predeterminate.

Pattern Comportamentali – Il pattern State – Problema

- Questo approccio può essere applicato anche agli oggetti.
- Immaginiamo di avere una classe `Document`.
- Un documento può essere in uno dei tre stati: `Draft`, `Moderation` e `Published`.



Pattern Comportamentali – Il pattern State – Problema

- Il metodo `publish` del documento funziona in modo leggermente diverso in ogni stato:
  - In `Draft`, sposta il documento alla moderazione.
  - In `Moderation`, rende il documento pubblico, ma solo se l'utente corrente è un amministratore.
  - In `Published`, non fa assolutamente nulla.
- Le macchine a stati sono solitamente implementate con molti operatori condizionali (`if` o `switch`) che selezionano il comportamento appropriato a seconda dello stato corrente dell'oggetto.
- Di solito, questo "stato" è solo un insieme di valori dei campi dell'oggetto.
- Anche chi non ha mai sentito parlare di macchine a stati finiti, probabilmente ha implementato uno stato almeno una volta.

Pattern Comportamentali – Il pattern State – Problema

- Quante volte ci è capitato, ad esempio, di gestire codice come questo?

```
Class Document is
   field state: string
   // …
   method publish () is
      switch (state)
         "draft":
            state = "moderation"
            break;
         "moderation":
            if (currentUser.role == ‘admin’)
               state = "published"
            break;
         "published":
            // Do nothing.
            break
      // …
```

- La più grande debolezza di una macchina a stati basata sulle condizioni si rivela una volta che iniziamo ad aggiungere sempre più stati e comportamenti dipendenti dallo stato alla classe `Document`.

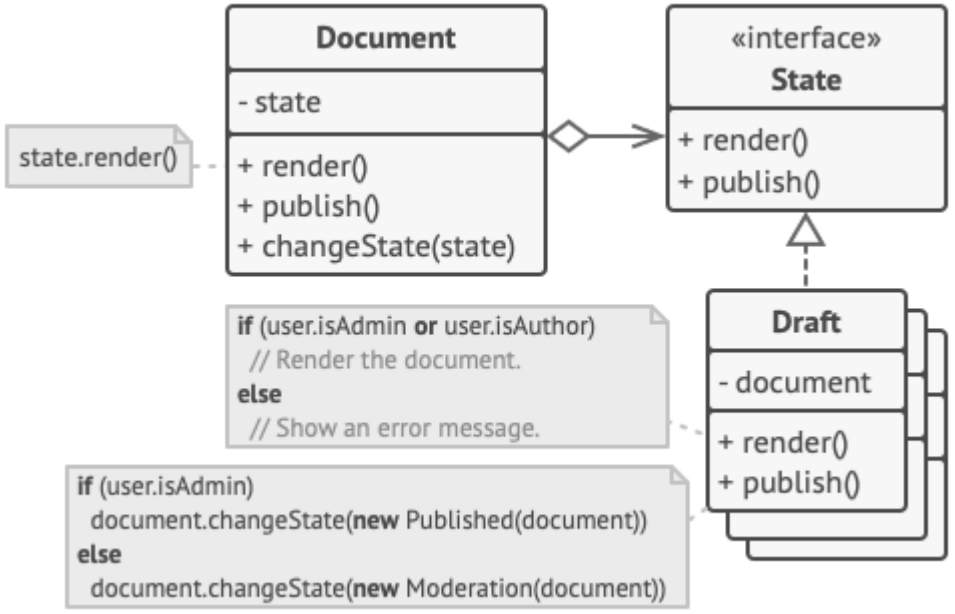
Pattern Comportamentali – Il pattern State – Problema

- La maggior parte dei metodi conterrà istruzioni condizionali mostruose che scelgono il comportamento corretto del metodo in base allo stato attuale.
- Un codice come questo è molto difficile da mantenere perché qualsiasi cambiamento nella logica di transizione può richiedere il cambiamento delle istruzioni condizionali relative allo stato in ogni metodo.
- Il problema tende a diventare più grande man mano che un progetto si evolve.
- È abbastanza difficile prevedere tutti i possibili stati e le possibili transizioni durante la fase di progettazione.
- Quindi, una macchina a stati snella costruita con un insieme limitato di istruzioni condizionali può crescere nel tempo fino a diventare un vero e proprio «casino».

Pattern Comportamentali – Il pattern State – Soluzione

- Il pattern State suggerisce di creare nuove classi per tutti i possibili stati di un oggetto e di estrarre tutti i comportamenti specifici dello stato in queste classi.

- Invece di implementare tutti i comportamenti da solo, l'oggetto originale, chiamato contesto, memorizza un riferimento a uno degli oggetti di stato che rappresenta il suo stato corrente, e delega tutto il lavoro relativo allo stato a quell'oggetto.



Pattern Comportamentali – Il pattern State – Soluzione

- Per passare il contesto in un altro stato, sostituiamo l'oggetto di stato attivo con un altro oggetto che rappresenta il nuovo stato.
- Questo è possibile solo se tutte le classi di stato seguono la stessa interfaccia e il contesto stesso opera con questi oggetti attraverso tale interfaccia.
- Questa struttura può sembrare simile al pattern Strategy, ma c'è una differenza fondamentale.
- Nel pattern State, gli stati particolari possono essere consapevoli l'uno dell'altro e avviare transizioni da uno stato all'altro, mentre le strategie non si conoscono tra loro quasi mai.
- Come analogia con il mondo reale, consideriamo i pulsanti e gli interruttori dello smartphone. Questi si comportano in modo diverso a seconda dello stato attuale del dispositivo:
  - Quando il telefono è sbloccato, la pressione dei tasti porta all'esecuzione di varie funzioni.
  - Quando il telefono è bloccato, premendo un pulsante qualsiasi si accede alla schermata di sblocco.
  - Quando la carica del telefono è bassa, premendo un pulsante qualsiasi viene visualizzata la schermata di ricarica.

Pattern Comportamentali – Il pattern Strategy – Intent

- Strategy è un design pattern comportamentale che permette di definire una famiglia di algoritmi, di mettere ciascuno di essi in una classe separata e di rendere i loro oggetti intercambiabili.

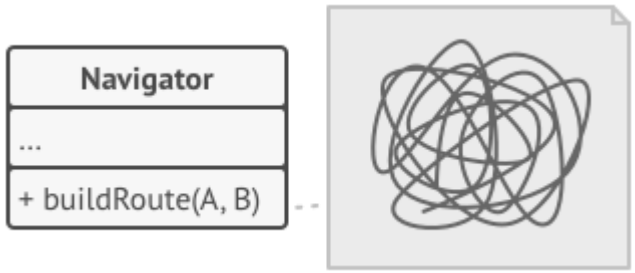


Pattern Comportamentali – Il pattern Strategy – Problema

- Supponiamo che un giorno abbiamo deciso di creare un'applicazione di navigazione per i viaggiatori occasionali.
- L'applicazione era incentrata su una bella mappa che aiutava gli utenti a orientarsi rapidamente in qualsiasi città.
- Una delle caratteristiche più richieste per l'app era la pianificazione automatica dei percorsi.
- L'utente doveva essere in grado di inserire un indirizzo e di vedere il percorso più veloce per raggiungere la destinazione visualizzata sulla mappa.
- La prima versione dell'app poteva costruire i percorsi solo su strade. Le persone che viaggiavano in auto scoppiavano di gioia. Ma a quanto pare, non tutti amano guidare in vacanza.
- Così, con il successivo aggiornamento, è stata aggiunta un'opzione per costruire percorsi a piedi.
- Subito dopo, è stata aggiunta un'altra opzione per permettere alle persone di usare i mezzi pubblici nei loro percorsi.
- Tuttavia, questo era solo l'inizio. In seguito si è pianificato di aggiungere la costruzione di percorsi per i ciclisti.
- E ancora più tardi, un'altra opzione per costruire percorsi attraverso tutte le attrazioni turistiche di una città.

Pattern Comportamentali – Il pattern Strategy – Problema

- Mentre dal punto di vista del business l'app è stata un successo, la parte tecnica ci ha causato molti mal di testa.
- Ogni volta che si aggiungeva un nuovo algoritmo di routing, la classe principale del navigatore raddoppiava di dimensioni.
- Ad un certo punto, l'app è diventata troppo difficile da mantenere.



- Qualsiasi modifica a uno degli algoritmi, sia che si trattasse di una semplice correzione di un bug o di un leggero aggiustamento del punteggio della strada, interessava l'intera classe, aumentando la possibilità di creare un errore nel codice già funzionante.
- Inoltre, il lavoro di squadra è diventato inefficiente.
- I nostri compagni di squadra, che erano stati assunti subito dopo il rilascio con successo, si lamentano di aver passato troppo tempo a risolvere i conflitti di fusione.

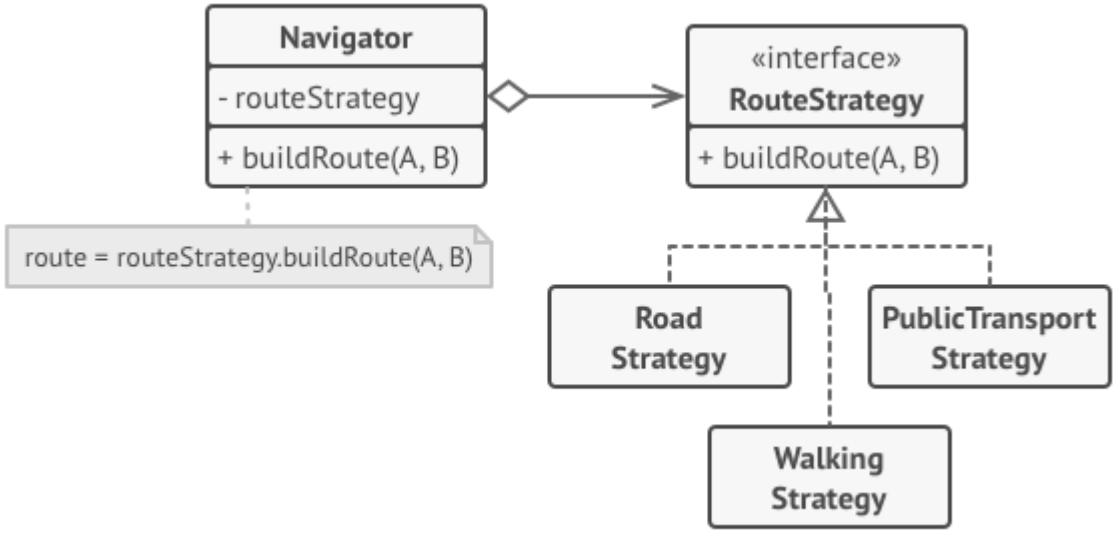
Pattern Comportamentali – Il pattern Strategy – Problema

- L'implementazione di una nuova funzionalità richiede di cambiare la stessa enorme classe, in conflitto con il codice prodotto da altre persone.

Pattern Comportamentali – Il pattern Strategy – Soluzione

- Il pattern Strategy suggerisce di prendere una classe che fa qualcosa di specifico in molti modi diversi ed estrarre tutti questi algoritmi in classi separate chiamate *strategie*.
- La classe originale, chiamata *contesto*, deve avere un campo per memorizzare un riferimento a una delle strategie.
- Il contesto delega il lavoro a un oggetto strategia collegato invece di eseguirlo da solo.
- Il contesto non è responsabile della selezione di un algoritmo appropriato per il lavoro.
- Al contrario, il client passa la strategia desiderata al contesto.
- In realtà, il contesto non sa molto sulle strategie.
- Lavora con tutte le strategie attraverso la stessa interfaccia generica, che espone solo un singolo metodo per l'attivazione dell'algoritmo incapsulato all'interno della strategia selezionata.
- In questo modo il contesto diventa indipendente dalle strategie concrete, in modo da poter aggiungere nuovi algoritmi o modificare quelli esistenti senza modificare il codice del contesto o altre strategie.

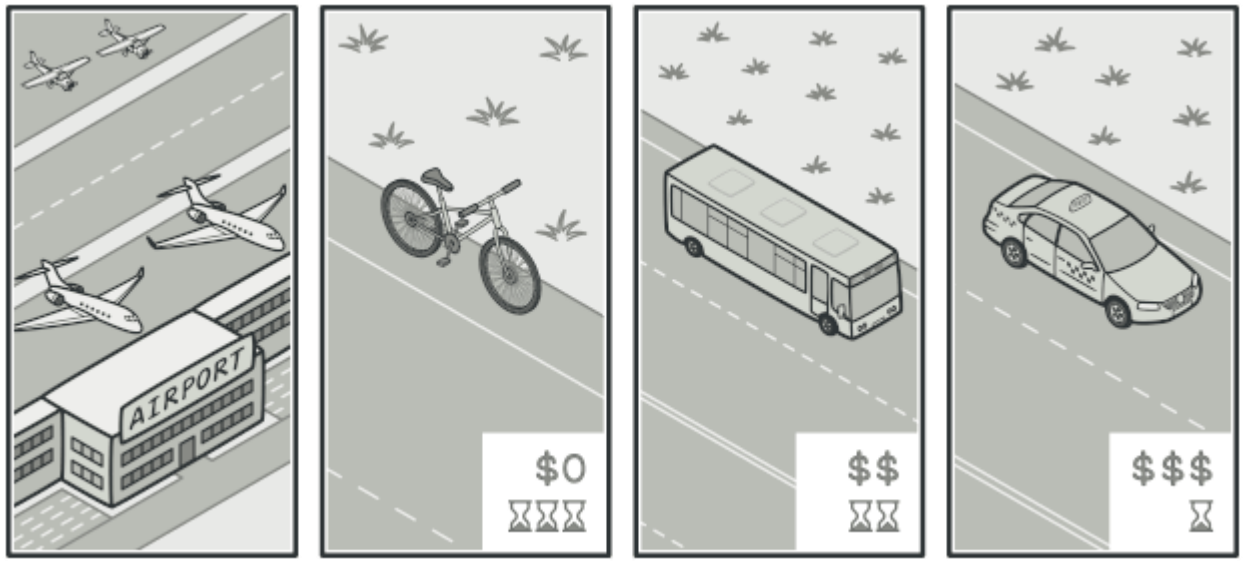
Pattern Comportamentali – Il pattern Strategy – Soluzione



- Nella nostra app di navigazione, ogni algoritmo di routing può essere estratto nella propria classe con un unico metodo `buildRoute`.
- Il metodo accetta un'origine e una destinazione e restituisce una raccolta dei checkpoint del percorso.
- Anche se con gli stessi argomenti, ogni classe di routing può costruire una rotta diversa.
- Alla classe principale del navigatore non importa quale algoritmo sia selezionato, poiché il suo compito principale è quello di restituire un insieme di checkpoint sulla mappa.

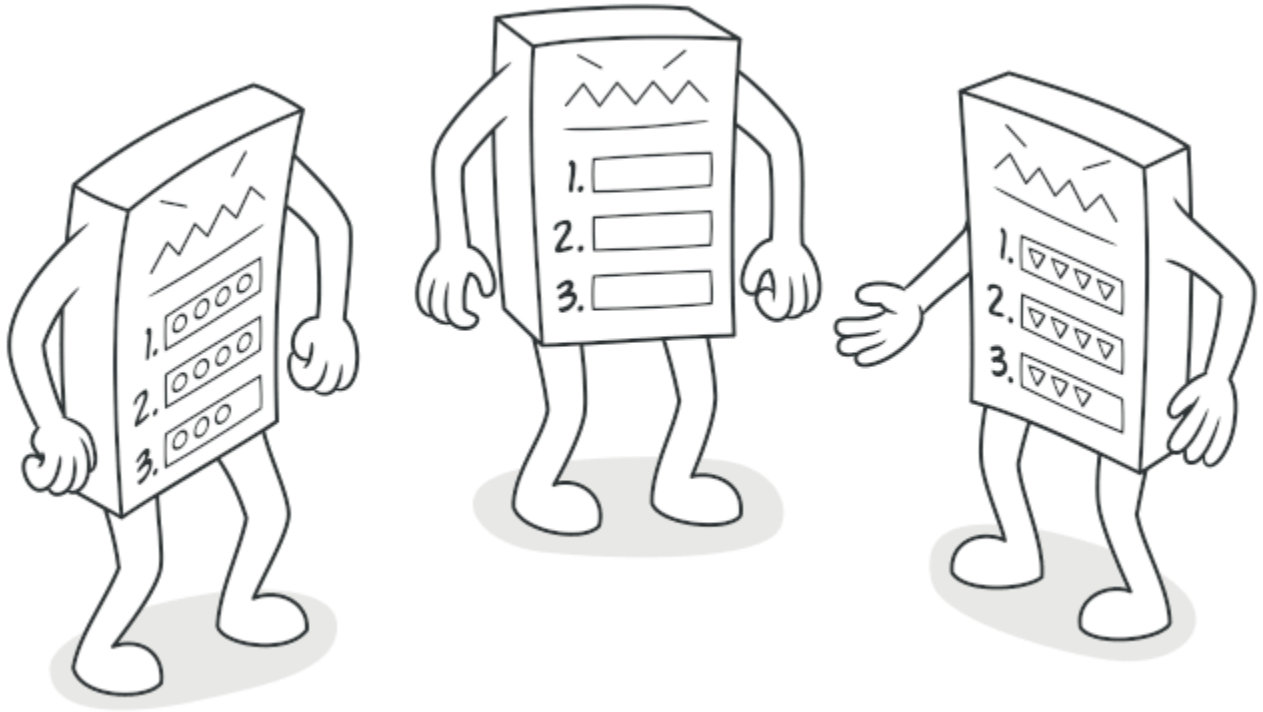
Pattern Comportamentali – Il pattern Strategy – Soluzione

- La classe ha un metodo per cambiare la strategia di routing attiva, così i suoi client, ad esempio i pulsanti nell'interfaccia utente, possono sostituire il comportamento di routing attualmente selezionato con un altro.
- Consideriamo un'analogia derivata dal mondo reale.
- Immaginiamo di dover andare all'aeroporto.
- Possiamo prendere un autobus, ordinare un taxi o salire in bicicletta. Queste sono le nostre strategie di trasporto.
- Possiamo scegliere una di queste strategie a seconda di fattori quali il budget o i vincoli di tempo.



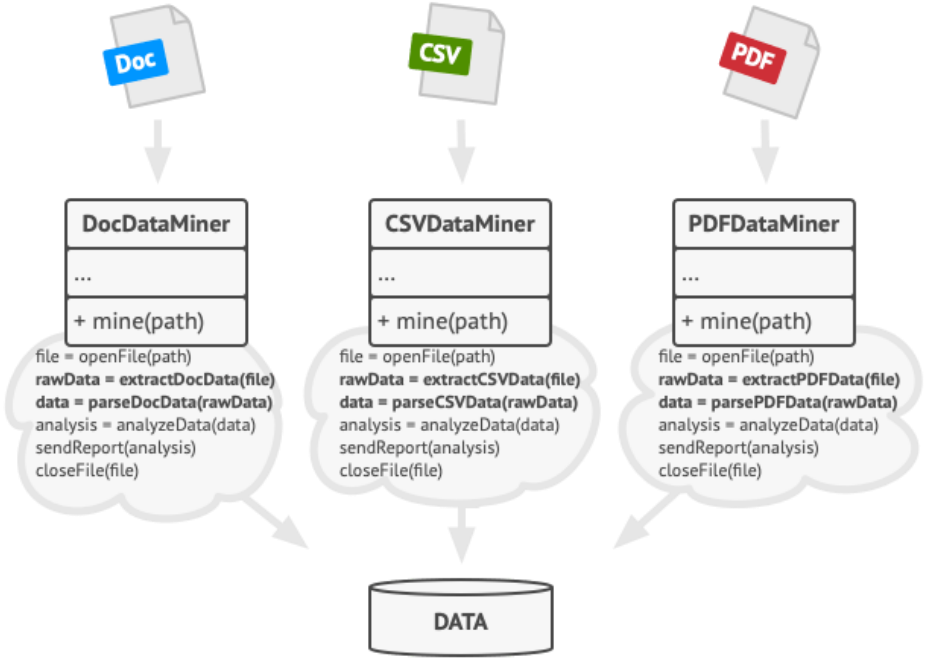
Pattern Comportamentali – Il pattern Template Method – Intent

- Template Method è un design pattern comportamentale che definisce lo scheletro di un algoritmo nella superclasse ma lascia che le sottoclassi sovrascrivano specifici passi dell'algoritmo senza modificarne la struttura.



Pattern Comportamentali – Il pattern Template Method – Problema

- Immaginiamo di creare un'applicazione di data mining che analizza i documenti aziendali.
- Gli utenti alimentano i documenti dell'applicazione in vari formati (PDF, DOC, CSV) e cercano di estrarre dati significativi da questi documenti in un formato uniforme.
- La prima versione dell'app potrebbe funzionare solo con i file DOC. Nella versione successiva siamo stati in grado di supportare i file CSV. Un mese dopo, gli abbiamo "insegnato" ad estrarre i dati dai file PDF.



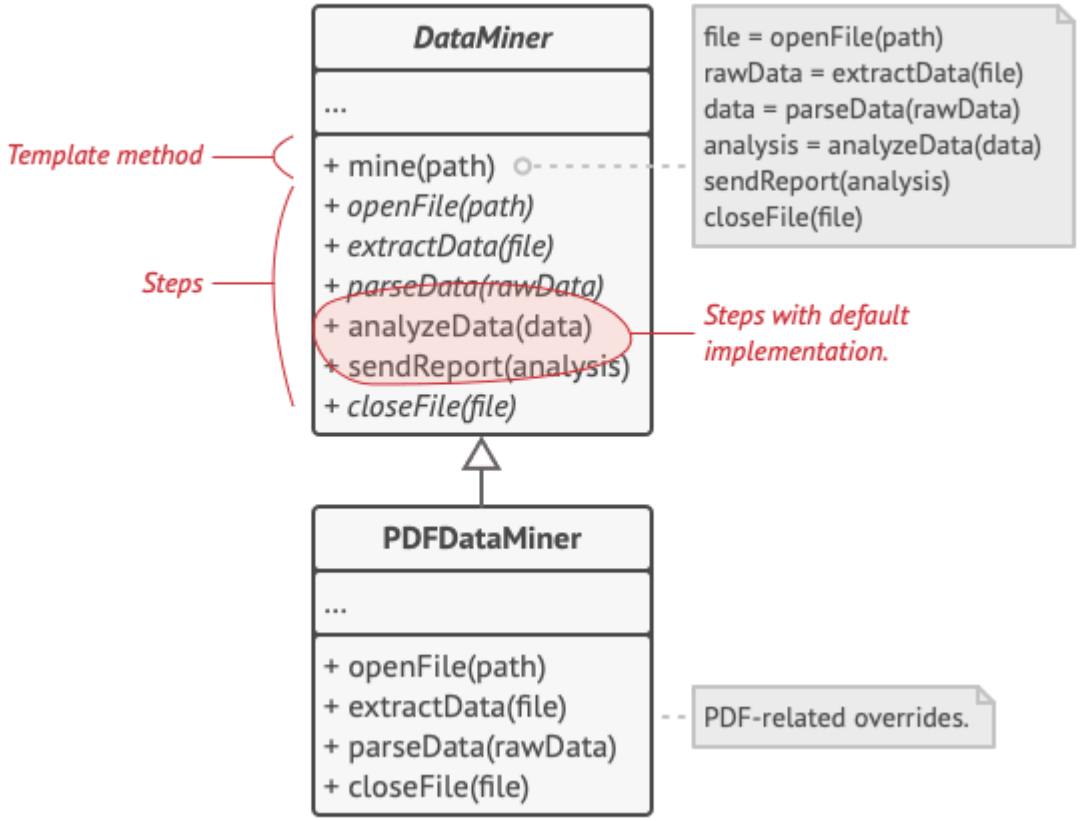
Pattern Comportamentali – Il pattern Template Method – Problema

- Ad un certo punto, abbiamo notato che tutte e tre le classi hanno molto codice simile.
- Mentre il codice per trattare i vari formati di dati era completamente diverso in tutte le classi, il codice per l'elaborazione e l'analisi dei dati è quasi identico.
- Non sarebbe bello eliminare la duplicazione del codice, lasciando intatta la struttura dell'algoritmo?
- C'era un altro problema legato al codice client che utilizzava queste classi.
- Aveva molte istruzioni condizionali che sceglievano l'azione appropriata a seconda della classe dell'oggetto di elaborazione.
- Se tutte e tre le classi di elaborazione avessero un'interfaccia comune o una classe di base, sarebbe possibile eliminare le istruzioni condizionali nel codice client e usare il polimorfismo quando si chiamano i metodi su un oggetto di elaborazione.

Pattern Comportamentali – Il pattern Template Method – Soluzione

- Il pattern Template Method suggerisce di suddividere un algoritmo in una serie di passi, trasformare questi passi in metodi e mettere una serie di chiamate a questi metodi all'interno di un singolo «template method».
- I passi possono essere astratti, o avere un'implementazione predefinita.
- Per utilizzare l'algoritmo, si suppone che il client debba fornire la propria sottoclasse, implementare tutti i passi astratti, e sovrascrivere alcuni di quelli opzionali se necessario (ma non il template method stesso).
- Vediamo come ciò funziona nella nostra applicazione di data mining.
- Possiamo creare una classe base per tutti e tre gli algoritmi di analisi.
- Questa classe definisce un template method che consiste in una serie di chiamate a varie fasi di elaborazione dei documenti.
- In un primo momento, possiamo dichiarare tutti i passi astratti, costringendo le sottoclassi a fornire le proprie implementazioni per questi metodi.
- Nel nostro caso, le sottoclassi hanno già tutte le implementazioni necessarie, quindi l'unica cosa da fare è adattare le signature dei metodi in modo che corrispondano ai metodi della superclasse.

Pattern Comportamentali – Il pattern Template Method – Soluzione



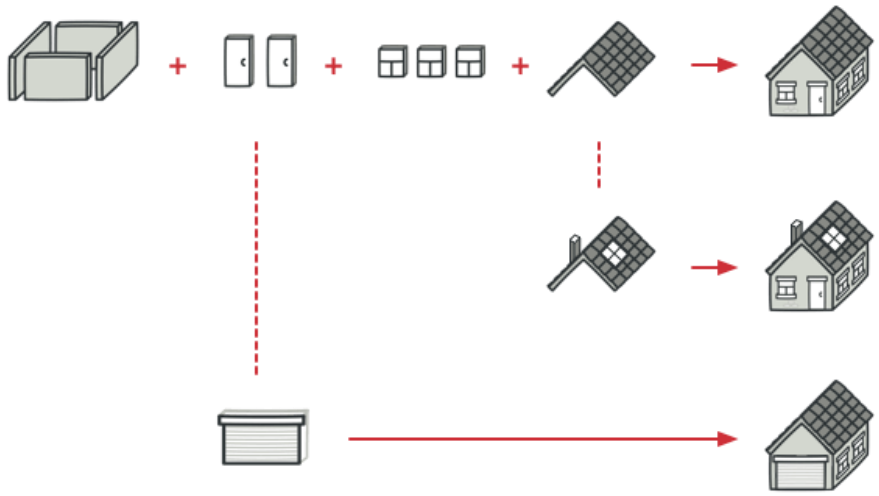
- Ora vediamo cosa possiamo fare per eliminare il codice duplicato.
- Sembra che il codice per aprire/chiudere i file ed estrarre/parserizzare i dati sia diverso per vari formati di dati, quindi è inutile toccare questi metodi.

Pattern Comportamentali – Il pattern Template Method – Soluzione

- Tuttavia, l'implementazione di altri passaggi, come l'analisi dei dati grezzi e la composizione dei report, è molto simile, quindi può essere portato su in una classe base, in modo tale che le sottoclassi possono condividere quel codice.
- Come possiamo vedere, abbiamo due tipi di passi:
  - i *passi astratti* devono essere implementati da ogni sottoclasse;
  - i *passi opzionali* hanno già qualche implementazione di default, ma possono ancora essere sovrascritti se necessario.
- C'è un altro tipo di passo, chiamato hook.
- Un hook è un passo opzionale con un corpo vuoto.
- Un template method funzionerebbe anche se un hook non venisse sovrascritto.
- Di solito, gli hook sono posizionati prima e dopo i passi cruciali degli algoritmi, fornendo sottoclassi con punti di estensione aggiuntivi per un algoritmo.

Pattern Comportamentali – Il pattern Template Method – Soluzione

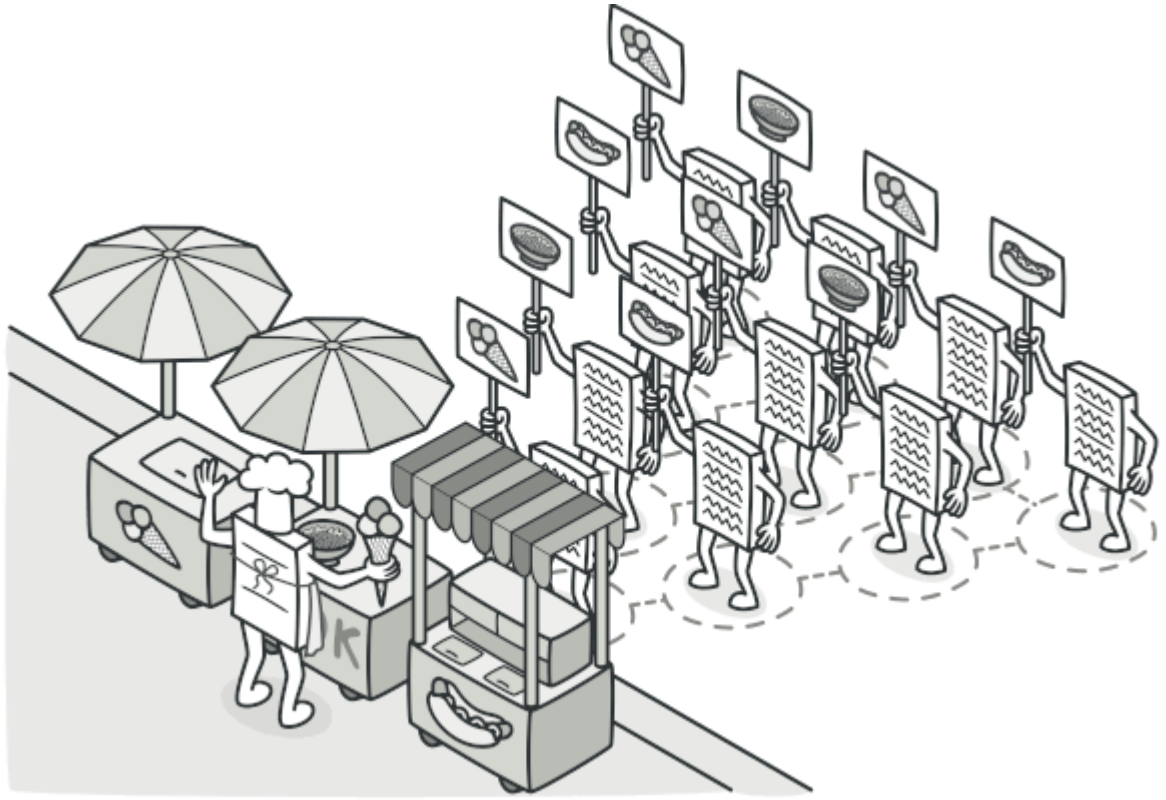
- Consideriamo un'analogia presa dal mondo reale.



- L'approccio del template method può essere utilizzato nella costruzione di alloggi di massa.
- Il piano architettonico per la costruzione di una casa standard può contenere diversi punti di estensione che permetterebbero a un potenziale proprietario di definire alcuni dettagli della casa risultante.
- Ogni fase della costruzione, come la posa delle fondamenta, la definizione dei perimetri, la costruzione dei muri, l'installazione di impianti idraulici e di cavi per l'acqua e l'elettricità, ecc. può essere leggermente modificata per rendere la casa risultante un po' diversa dalle altre.

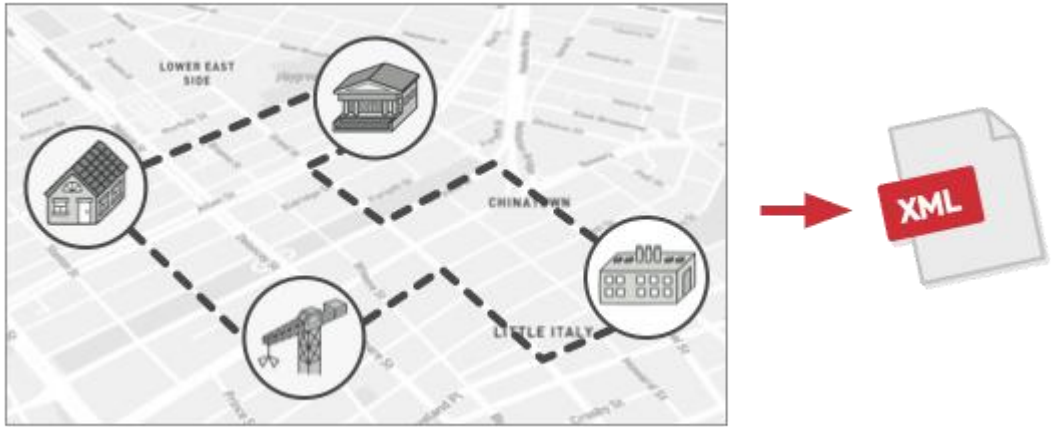
Pattern Comportamentali – Il pattern Visitor – Intent

- Visitor è un design pattern comportamentale che consente di separare gli algoritmi dagli oggetti su cui operano.



Pattern Comportamentali – Il pattern Visitor – Problema

- Immaginiamo che il nostro team sviluppi un'app che funzioni con informazioni geografiche strutturate come un unico colossale grafo.
- Ogni nodo del grafo può rappresentare un'entità complessa come una città, ma anche cose più granulari come industrie, aree turistiche, ecc.
- I nodi sono collegati con gli altri se c'è una strada tra gli oggetti reali che rappresentano.
- In backend, ogni tipo di nodo è rappresentato da una propria classe, mentre ogni specifico nodo è un oggetto.
- Ad un certo punto, abbiamo la necessità di esportare il grafo in formato XML. All'inizio il lavoro sembra piuttosto semplice.

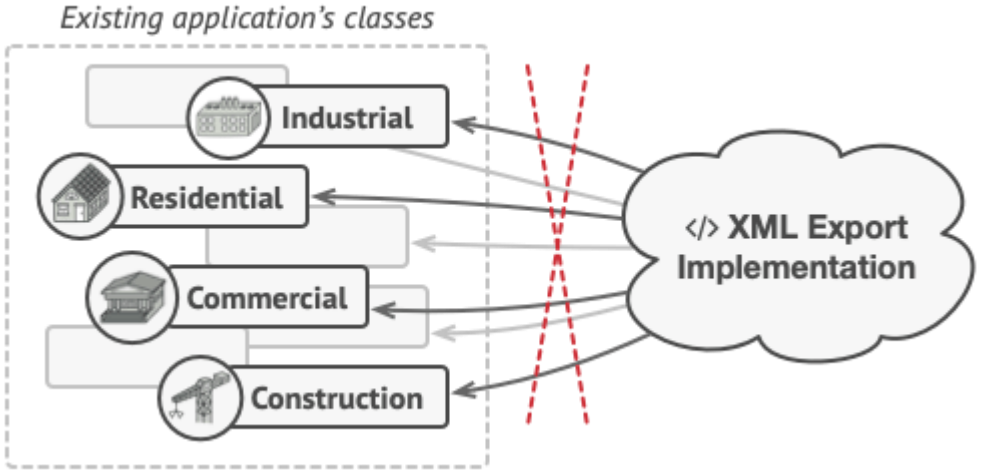


Pattern Comportamentali – Il pattern Visitor – Problema

- Si pensa di aggiungere un metodo di esportazione ad ogni classe nodo e poi sfruttare la ricorsione per andare in ogni nodo del grafo, eseguendo il metodo di esportazione.

- La soluzione è semplice ed elegante: grazie al polimorfismo, non si è accoppiato il codice che chiamava il metodo di esportazione a classi concrete di nodi.

- Purtroppo, l'architetto del sistema si è rifiutato di permettere di modificare le classi di nodi esistenti. Ha detto che il codice era già in produzione e non voleva rischiare di rovinarlo a causa di un potenziale bug nelle nostre modifiche.



Pattern Comportamentali – Il pattern Visitor – Problema

- Inoltre, ci ha chiesto che senso ha avere il codice di esportazione XML all'interno delle classi dei nodi.
- Il compito principale di queste classi è quello di lavorare con i geodati.
- Il comportamento di esportazione XML sembrerebbe alieno.
- C'era un'altra ragione per il rifiuto.
- È molto probabile che dopo l'implementazione di questa funzione, qualcuno del reparto marketing ci chieda di fornire la possibilità di esportare in un formato diverso, o di richiedere altre cose strane.
- Questo ci avrebbe costretto a cambiare di nuovo quelle classi preziose e fragili.

Pattern Comportamentali – Il pattern Visitor – Soluzione

- Il pattern Visitor suggerisce di collocare il nuovo comportamento in una classe separata chiamata *visitor*, invece di cercare di integrarlo nelle classi esistenti.

- L'oggetto originale che doveva eseguire il comportamento viene ora passato a uno dei metodi del visitor come argomento, fornendo al metodo l'accesso a tutti i dati necessari contenuti all'interno dell'oggetto.

- Ora, cosa succede se questo comportamento può essere eseguito su oggetti di classi diverse?

- Per esempio, nel nostro caso con l'esportazione XML, l'implementazione effettiva sarà probabilmente un po' diversa tra le varie classi di nodi.

- Così, la classe visitor può definire non uno, ma un insieme di metodi, ognuno dei quali potrebbe ricevere argomenti di tipo diverso, come nel seguente codice:

```
class ExportVisitor implements Visitor is
    method doForCity(City c) { ... }
    method doForIndustry(Industry f) { ... }
    method doForSightSeeing(SightSeeing ss) { ... }
    // ...
```

Pattern Comportamentali – Il pattern Visitor – Soluzione

- Ma come chiameremmo esattamente questi metodi, soprattutto quando si tratta dell'intero grafo?
- Questi metodi hanno signature diverse, quindi non possiamo usare il polimorfismo.
- Per scegliere un vero e proprio metodo visitatore che sia in grado di elaborare un dato oggetto, dovremmo controllare la sua classe.
- Tutto ciò diventerebbe un incubo.

```
foreach (Node node in graph)
    if (node instanceof City)
        exportVisitor.doForCity((City) node)
    if (node instanceof Industry)
        exportVisitor.doForIndustry((Industry) node)
    // ...
}
```

- Potremmo chiederci: perché non usiamo l'overloading dei metodi? Potremmo dare a tutti i metodi lo stesso nome, anche se supportano diversi insiemi di parametri.

Pattern Comportamentali – Il pattern Visitor – Soluzione

- Purtroppo, anche supponendo che il nostro linguaggio di programmazione lo supporti (come fanno Java e C#), il polimorfismo non è in grado di aiutarci.
- Poiché l'esatta classe di un oggetto nodo è sconosciuta in anticipo, il meccanismo di overloading non sarà in grado di determinare il metodo corretto da eseguire.
- Sarà eseguito il metodo di default che riceve un oggetto della classe base `Node`.
- Il pattern visitor affronta questo problema.
- Esso utilizza una tecnica chiamata Double Dispatch, che aiuta ad eseguire il metodo corretto su un oggetto senza istruzioni condizionali ingombranti.
- Invece di lasciare che il client selezioni una versione corretta del metodo da chiamare, viene delegata questa scelta agli oggetti che stiamo passando al visitor come argomento.
- Dato che gli oggetti conoscono le proprie classi, saranno in grado di scegliere un metodo corretto sul visitor in modo meno ingombrante rispetto alla pesante istruzione condizionale.

Pattern Comportamentali – Il pattern Visitor – Soluzione

- Essi "accettano" un visitor e gli dicono quale metodo di visita deve essere eseguito.

```
// Client code
foreach (Node node in graph)
    node.accept(exportVisitor)

// City
class City is
    method accept(Visitor v) is
        v.doForCity(this)
    // ...

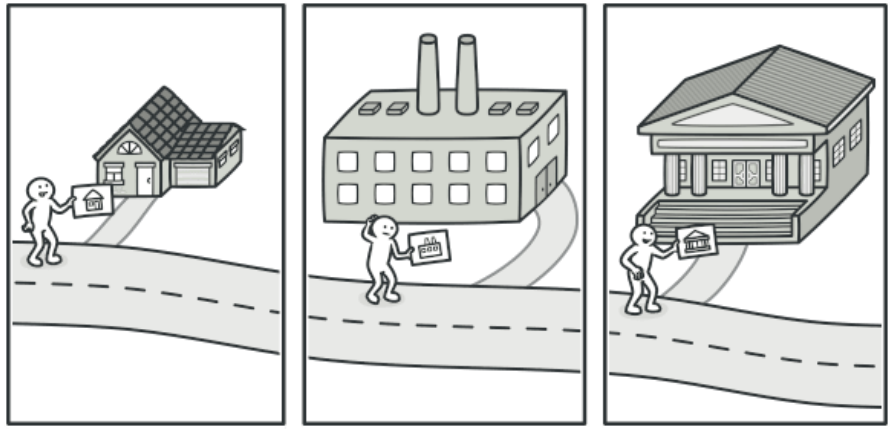
// Industry
class Industry is
    method accept(Visitor v) is
        v.doForIndustry(this)
    // ...
```

- Abbiamo dovuto cambiare le classi dei nodi, dopo tutto.

- Ma almeno il cambiamento è banale e ci permette di aggiungere altri comportamenti senza alterare il codice ancora una volta.

Pattern Comportamentali – Il pattern Visitor – Soluzione

- Ora, se estraiamo un'interfaccia comune per tutti i visitor, tutti i nodi esistenti possono lavorare con qualsiasi visitor che introduciamo nell'app.
- Se ci troviamo ad introdurre un nuovo comportamento relativo ai nodi, tutto quello che dobbiamo fare è implementare una nuova classe di visitor.
- Consideriamo un’analogia presa dal mondo reale.
- Immaginiamo un agente assicurativo esperto, desideroso di acquisire nuovi clienti.
- Può visitare ogni edificio di un quartiere, cercando di vendere assicurazioni a tutti quelli che incontra.



Pattern Comportamentali – Il pattern Visitor – Soluzione

- A seconda del tipo di organizzazione che occupa l'edificio, può offrire polizze assicurative specializzate.
  - Se si tratta di un edificio residenziale, vende assicurazioni mediche.
  - Se si tratta di una banca, vende assicurazioni contro il furto.
  - Se si tratta di una caffetteria, vende assicurazioni contro gli incendi e le inondazioni.